



Εισαγωγή στην Παράλληλη Επεξεργασία

Αθανάσιος Ι. Μάργαρης

ΚΕΦΑΛΑΙΟ 2 Ζητήματα Παράλληλου Προγραμματισμού



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Η διαδικασία **σχεδίασης** και **υλοποίησης** μιας παράλληλης εφαρμογής **δεν αποτελεί** μία **εύκολη** και **τετριμμένη** διαδικασία.

Για τον λόγο αυτό συνίσταται η χρήση μιας **μεθοδικής προσέγγισης**, η σωστή εφαρμογή της οποίας διασφαλίζει πως η παράλληλη εφαρμογή μας χαρακτηρίζεται από **συγκεκριμένες** επιθυμητές ιδιότητες.

Το πρώτο ερώτημα στο οποίο θα πρέπει να απαντήσουμε, είναι το εξής:

Επιδέχεται παραλληλοποίηση το πρόβλημα προς μελέτη?

Όπως σχολιάζει χαρακτηριστικά ο **Wernher von Braun**

Το γεγονός πως η εγκυμοσύνη σε μία γυναίκα διαρκεί εννέα μήνες, δεν σημαίνει πως εάν μείνουν ταυτόχρονα έγκυες εννέα γυναίκες, το μωρό θα γεννηθεί σε έναν μήνα !!!

Υπάρχουν **τρεις κατηγορίες** προβλημάτων:

- Προβλήματα που **δεν παραλληλοποιούνται** (συνήθως αλγόριθμοι αναδρομικής φύσεως)
- Προβλήματα που **παραλληλοποιούνται σε κάποιο βαθμό** και κατά συνέπεια χαρακτηρίζονται από την παρουσία ενός σειριακού και ενός παράλληλου τμήματος υπολογισμών
- Προβλήματα που παραλληλοποιούνται **πλήρως**



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Η διαδικασία υπολογισμού των **διαδοχικών όρων** της ακολουθίας **Fibonacci** που στηρίζεται στην αναδρομική έκφραση

$$F(n+2) = F(n+1) + F(n)$$

δεν μπορεί να παραλληλοποιηθεί λόγω της **εξάρτησης** που υφίσταται ανάμεσα στους **διαδοχικούς όρους** της ακολουθίας

Αντίθετα, η διαδικασία **αριθμητικού υπολογισμού** του **ολοκληρώματος** μιας συνάρτησης με την **μέθοδο των τραπεζίων** παραλληλοποιείται **πλήρως**, αφού το εμβαδόν του κάθε τραπεζίου, **δεν εξαρτάται** από την τιμή κάποιου άλλου εμβαδού.

Επομένως, οι τιμές όλων των εμβαδών μπορούν να υπολογιστούν **παράλληλα** από **διαφορετικές** διεργασίες ή νήματα.

Εφόσον καταλήξουμε στο συμπέρασμα πως η εφαρμογή μας **μπορεί να παραλληλοποιηθεί**, θα πρέπει να **αναλύσουμε** το πρόβλημα, εντοπίζοντας τα τμήματα του προβλήματος στα οποία θα εφαρμόσουμε την **διαδικασία της παραλληλοποίησης**.

Αυτή η διαδικασία της **παραλληλοποίησης** η οποία περιλαμβάνει μεταξύ άλλων και τον εντοπισμό των **εξαρτήσεων** που υφίστανται ανάμεσα στις δηλώσεις του προγράμματος, σε ορισμένες περιπτώσεις μπορεί να γίνει και από τον ίδιο τον **μεταγλωττιστή** τον οποίο εφοδιάζουμε με τις κατάλληλες οδηγίες.



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Οι βασικές οντότητες μιας παράλληλης εφαρμογής

Μιλώντας γενικά, ένας **παράλληλος υπολογισμός** υλοποιείται μέσω της **ταυτόχρονης εκτέλεσης** μιας ή περισσότερων διεργασιών, το πλήθος των οποίων μπορεί να **μεταβάλλεται δυναμικά** κατά την πραγματοποίηση των υπολογισμών.

Η κάθε διεργασία χρησιμοποιεί **την τοπική μνήμη του κόμβου στον οποίο εκτελείται**, έχοντας όμως τη δυνατότητα να **επικοινωνήσει** με τις διεργασίες των υπόλοιπων κόμβων, δια της **αποστολής** και **λήψης** των κατάλληλων μηνυμάτων.

Σε ορισμένες περιπτώσεις η κάθε διεργασία μπορεί να **δημιουργήσει** και να **τερματίσει** θυγατρικές διεργασίες.

Η **ανταλλαγή δεδομένων** μεταξύ των διεργασιών, στηρίζεται στην χρήση **καναλιών επικοινωνίας**, για τα οποία υφίσταται η δυνατότητα της **δυναμικής** δέσμευσης και αποδέσμευσής τους, ενώ η **ανάγνωση** δεδομένων από αυτά τα κανάλια και η **αποστολή** δεδομένων σε αυτά, πραγματοποιείται μέσα από τις κατάλληλες **θύρες επικοινωνίας**.

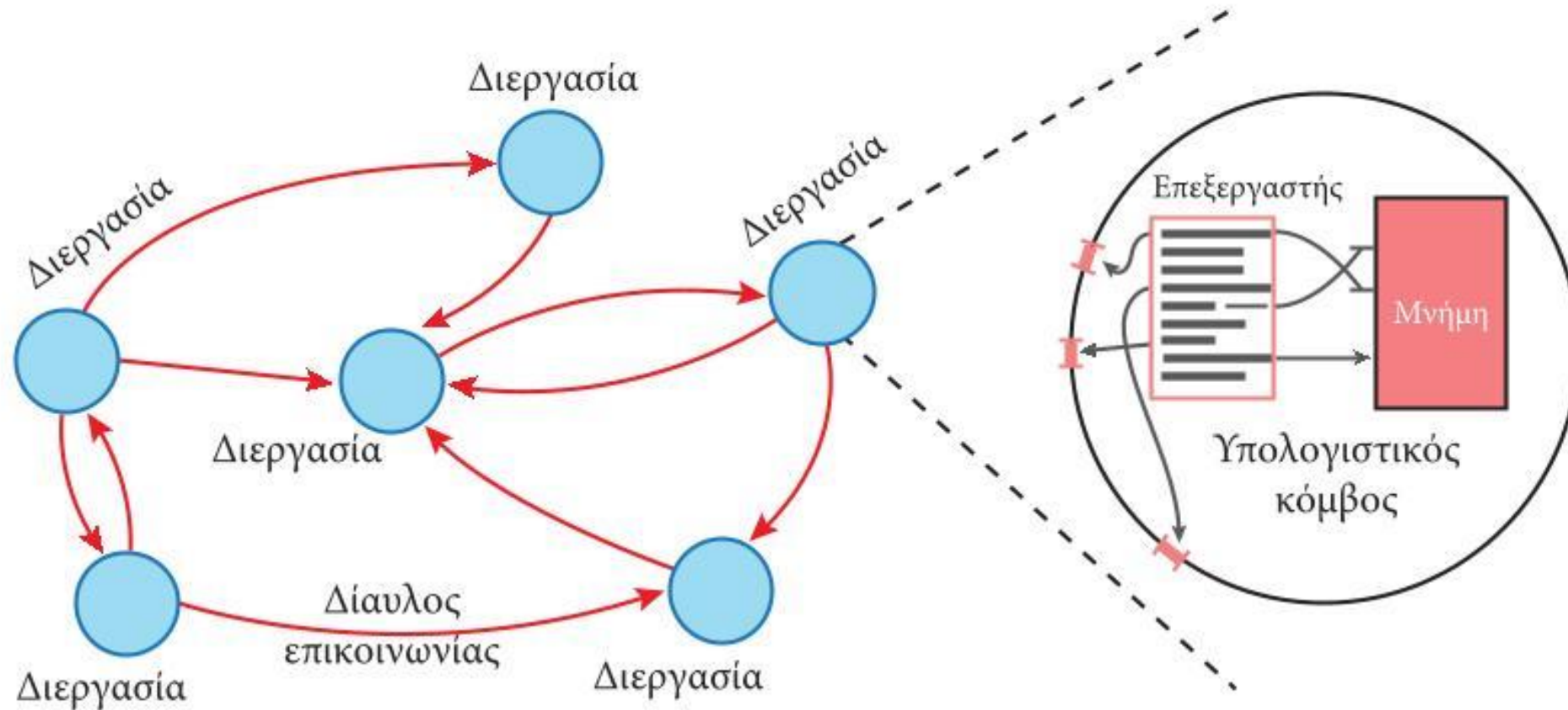
Η **κατανομή** των διεργασιών στους πυρήνες του συστήματος, μπορεί να γίνει **με πολλούς τρόπους** έτσι ώστε, οι διάφορες δυνατότητες που υφίστανται όσον αφορά σε αυτή την κατανομή, **να μην επηρεάζουν** τη σημασιολογία του προγράμματος.

Τα **αποτελέσματα** που προκύπτουν από την εκτέλεση της εφαρμογής, **δεν θα πρέπει** να εξαρτώνται από την **κατανομή** των διεργασιών στους κόμβους, από τον **κόμβο** στον οποίο εκτελείται η κάθε διεργασία, καθώς και από το **πλήθος** των κόμβων.



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Οι βασικές οντότητες μιας παράλληλης εφαρμογής

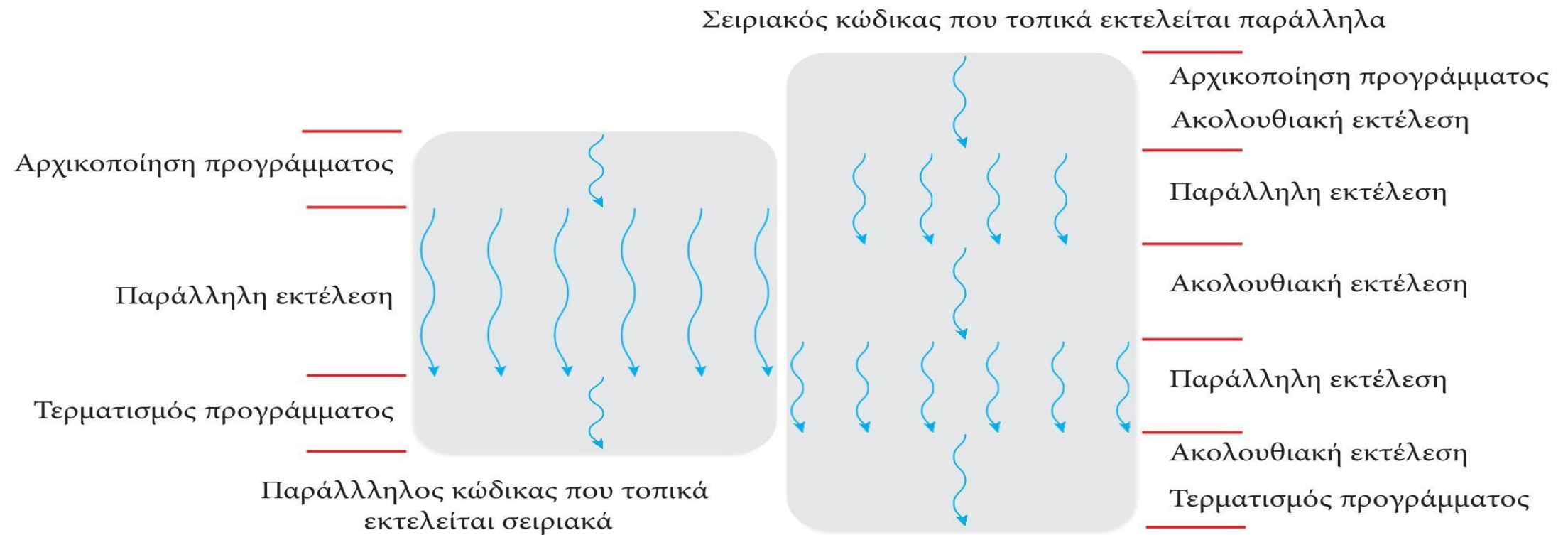


Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Οι δύο τύποι παράλληλων εφαρμογών

Ο κώδικας μιας εφαρμογής διαθέτει τόσο **παράλληλη** όσο και **σειριακή** συνιστώσα και μπορεί να ανήκει σε δύο τύπους:

- **Παράλληλος** κώδικας, ο οποίος τοπικά εκτελείται **σειριακά** (Globally Parallel, Locally Sequential, **GPLS**)
- **Σειριακός** κώδικας ο οποίος τοπικά εκτελείται **παράλληλα** (Globally Sequential, Locally Parallel, **GSLP**)



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Οι ιδιότητες των παράλληλων εφαρμογών

Ταυτοχρονισμός: Η παράλληλη εφαρμογή έχει την δυνατότητα να εκτελεί **περισσότερες από μία** εντολές του προγράμματος σε **ένα και μόνο** βήμα.

Κλιμακωσιμότητα ή **επεκτασιμότητα:** Αφορά στον **τρόπο** με τον οποίο θα **μεταβληθεί** η τιμή της **επιτάχυνσης** του συστήματος, ως συνέπεια της **μεταβολής του πλήθους** των επεξεργαστών.

Τοπικότητα: Τα δεδομένα του τοπικού κόμβου προσπελαύνονται **με πολύ μεγαλύτερη συχνότητα** από εκείνη που χαρακτηρίζει την προσπέλαση των δεδομένων που βρίσκονται αποθηκευμένα στους απομακρυσμένους κόμβους.

Απόδοση: η απόδοση μιας παράλληλης εφαρμογής στο σύνολό της, σχετίζεται άμεσα με τον **τρόπο της απεικόνισης** των διεργασιών του συστήματος στους **επεξεργαστές** που περιλαμβάνονται σε αυτό.

Ανεξαρτησία από την απεικόνιση: Τα αποτελέσματα που θα λάβουμε από την εκτέλεση της εφαρμογής, **δεν εξαρτώνται** από την τρέχουσα απεικόνιση, δηλαδή από τα **στοιχεία επεξεργασίας** στα οποία εκτελούνται οι διεργασίες της εφαρμογής.

Τμηματοποίηση: η εφαρμογή αναπτύσσεται σε **ανεξάρτητα τμήματα**, τα οποία τελικά θα **συνδυαστούν** μεταξύ τους προκειμένου να υλοποιήσουν την εφαρμογή στο σύνολό της.

Αιτιοκρατία ή **ντετερμινισμός:** η **επαναληπτική** εκτέλεση της παράλληλης εφαρμογής με την **ίδια είσοδο**, θα πρέπει να οδηγεί πάντοτε στην **ίδια έξοδο**.



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Η μεθοδική προσέγγιση PCAM

Στην **γενική περίπτωση**, η σχεδίαση μιας **παράλληλης** εφαρμογής, περιλαμβάνει

- τον προσδιορισμό των **τμημάτων** του κώδικα που **δεν μπορούν** να διασπαστούν σε μικρότερα κομμάτια και των οποίων οι **αλληλεπιδράσεις**, εφόσον υφίστανται, περιγράφονται με τη βοήθεια **γράφων εξάρτησης**.
- την ομαδοποίηση των εργασιών σε **διεργασίες** ή **νήματα**, ανάλογα με το χρησιμοποιούμενο **μοντέλο προγραμματισμού**
- τον **χρονοπρογραμματισμό** των **διεργασιών** ή των **νημάτων** (προσδιορισμός **πυρήνα** και **χρονικής στιγμής εκτέλεσης**)

Τα παραπάνω τρία στάδια συνοδεύονται και από άλλες διαδικασίες, όπως είναι η **δέσμευση υπολογιστικών** και **αποθηκευτικών πόρων** και ο χειρισμός των **συνθηκών ανταγωνισμού** που σχετίζονται με την χρήση τους.

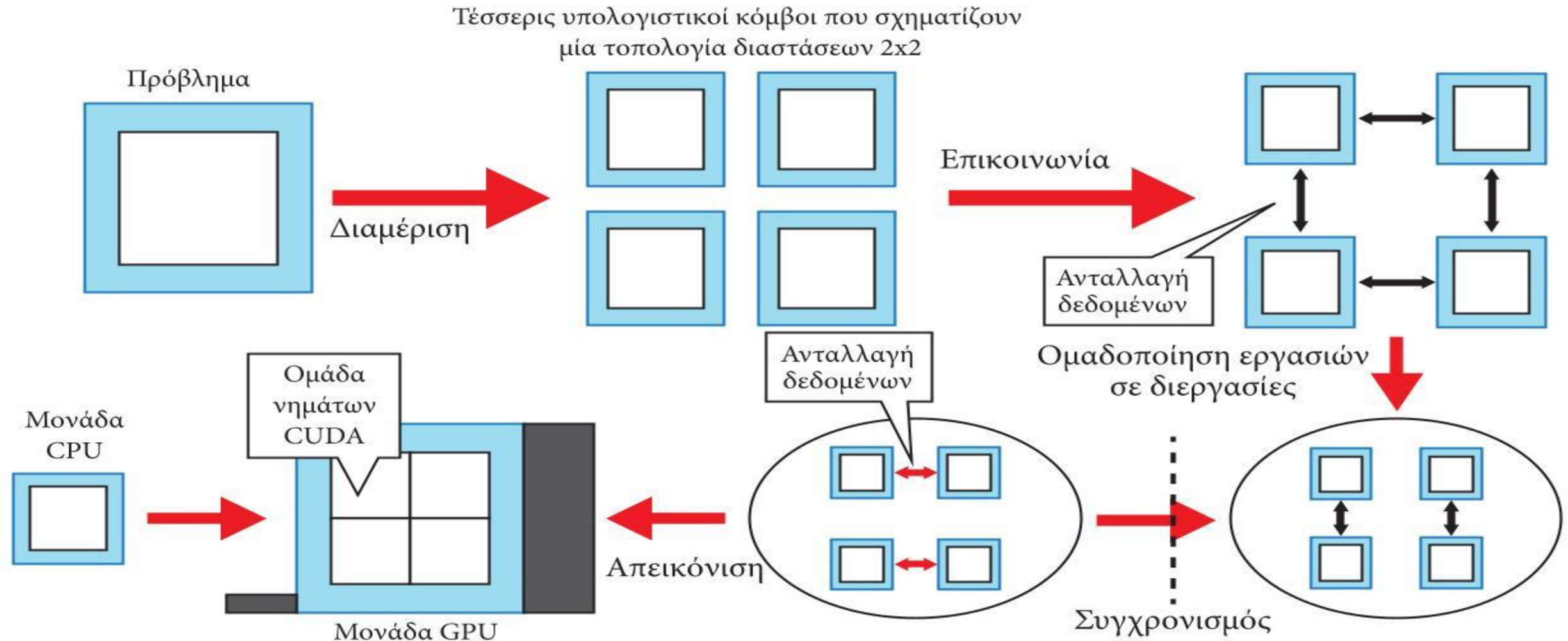
Η **μεθοδική προσέγγιση PCAM** του **Ian Foster** οδηγεί σε εφαρμογές που υποστηρίζουν τα χαρακτηριστικά του **ταυτοχρονισμού**, της **τοπικότητας** και της **κλιμάκωσης**. Αυτή η προσέγγιση αποτελείται από τα επόμενα τέσσερα στάδια:

- **Partitioning** (Διαμέριση)
- **Communication** (Επικοινωνία)
- **Agglomeration** (Ομαδοποίηση)
- **Mapping** (Απεικόνιση)



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Η μεθοδική προσέγγιση PCAM



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Η μεθοδική προσέγγιση PCAM - Διαμέριση

Το πρόβλημα **διαχωρίζεται** σε **μικρά τμήματα** τα οποία κατανέμονται προς επεξεργασία στις διεργασίες της εφαρμογής.

Ο διαχωρισμός εφαρμόζεται στα **δεδομένα** ή / και στους **υπολογισμούς** που εφαρμόζονται επί αυτών, με την διαμέριση να **μην είναι** οριστική, αλλά να μπορεί να **αναθεωρηθεί** στο μέλλον.

Ζητήματα που θίγονται σε αυτό το στάδιο, αφορούν

- στο **πλήθος των διεργασιών** που συνήθως είναι **μία τάξη μεγέθους** μεγαλύτερο από το πλήθος των επεξεργαστών
- στην **δυνατότητα υλοποίησης** του πλήθους των υπολογισμών που **πραγματικά** απαιτούνται
- στο **μέγεθος** των διεργασιών που σχετίζεται με το χαρακτηριστικό της **ισοκατανομής υπολογιστικού φόρτου**
- στην δυνατότητα **κλιμάκωσης** του πλήθους των διεργασιών με το **μέγεθος** του προβλήματος
- στην δυνατότητα ορισμού **πρόσθετων εναλλακτικών** διαμερίσεων.

Υπάρχουν **δύο τύποι** διαμερίσεων και πιο συγκεκριμένα:

Διαμέριση δεδομένων: αρχικά διαμερίζονται τα **δεδομένα** και στη συνέχεια οι **υπολογισμοί**

Διαμέριση λειτουργιών: αρχικά διαμερίζονται οι **υπολογισμοί** και στη συνέχεια τα **δεδομένα**



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Η μεθοδική προσέγγιση PCAM – Διαμέριση δεδομένων

Τα **δεδομένα** του προβλήματος διαχωρίζονται σε **μικρά τμήματα** που έχουν **προσεγγιστικά** το **ίδιο** μέγεθος

Στη συνέχεια προχωρούμε σε **διαμέριση** των **υπολογισμών**, συσχετίζοντας την **κάθε λειτουργία** με τα δεδομένα εκείνα πάνω στα οποία θα εφαρμοσθεί.

Αυτή η διαδικασία οδηγεί στην δημιουργία ενός πλήθους **διεργασιών** η καθεμία από τις οποίες χρησιμοποιηθεί το δικό της σύνολο δεδομένων, ενώ υφίσταται η δυνατότητα εφαρμογής ενός **υπολογισμού** σε δεδομένα που ανήκουν σε **διαφορετικές διεργασίες**, με κόστος την κατάλληλη σε κάθε περίπτωση **επικοινωνία δεδομένων** ανάμεσα στις διεργασίες του συστήματος.



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

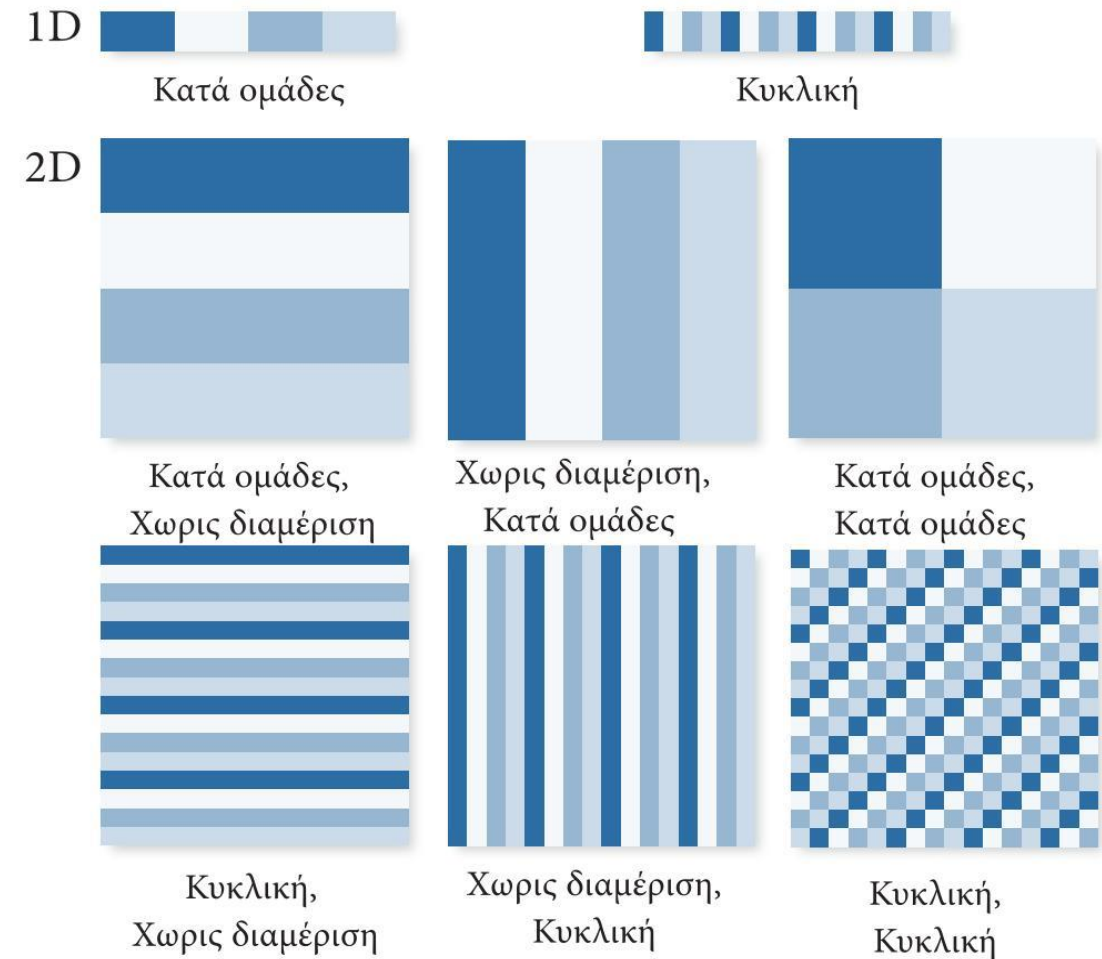
Η μεθοδική προσέγγιση PCAM – Διαμέριση δεδομένων

Εάν τα δεδομένα του προβλήματος είναι οργανωμένα σε **μονοδιάστατες**, **δισδιάστατες** ή **τρισδιάστατες** διατάξεις καταφεύγουμε σε διαμερίσεις **γεωμετρικού τύπου** όπως αυτές που παρουσιάζονται στο επισυναπτόμενο σχήμα.

Εάν τα δεδομένα του προβλήματος είναι οργανωμένα σε **αναδρομικές** δομές δεδομένων (λίστες, δέντρα, γράφους), καταφεύγουμε σε διαμερίσεις **αναδρομικού τύπου**.

Εδώ, η δομή δεδομένων **υποδιαιρείται** σε **επιμέρους** στοιχεία και στη συνέχεια, το **κάθε** στοιχείο ή ομάδα στοιχείων, εκχωρείται για επεξεργασία σε κάποια **διεργασία** ή **νήμα**.

Η τεχνική που ακολουθείται παραπέμπει στον **δυναμικό προγραμματισμό**, όπου τα προβλήματα **μεγάλου** μεγέθους επιλύονται χρησιμοποιώντας **τις λύσεις προβλημάτων μικρότερου μεγέθους**, οι οποίες έχουν υπολογιστεί και αποθηκευτεί σε κάποιο προηγούμενο βήμα.



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Η μεθοδική προσέγγιση PCAM – Διαμέριση υπολογισμών

Η έμφαση δίδεται στους υπολογισμούς και στις διαδικασίες επίλυσης παρά στα δεδομένα επί των οποίων εφαρμόζονται.

Εάν διαμερίσουμε τους υπολογισμούς σε τμήματα ανεξάρτητα μεταξύ τους, μπορούμε να προχωρήσουμε σε διαμέριση δεδομένων, με την διαμέριση να είναι πλήρης εάν τα σύνολα των δεδομένων που θα προκύψουν είναι ξένα μεταξύ τους.

Αντίθετα, η μερική επικάλυψη των συνόλων δεδομένων, απαιτεί διαδικασίες επικοινωνίας, έτσι ώστε να μην υφίστανται πολλαπλά αντίγραφα των δεδομένων, για χρήση από τις διεργασίες του συστήματος.

Εάν γνωρίζουμε το πλήθος των απαιτούμενων εργασιών, η διαμέριση υπολογισμών χαρακτηρίζεται ως γραμμική, ενώ μπορούμε να προσδιορίσουμε τους ανεξάρτητους υπολογισμούς και να τους αναθέσουμε για παράλληλη εκτέλεση.

Εάν η εκ των προτέρων ανάθεση εργασιών δεν είναι εφικτή, ο καταμερισμός του υπολογιστικού έργου λαμβάνει χώρα κατά την φάση της εκτέλεσης της εφαρμογής, καταφεύγοντας σε αναδρομικές τεχνικές, π.χ. διαίρει και βασίλευε.

Ένας τρίτος τύπος διαμέρισης, σχετίζεται με την ροή των δεδομένων που χαρακτηρίζει την χρονική εξέλιξη της εφαρμογής.

Σε αυτόν τον τύπο διαμέρισης, το κριτήριο που υπαγορεύει τον τρόπο με τον οποίο αυτή θα πραγματοποιηθεί, είναι το αποτέλεσμα του χαρακτηρισμού της ροής της επικοινωνίας, ως σταθερής ή όχι.



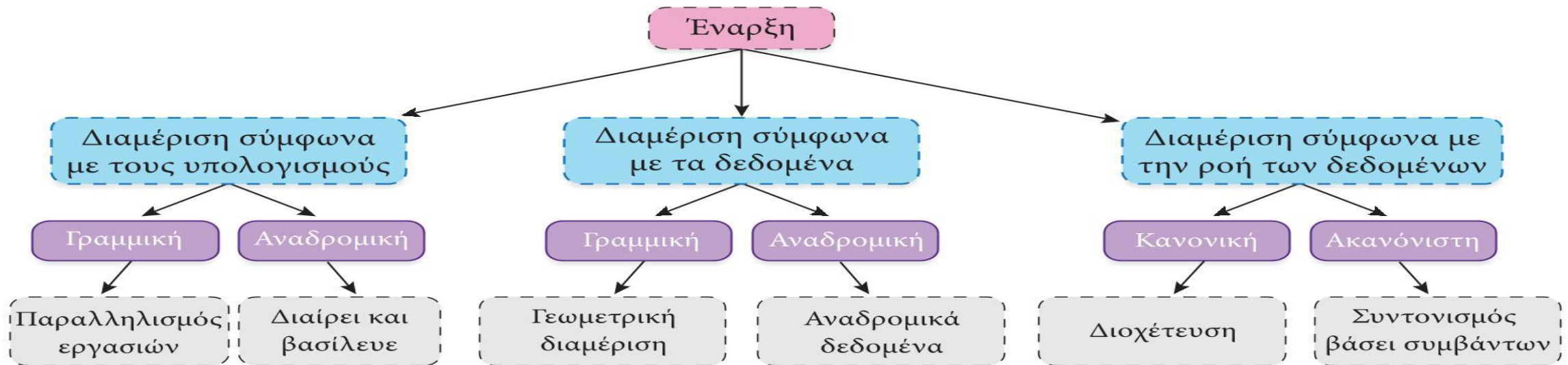
Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Η μεθοδική προσέγγιση PCAM – Διαμέριση υπολογισμών

Εάν η ροή της επικοινωνίας **είναι σταθερή** και η φύση της παράλληλης εφαρμογής **ευνοεί** την υλοποίηση των υπολογισμών **ως μία ακολουθία διαφορετικών φάσεων υπολογισμών** οι οποίες **επικοινωνούν** μεταξύ τους και εκτελούνται **ταυτόχρονα** χρησιμοποιώντας κάθε φορά διαφορετικά δεδομένα, τότε χρησιμοποιείται αυτός ο τύπος διαμέρισης.

Εάν η ροή της επικοινωνίας **δεν είναι σταθερή**, οι αποφάσεις σχετικά με την διαμέριση της ροής των δεδομένων, λαμβάνονται σε **πραγματικό χρόνο**, αφού το **είδος**, ο **τύπος**, η **συχνότητα** και η **διάρκεια** της επικοινωνίας, **δεν** είναι γνωστά εξ αρχής.

Αυτός ο τύπος διαμέρισης στηρίζεται στην χρήση των κατάλληλων σε κάθε περίπτωση **συμβάντων**.



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Η μεθοδική προσέγγιση PCAM – Επικοινωνία ανάμεσα στις διεργασίες

Στην γενική περίπτωση η **διαμέριση υπολογισμών** και **δεδομένων** απαιτεί την **επικοινωνία** των διεργασιών της εφαρμογής, αφού ο υπολογισμός από μία διεργασία μπορεί να απαιτεί **δεδομένα** που υπολογίζονται από **άλλες** διεργασίες.

Παράγοντες που εμπλέκονται σε αυτή την διαδικασία επικοινωνίας, σχετίζονται με ζητήματα όπως είναι

- η **ισοκατανομή του υπολογιστικού φόρτου** στους διαθέσιμους επεξεργαστές
- το **πλήθος των διεργασιών** της εφαρμογής με τις οποίες επικοινωνεί η κάθε διεργασία
- η δυνατότητα της **ταυτόχρονης εκτέλεσης** των διεργασιών
- το ενδεχόμενο του **συγχρονισμού μεταξύ των διεργασιών** που οδηγεί σε καθυστερήσεις
- το **κόστος της επικοινωνίας** που περιλαμβάνει επιπλέον το κόστος συσκευασίας και αποστολής δεδομένων

Αυτή η επικοινωνία των διεργασιών γίνεται με **άμεσο** ή με **έμμεσο** τρόπο και είναι **πιο εύκολη** σε περιπτώσεις **διαμέρισης υπολογισμών**, καθώς η ροή της πληροφορίας στην επικοινωνία **ταυτίζεται** με τη ροή των δεδομένων της εφαρμογής.

Σε προβλήματα που στηρίζονται στην **διαμέριση δεδομένων** και σε υπολογισμούς επί δεδομένων **διαφορετικών** διεργασιών ο καθορισμός των παραμέτρων της επικοινωνίας, θα πρέπει να γίνει με **εξαιρετικά μεγάλη** προσοχή.



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Η μεθοδική προσέγγιση PCAM – Επικοινωνία ανάμεσα στις διεργασίες

Μία διαδικασία επικοινωνίας μεταξύ διεργασιών μπορεί να χαρακτηριστεί με τους ακόλουθους τρόπους:

- **Τοπική** ή **καθολική**, ανάλογα, με το πλήθος των διεργασιών που συμμετέχουν σε αυτή. Στις **τοπικές** διαδικασίες η διεργασία επικοινωνεί **μόνο** με τις γειτονικές διεργασίες, ενώ στις **καθολικές** διαδικασίες, **επικοινωνεί** με όλες τις διεργασίες.
- **Σύγχρονη (παρεμποδιστική)** ή **ασύγχρονη (μη παρεμποδιστική)**, ανάλογα με το εάν λαμβάνει χώρα **συγχρονισμός** μεταξύ των διεργασιών ή **όχι**.
- **Δομημένη** ή **μη δομημένη**, ανάλογα με το εάν οι γειτονικές διεργασίες με τις οποίες η τρέχουσα διεργασία ανταλλάσσει δεδομένα χαρακτηρίζονται από κάποια **δομή** (π.χ. σχηματίζουν **πλέγμα** ή **δομή δέντρου**) ή **όχι**.
- **Στατική** ή **δυναμική** ανάλογα με το εάν το πλήθος των διεργασιών **μεταβάλλεται με την πάροδο του χρόνου** ή **όχι**.
- **Από σημείο σε σημείο** ή **συλλογική** ανάλογα με το εάν πραγματοποιείται **ανάμεσα σε δύο μόνο διεργασίες** ή **ανάμεσα σε όλες τις διεργασίες** μιας ομάδας διεργασιών.

Οι **μη δομημένες** επικοινωνίες ενδέχεται να καταστήσουν **αρκετά πολύπλοκα** τα **δύο** τελευταία στάδια της προσέγγισης PCAM, γεγονός που απαιτεί τη χρήση **εξελιγμένων αλγορίθμων** για την πραγματοποίησή τους.



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Η μεθοδική προσέγγιση PCAM – Ομαδοποίηση εργασιών σε διεργασίες

Στην **τρίτη φάση** της προσέγγισης **PCAM**, οι εργασίες ομαδοποιούνται σε **διεργασίες μεγαλύτερου μεγέθους**, κάτι που οδηγεί στην **ελάττωση** του πλήθους τους με στόχο την **αποκατάσταση** των επόμενων **συνθηκών**.

- Την **εξισορρόπηση** του **φόρτου εργασίας** που σχετίζεται με παραμέτρους όπως το **ποσοστό του κώδικα** που εκτελεί η κάθε διεργασία, η χρήση **εισόδου και εξόδου**, η χρήση της **μνήμης** και η **επικοινωνία** μεταξύ των διεργασιών.
- Την **ελαχιστοποίηση της επικοινωνίας** μεταξύ των **διεργασιών**, δηλαδή μεταξύ των **επεξεργαστών** που τις εκτελούν, με στόχο την επίτευξη **μικρότερου κόστους επικοινωνίας** που συνήθως έχει τη μορφή μιας **χρονικής υστέρησης**.

Αυτή η **ομαδοποίηση** εργασιών σε διεργασίες μπορεί να είναι:

Στατική: Πραγματοποιείται από τον **μεταγλωττιστή**, ο οποίος θα πρέπει να γνωρίζει τόσο την **ανάλυση** του αλγορίθμου σε εργασίες όσο και την **τοπολογία** του **γράφου εξάρτησης**.

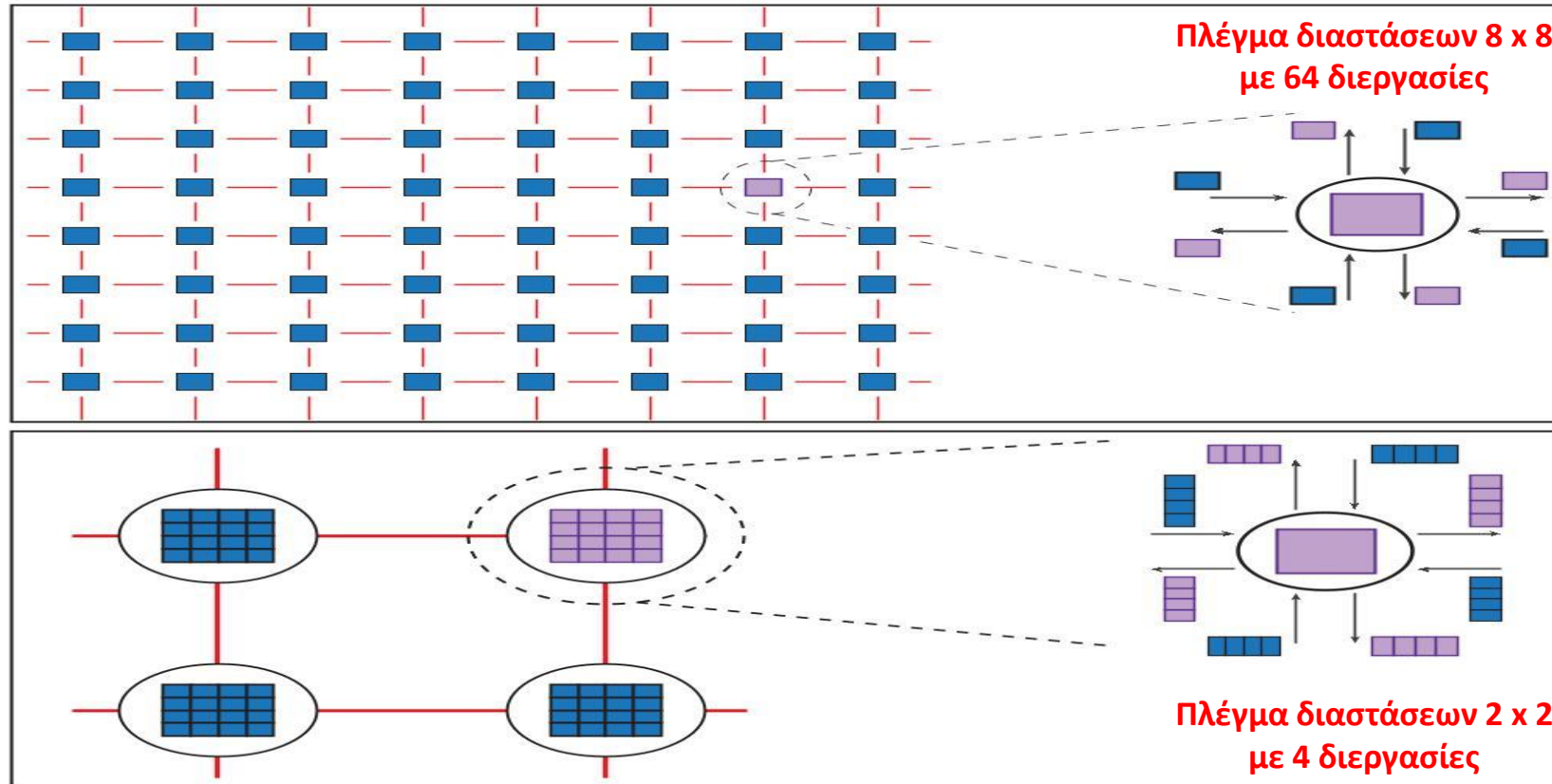
Ημιστατική: Οι εργασίες **ομαδοποιούνται** με τη βοήθεια κάποιου ειδικού **αλγόριθμου**, ενώ υφίσταται η δυνατότητα **τροποποίησης** αυτής της ομαδοποίησης μέσω **ελέγχου** που στηρίζεται στην καταγραφή του **υπολογιστικού φόρτου**.

Δυναμική: είναι κατάλληλη σε περιπτώσεις στις οποίες ο **υπολογισμός** του φόρτου εργασίας είναι **αδύνατος** και στηρίζεται στην χρήση μιας ή περισσότερων **ουρών αναμονής** στις οποίες **προστίθενται** και **αφαιρούνται** διεργασίες.



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Η μεθοδική προσέγγιση PCAM – Ομαδοποίηση εργασιών σε διεργασίες



Η κάθε διεργασία υπολογίζει 1 σημείο

Μεγάλο κόστος επικοινωνίας αφού διακινούνται πολλά δεδομένα

Η κάθε διεργασία υπολογίζει 16 σημεία

Ο καταμερισμός υπολογιστικού έργου έτσι ώστε οι διεργασίες να είναι συνεχώς απασχολημένες γίνεται πιο δύσκολα

Ο **λόγος** του χρονικού διαστήματος **υπολογισμών** προς το χρονικό διάστημα **επικοινωνίας** ονομάζεται **βαθμός επιμερισμού**. Η **χρονική επικάλυψη** των διαδικασιών **υπολογισμού** και **επικοινωνίας** μπορεί να οδηγήσει σε **αύξηση** της απόδοσης.

Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Η μεθοδική προσέγγιση PCAM – Απεικόνιση διεργασιών σε επεξεργαστές

Το πρόβλημα της **κατανομής** των διεργασιών στους **επεξεργαστές** του συστήματος, είναι ένα πρόβλημα **NP-complete** με λύση που βρίσκεται μόνο σε **τετριμμένες** περιπτώσεις ενώ γενικά καταφεύγουμε σε **ευρετικές** μεθόδους.

Ο στόχος είναι η **ελαχιστοποίηση** του **συνολικού χρόνου εκτέλεσης** της εφαρμογής, κάτι που γίνεται με **δύο** τρόπους:

- τοποθετώντας τις διεργασίες του συστήματος σε **διαφορετικούς επεξεργαστές**, έτσι ώστε να εκτελεστούν **ταυτόχρονα**
- τοποθετώντας τις διεργασίες που χαρακτηρίζονται από **μεγάλη συχνότητα επικοινωνίας** στον **ίδιο επεξεργαστή**, έτσι ώστε να **ελαχιστοποιήσουμε** το **κόστος** της **επικοινωνίας** μέσω του δικτύου.

Αυτή η ανάθεση εργασιών ενδεχομένως επηρεάζεται **από περιορισμούς διαθεσιμότητας** των **πόρων** του συστήματος.

Εάν χρησιμοποιείται **διαμέριση δεδομένων** και υφίσταται **μεταβλητός φόρτος εργασίας ανά επεξεργαστή** και **ανομοιόμορφη κυκλοφορία δικτύου** απαιτείται η **αποκατάσταση συνθηκών εξισορρόπησης φορτίου** μέσω τεχνικών όπως

η **αναδρομική διχοτόμηση** που είναι παράδειγμα της μεθόδου **διαίρει και βασίλευε**
η **κατανομή** των διεργασιών στους επεξεργαστές του συστήματος με **τυχαίο τρόπο**, έτσι ώστε, **στατιστικά** ο κάθε επεξεργαστής να συσχετισθεί **με τον ίδιο κατά μέσο όρο φόρτο** εργασίας.



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Η μεθοδική προσέγγιση PCAM – Απεικόνιση διεργασιών σε επεξεργαστές

Αυτή η φάση της μεθοδολογίας PCAM, σχετίζεται με το ζήτημα της **ανάθεσης υπολογισμών σε διεργασίες** που υλοποιείται με **τρεις** διαφορετικούς τρόπους.

- **Μοντέλο ανάθεσης / διεκπεραίωσης**: υπάρχει μία **διεργασία - συντονιστής** που διαβάζει δεδομένα εισόδου από διάφορες πηγές και τα **διανέμει** στις υπόλοιπες διεργασίες για διεκπεραίωση.
- **Μοντέλο γραμμής παραγωγής**: μία λειτουργία διασπάται σε ένα πλήθος **υπολειτουργιών**, η **καθεμία** από τις οποίες εξυπηρετείται και από μία **διαφορετική** διεργασία.
- **Μοντέλο ομότιμων διεργασιών**: λειτουργεί όπως το **μοντέλο ανάθεσης / διεκπεραίωσης**, με τη διαφορά πως η διεργασία συντονιστής **πραγματοποιεί και η ίδια** κάποιο ποσοστό του συνολικού πλήθους των υπολογισμών.

Η **ανάθεση διεργασιών** στους επεξεργαστές, είναι γνωστή ως **χρονοδρομολόγηση** και μπορεί να πραγματοποιηθεί τόσο από την ίδια την **παράλληλη εφαρμογή** όσο και από το υποκείμενο **λειτουργικό σύστημα**.

Οι **αλγόριθμοι χρονοδρομολόγησης** μπορεί να είναι **ντετερμινιστικοί**, όταν ο χρόνος εκτέλεσης είναι **σταθερός** και **γνωστός εκ των προτέρων**, όσο και **μη ντετερμινιστικοί**, που σχετίζονται με **διακλαδώσεις** και πάσης φύσεως **πηγές πολυπλοκότητας** (αστοχίες κρυφής μνήμης, ανταλλαγή σελίδων ανάμεσα στη μνήμη και στο δίσκο κ.λπ.).



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Παραλληλοποίηση ατμοσφαιρικών μοντέλων

Ένα πολύ γνωστό πρόβλημα στο οποίο μπορεί να εφαρμοσθεί η μέθοδος PCAM αφορά στην **μοντελοποίηση της ατμόσφαιρας** και σχετίζεται με την **προσομοίωση ατμοσφαιρικών διαδικασιών**, όπως η **κίνηση των νεφών** και του **ανέμου** με εφαρμογές στην **πρόβλεψη του καιρού** και στη μελέτη της **επίδρασης του κλίματος** στην **συγκέντρωση του CO₂**.

Στο πρόβλημα εμπλέκονται **γνωστές εξισώσεις** της φυσικής όπως είναι

- η εξίσωση διατήρησης της μάζας
- οι εξισώσεις διατήρησης της ορμής
- η εξίσωση της υδροστατικής προσέγγισης
- η εξίσωση διατήρησης της ενέργειας
- η καταστατική εξίσωση των αερίων

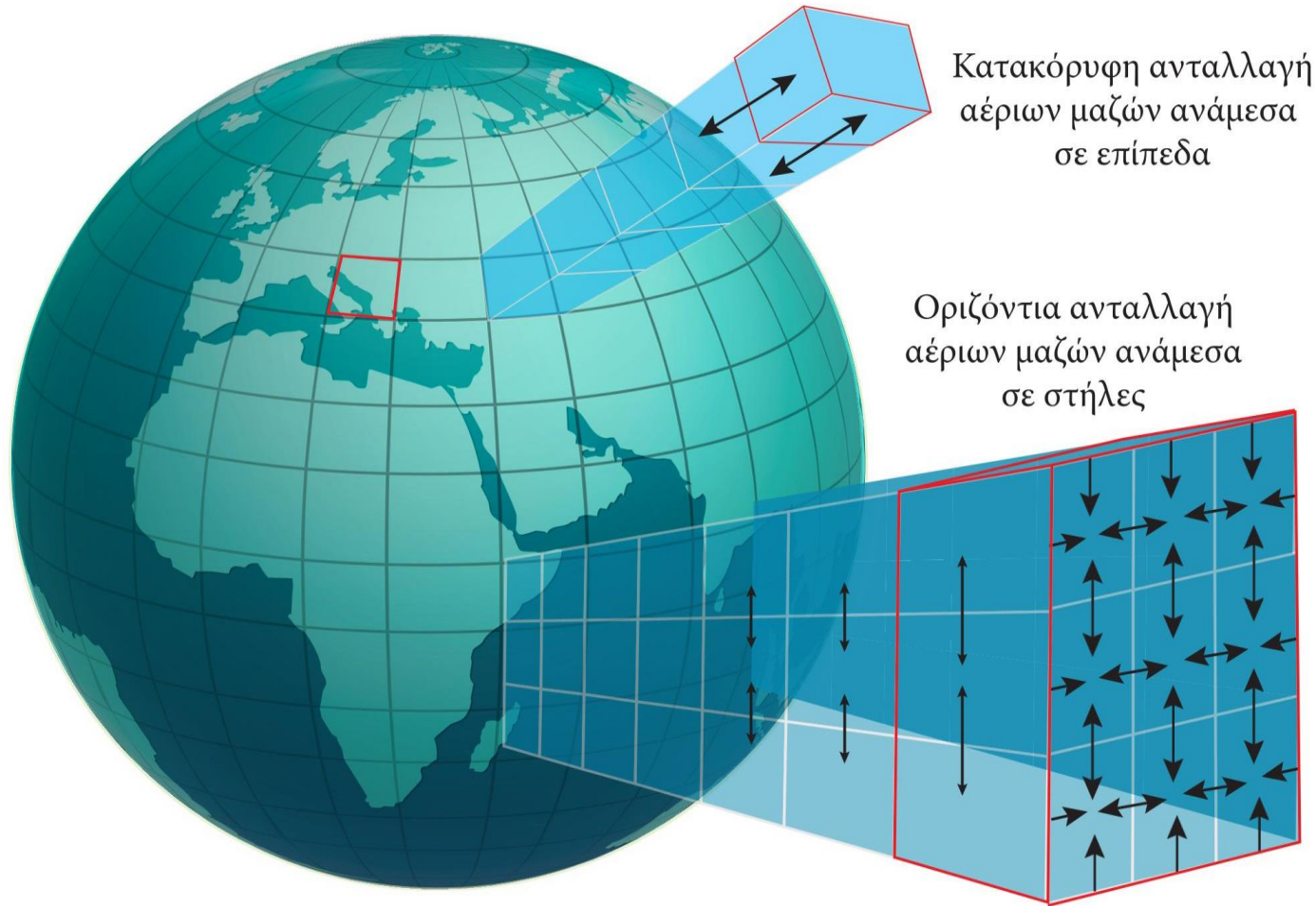
Θεωρούμε σημεία **ομοιόμορφα κατανεμημένα** σε ένα πλέγμα G σχήματος **ορθογώνιου παραλληλεπιπέδου**, με διαστάσεις $M \times N \times L$ (για τις κατευθύνσεις x , y και z), με τιμές $15 \leq M \leq 30$ και $50 \leq N \leq 500$, ενώ επιπλέον δεχόμαστε ότι $M \approx 2N$.

Το πλέγμα είναι **περιοδικό** ως προς τις διαστάσεις x και y , ενώ οι **εξαρτήσεις** που υφίστανται ανάμεσα στα δεδομένα του προβλήματος που διατηρούνται για αυτό το πλέγμα και τα οποία αφορούν σε καταγεγραμμένες τιμές φυσικών μεγεθών, όπως είναι η **πίεση**, η **θερμοκρασία**, η **ταχύτητα του ανέμου** και η **υγρασία**, περιορίζονται μεταξύ των σημείων που ανήκουν σε **κατακόρυφες στήλες** της ατμόσφαιρας



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Παραλληλοποίηση ατμοσφαιρικών μοντέλων



Οι εξισώσεις που εμπλέκονται στην **μαθηματική** περιγραφή του **ατμοσφαιρικού μοντέλου**

$$\frac{\partial \rho}{\partial t} = -\frac{1}{R \cos \varphi} \left(\frac{\partial}{\partial \lambda} (\rho u) + \frac{\partial}{\partial y} (\rho v \cos \varphi) \right) - \frac{\partial}{\partial z} (\rho w)$$

$$\frac{du}{dt} - \left(f + u \frac{\tan \varphi}{R} \right) v = -\frac{1}{R \cos \varphi} \frac{1}{\rho} \frac{\partial p}{\partial \lambda}$$

$$\frac{dv}{dt} + \left(f + u \frac{\tan \varphi}{R} \right) u = -\frac{1}{\rho R} \frac{\partial p}{\partial \varphi} + F_{\varphi}$$

$$g = -\frac{1}{\rho} \frac{\partial p}{\partial z}$$

$$C_p \frac{dT}{dt} - \frac{1}{\rho} \frac{dp}{dt} = Q$$

$$P = nRT$$

Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Παραλληλοποίηση ατμοσφαιρικών μοντέλων

Η εφαρμογή της **μεθοδικής προσέγγισης PCAM** για αυτό το πρόβλημα, πραγματοποιείται με τον ακόλουθο τρόπο:

1. Διαμέριση δεδομένων. Πραγματοποιείται σύμφωνα με τη μέθοδο των πεπερασμένων διαφορών.

Για τον **μέγιστο βαθμό παραλληλισμού**, αντιστοιχούμε **μία εργασία σε κάθε σημείο του πλέγματος**, δημιουργώντας επομένως **$M \times N \times L$** εργασίες, με την **πολυπλοκότητα** των **υπολογισμών** και τον **όγκο** των **δεδομένων** να είναι της τάξεως του **$O(1)$** για το κάθε χρονικό βήμα μεταβολής.

2. Επικοινωνία ανάμεσα στις εργασίες. Απαιτούνται **τρία είδη υπολογισμών** που υπαγορεύουν και τον τρόπο επικοινωνίας:

- **Υπολογισμούς σε κάθε σημείο του πλέγματος από την αντίστοιχη εργασία.** Η κάθε εργασία απαιτεί τα δεδομένα των **οκτώ** γειτονικών διεργασιών κατά την **οριζόντια** διεύθυνση και των **δύο** γειτονικών διεργασιών κατά την **κατακόρυφη** διεύθυνση.
- **Καθολικούς υπολογισμούς**, που συνίστανται στον **περιοδικό υπολογισμό** της συνολικής μάζας της ατμόσφαιρας, έτσι ώστε να διασφαλιστεί πως η προσομοίωση λειτουργεί με τον σωστό τρόπο.
- **Υπολογισμούς της τιμής των φυσικών μεγεθών στο κάθε σημείο του πλέγματος**, οι οποίοι επίσης απαιτούν την επικοινωνία ανάμεσα στις διεργασίες της παράλληλης εφαρμογής.



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Παραλληλοποίηση ατμοσφαιρικών μοντέλων

3. Ομαδοποίηση εργασιών σε διεργασίες. Το **συνολικό πλήθος** των εργασιών που είναι ίσο με $M \times N \times L$, είναι της τάξεως του 10^5 έως 10^7 και επειδή κατά πάσα πιθανότητα αυτός ο αριθμός είναι **πολύ μεγαλύτερος** από το επιθυμητό πλήθος των εργασιών, θα πρέπει να καταφύγουμε σε μία **διαδικασία ομαδοποίησης**.

Αυτή η ομαδοποίηση μπορεί να πραγματοποιηθεί τόσο κατά την **οριζόντια** όσο και κατά την **κατακόρυφη** διεύθυνση, που είναι και η πιο **σημαντική**, διότι το πλήθος των υπολογισμών που απαιτούνται κατά μήκος αυτής της διεύθυνσης είναι **πολύ μεγαλύτερο** σε σχέση με το πλήθος των υπολογισμών κατά την οριζόντια διεύθυνση.

4. Απεικόνιση διεργασιών σε επεξεργαστές. Εάν **δεν υπάρχουν** ασυμμετρίες φορτίου, δηλαδή εάν η **κάθε** διεργασία πραγματοποιεί **το ίδιο πλήθος υπολογισμών** με τις υπόλοιπες, μπορούμε να **υποδιαιρέσουμε** το πλέγμα των διεργασιών σε **ομάδες διεργασιών** και να εκχωρήσουμε την κάθε ομάδα σε έναν απλό επεξεργαστή.

Εάν απαιτείται **περαιτέρω ομαδοποίηση**, αυτή μπορεί να πραγματοποιηθεί **χωρίς** πρόβλημα.

Ωστόσο, αποδεικνύεται, πως σε κάποια **υπολογιστικά μοντέλα ατμόσφαιρας**, υπάρχει **ανισορροπία φορτίου** και ίσως μία τέτοια απεικόνιση **να μην είναι αποδοτική**, οπότε ενδεχομένως θα πρέπει να καταφύγουμε σε μία **κυκλική** απεικόνιση.



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Γράφοι και συναρτήσεις κόστους

Οι **δομές γράφων** που αποτελούν **δημοφιλείς δομές δεδομένων** για την περιγραφή καταστάσεων που σχετίζονται με τον χώρο της **παράλληλης επεξεργασίας** και όχι μόνο, ορίζονται ως ένα **σύνολο κορυφών** που συνδέονται με **προσανατολισμένα τόξα** και ανάλογα με την περίπτωση, περιγράφουν το κατάλληλο κάθε φορά **χαρακτηριστικό** του συστήματος.

Ο **γράφος αλληλεπίδρασης εργασιών** (Task Interaction Graph, TIG) ορίζεται ως $G(V,E)$ με τις **κορυφές** του συνόλου V να αναπαριστούν τις **εργασίες** και τις **ακμές** του συνόλου E να αντιστοιχούν στις **εξαρτήσεις επικοινωνίας δεδομένων**.

Το **βάρος** w_i που σχετίζεται με μία **εργασία** i , αναπαριστά τον **υπολογιστικό φόρτο** που έχει ανατεθεί σε αυτήν την εργασία, ενώ το βάρος c_{ij} μιας ακμής (i,j) , αναπαριστά την σχετική ποσότητα της επικοινωνίας ανάμεσα στις εργασίες i και j .

Ένας **παράλληλος υπολογιστής** αναπαρίσταται ως ένας **γράφος** της μορφής $G(P,E_p)$ στον οποίο οι **κορυφές** του συνόλου P αναπαριστούν τους **επεξεργαστές** του συστήματος, ενώ οι ακμές του συνόλου E_p αναπαριστούν τους διαύλους επικοινωνίας.

Εδώ **δεν** χρησιμοποιούνται **τιμές βαρών** που να σχετίζονται με τις **κορυφές** και τις **ακμές**, αφού όλοι οι υπολογιστικοί κόμβοι υποτίθεται πως χαρακτηρίζονται **από την ίδια υπολογιστική ισχύ**, ενώ όλοι οι **διάυλοι επικοινωνίας** υποτίθεται πως είναι σε θέση να υποστηρίξουν **τον ίδιο ρυθμό μεταφοράς δεδομένων**.

Σε **κάθε** χρονική στιγμή, ο κάθε επεξεργαστής **είτε** πραγματοποιεί κάποιους υπολογισμούς **είτε** συμμετέχει σε μία διαδικασία επικοινωνίας δεδομένων, **χωρίς** όμως να είναι σε θέση να πραγματοποιήσει αυτές τις δύο διαδικασίες **ταυτόχρονα**.



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Γράφοι και συναρτήσεις κόστους

Το **κόστος επικοινωνίας** είναι ανάλογο του **μεγέθους** του μηνύματος και της **αποστάσεως** αποστολέα / παραλήπτη.

Η **απόσταση** ανάμεσα σε δύο κορυφές, είναι ίση με το **ελάχιστο μήκος της διαδρομής** ανάμεσα στις δύο κορυφές

Η ανάθεση εργασιών σε επεξεργαστές, αποτελεί μία απεικόνιση της μορφής $M: V \rightarrow P$, με την παράμετρο $M(i)$ να προσδιορίζει τον **επεξεργαστή** στον οποίο έχει ανατεθεί για εκτέλεση η **εργασία** i .

Το **σύνολο εργασιών** TS_q (Task Set) ενός επεξεργαστή q περιλαμβάνει **όλες** τις εργασίες που έχουν ανατεθεί σε αυτόν για εκτέλεση και κατά συνέπεια, ορίζεται ως

$$TS_q = \{i \mid M(i) = q\}, q = 1, 2, \dots, K$$

Ο **υπολογιστικός φόρτος (Work Load, WL)** του επεξεργαστή q , ορίζεται ως το **άθροισμα των βαρών** w_i **όλων** των εργασιών που έχουν ανατεθεί σε αυτόν, ή σε μαθηματική γραφή,

$$WL_q = \sum_{j \in TS_q} w_j, \quad j = 1, 2, \dots, K$$

ενώ το **ιδανικό μέσο φορτίο** δίδεται από την εξίσωση $\overline{WL} = \frac{1}{K} \sum_{i=1}^k WL_i$



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Γράφοι και συναρτήσεις κόστους

Το **σύνολο επικοινωνίας** (Communication Set, CS_q) ενός επεξεργαστή q , ορίζεται ως το **σύνολο των ακμών** του γράφου **TIG** που **παρεμβάλλονται** μεταξύ αυτού και **κάποιου άλλου** επεξεργαστή κάτω από την απεικόνιση $M(i)$, δηλαδή ως

$$CS_q = \{(i, j) \mid M(i) = q \text{ και } M(j) \geq q\}, q = 1, 2, \dots, K$$

Ο **φόρτος επικοινωνίας** (Communication Load, CL) του επεξεργαστή q , ορίζεται ως το **συνολικό σταθμισμένο κόστος** των **ακμών** του συνόλου επικοινωνίας, με την **κάθε ακμή** να σχετίζεται με μία **τιμή βάρους** η οποία ορίζει το **μήκος της φυσικής διαδρομής** που θα πρέπει να διανυθεί κάτω από την απεικόνιση M . Θα είναι δηλαδή,

$$CL_q = \sum_{(i,j) \in CS_q} c_{ij} d_{M(i)M(j)}, q = 1, 2, \dots, K$$

Η **ποσοτική ανάλυση** και **συγκριτική αξιολόγηση** των **εναλλακτικών** αρχιτεκτονικών, καθώς και των πάσης φύσεως **λύσεων** που μπορούν να χρησιμοποιηθούν στην **παραλληλοποίηση** μιας εφαρμογής, στηρίζεται στη χρήση **συναρτήσεων κόστους**

Σύμφωνα με τους **Ercal, Ramanujam & Sadayappan**, το **γενικότερο** ζήτημα της **απεικόνισης εργασιών σε επεξεργαστές**, στηρίζεται στη χρήση **δύο** τύπων συναρτήσεων κόστους και πιο συγκεκριμένα, τις συναρτήσεις τύπου **minmax** και τις συναρτήσεις του **αθροιστικού συνολικού κόστους**.



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Γράφοι και συναρτήσεις κόστους

Στο **πρώτο** μοντέλο, υπολογίζεται το **συνολικό χρονικό διάστημα εκτέλεσης** της εφαρμογής (το **άθροισμα** των χρονικών διαστημάτων πραγματοποίησης **υπολογισμών** και **επικοινωνίας**) για **κάθε** επεξεργαστή και για μία **συγκεκριμένη** ανάθεση εργασιών και καταβάλλεται προσπάθεια **ελαχιστοποίησης** του **μέγιστου** κόστους για **όλους** τους επεξεργαστές.

Κατά συνέπεια, αυτό το πρόβλημα **μαθηματικά** διατυπώνεται ως

$$\min_M \left\{ \max_q (K_c^{\text{MM}} \times \text{CL}_q) + K_e^{\text{MM}} \times \text{WL}_q \right\}$$

Το **δεύτερο** μοντέλο στηρίζεται στην καταμέτρηση της **απόκλισης** από την **ιδανική** περίπτωση σύμφωνα με την οποία ο υπολογιστικός φόρτος θα πρέπει να κατανεμηθεί **ομοιόμορφα** στους επεξεργαστές, με μηδενικό κόστος επικοινωνίας.

Εάν ως κριτήριο αξιολόγησης, χρησιμοποιήσουμε την **κατανομή του υπολογιστικού φόρτου**, μπορούμε να **εκφράσουμε** το σχετιζόμενο **κόστος πραγματοποίησης υπολογισμών** (αφού η ιδανική περίπτωση χαρακτηρίζεται από **μηδενικό** κόστος επικοινωνίας) ως το **άθροισμα** (για όλους τους επεξεργαστές) των **απόλυτων τιμών** της απόκλισης που υφίσταται ανάμεσα στον **πραγματικό** υπολογιστικό φόρτο και στον μέσο **υπολογιστικό** φόρτο, ο οποίος μπορεί να υπολογιστεί.



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Γράφοι και συναρτήσεις κόστους

Στηριζόμενοι σε όλα τα παραπάνω, το κόστος για αυτό το δεύτερο μοντέλο, ορίζεται από την εξίσωση

Κόστος (M) = Ποινή για την επικοινωνία + Ποινή για την απουσία ισοτανανομής υπολογιστικού φόρτου

$$\min_M \left\{ K_c^{\text{sc}} \times \sum_{i=1}^K \text{CL}_i + K_e^{\text{sc}} \times \sum_{i=1}^K |\text{WL}_i - \overline{\text{WL}}| \right\}$$

Προκειμένου η επιλογή των πολλαπλασιαστικών συντελεστών στις παραπάνω εξισώσεις να μην γίνει με αυθαίρετο τρόπο, οι Ercal, Ramaniyam & Sadayappan προτείνουν ένα εναλλακτικό κριτήριο αθροιστικού κόστους, στο οποίο η ελαχιστοποίηση αφορά μόνο στο αθροιστικό κόστος επικοινωνίας που δίδεται από την εξίσωση

$$\min_M \left(\sum_q \text{CL}_q \right) \quad \text{υπό τον περιορισμό} \quad \frac{|\text{WL}_q - \overline{\text{WL}}|}{\overline{\text{WL}}} < \text{tol}, q = 1, 2, \dots, K$$

όπου tol (tolerance) μία τιμή ανοχής, η οποία θα πρέπει να καθοριστεί. Ο περιορισμός που ορίζεται από την παραπάνω συνθήκη, σχετίζεται με το ζήτημα της κατανομής υπολογιστικού φόρτου



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Ποσοτικά κριτήρια αξιολόγησης παράλληλων εφαρμογών

Επιτρέπουν τον ποσοτικό χαρακτηρισμό της απόδοσης μιας παράλληλης εφαρμογής, με βάση την γνώση

- του χρόνου εκτέλεσης και της δυνατότητας κλιμάκωσης του προβλήματος
- των μηχανισμών στους οποίους στηρίζεται η δημιουργία και η αποθήκευση δεδομένων
- του τύπου της επικοινωνίας των διεργασιών δια της χρήσεως του υποκείμενου δικτυακού εξοπλισμού

Επιτάχυνση

Η επιτάχυνση, ορίζεται ως ο λόγος του χρόνου εκτέλεσης T_s μιας εφαρμογής η οποία εκτελείται σειριακά, προς τον χρόνο εκτέλεσης T_p της ίδιας εφαρμογής εάν αυτή παραλληλοποιηθεί.

Ο χρόνος της παράλληλης εκτέλεσης της εφαρμογής T_p σε ένα σύνολο $p > 1$ επεξεργαστών, ορίζεται ως το χρονικό διάστημα που μεσολαβεί από την είσοδο του πρώτου δεδομένου προς επεξεργασία, μέχρι την εμφάνιση του πρώτου αποτελέσματος.

Θα είναι λοιπόν

$$\psi(n, p) = \frac{T_s}{T_p}$$

όπου το n εκφράζει το μέγεθος του προβλήματος και p το πλήθος των διαθέσιμων επεξεργαστών



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Ποσοτικά κριτήρια αξιολόγησης παράλληλων εφαρμογών

Εάν ως **χρόνο σειριακής εκτέλεσης** θεωρήσουμε τον χρόνο εκτέλεσης **της πιο γρήγορης υλοποίησης** του **σειριακού** αλγορίθμου ή **επιτάχυνση** που υπολογίζεται με βάση την τιμή αυτού του χρόνου, χαρακτηρίζεται ως **απόλυτη** επιτάχυνση.

Εάν $\sigma(n)$ και $\phi(n)$ είναι τα χρονικά διαστήματα εκτέλεσης των **σειριακών** και των **παράλληλων** υπολογισμών και $k(n,p)$ το χρονικό διάστημα που απαιτείται για την πραγματοποίηση άλλων διαδικασιών (π.χ. **επικοινωνία** ανάμεσα στους επεξεργαστές για την ανταλλαγή δεδομένων και **όχι μόνο**), θα είναι

$$\psi(n, p) = \frac{\sigma(n) + \phi(n)}{\sigma(n) + [\phi(n)/p] + k(n, p)}$$

Πράγματι, στην περίπτωση εκτέλεσης ενός **σειριακού** προγράμματος σε **έναν και μοναδικό επεξεργαστή**, σε κάθε κύκλο του ρολογιού θα εκτελείται **μία και μοναδική εντολή**, ενώ θα είναι $k(n,p) = 0$. Επομένως, $T_s = \sigma(n) + \phi(n)$.

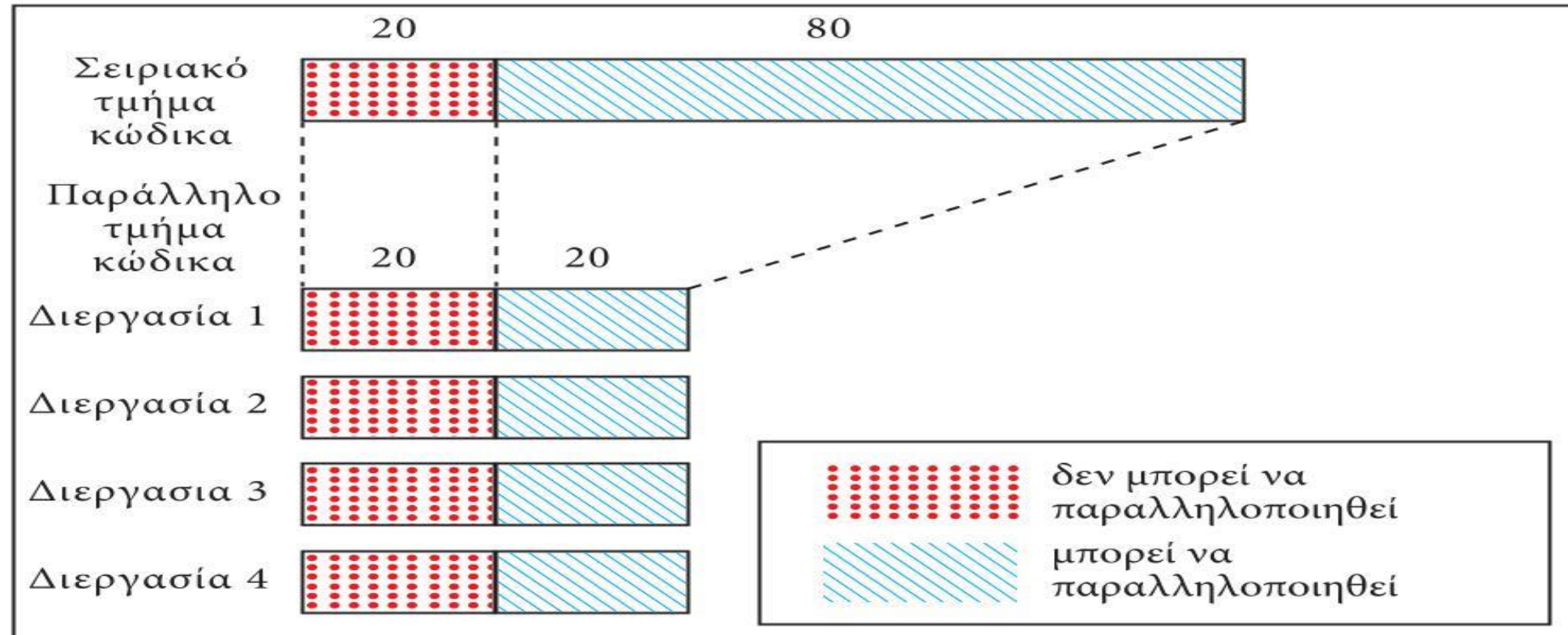
Από την άλλη πλευρά, στην **ιδανική** περίπτωση κατά την οποία οι υπολογισμοί είναι **ανεξάρτητοι** μεταξύ τους το **χρονικό διάστημα εκτέλεσης** των υπολογισμών στον **καθέναν** από τους p επεξεργαστές είναι ίσο $\phi(n)/p$.

Σε αυτό το χρονικό διάστημα θα πρέπει να **προσθέσουμε** τον χρόνο εκτέλεσης **σειριακών** υπολογισμών $\sigma(n)$ και τον χρόνο πραγματοποίησης των υπόλοιπων διαδικασιών $k(n,p)$, από όπου προκύπτει το ζητούμενο.



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Ποσοτικά κριτήρια αξιολόγησης παράλληλων εφαρμογών



Εδώ, **μόνο το 80%** των υπολογισμών μπορεί να **παραλληλοποιηθεί**, ενώ το **20%** θα εκτελεσθεί με **σειριακό** τρόπο. Αυτό σημαίνει πως η τιμή της επιτάχυνσης για την περίπτωση αυτή θα είναι ίση με **2.5** και **όχι** ίση με **4** όπως θα συνέβαινε στην **ιδανική** περίπτωση, κατά την οποία η παραλληλοποίηση μπορούσε να εφαρμοσθεί **στο σύνολο** των υπολογισμών.

Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

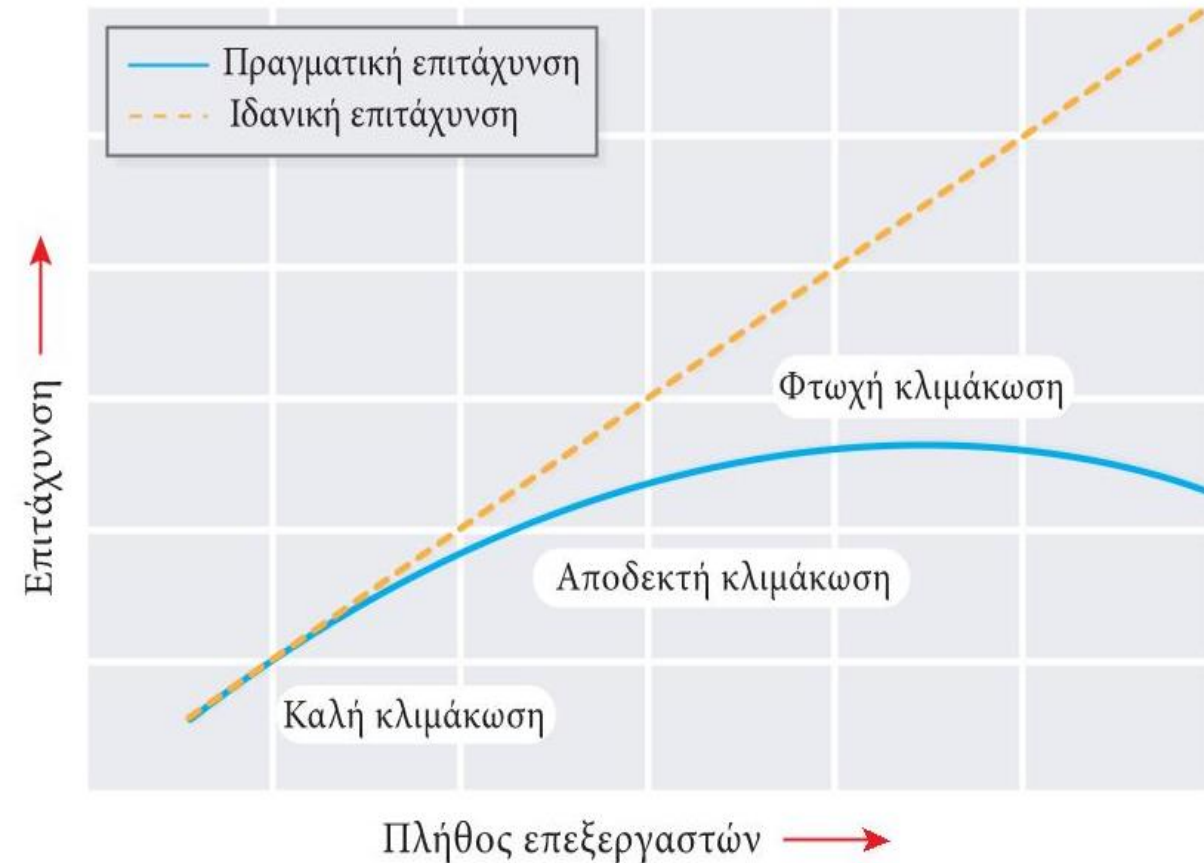
Ποσοτικά κριτήρια αξιολόγησης παράλληλων εφαρμογών

Η διαφορά ανάμεσα στην **ιδανική** επιτάχυνση που αντιστοιχεί στην παραλληλοποίηση **του συνόλου των υπολογισμών** και στην **πραγματική** επιτάχυνση που αντιστοιχεί στην περίπτωση **της ύπαρξης σειριακών υπολογισμών** απεικονίζεται στο σχήμα.

Η ισχύς της παραπάνω σχέσης, προϋποθέτει πως οι παράλληλοι υπολογισμοί **κατανέμονται επακριβώς** στους επεξεργαστές του συστήματος, έτσι ώστε ο **καθένας** από αυτούς, να ολοκληρώσει τους υπολογισμούς του σε χρονικό διάστημα ίσο με $\phi(n)/p$.

Εάν αυτός ο κανόνας **δεν ισχύει**, είναι προφανές πως ο χρόνος πραγματοποίησης των παράλληλων υπολογισμών T_p , θα είναι **μεγαλύτερος** και επομένως, το μέγεθος της επιτάχυνσης – που είναι αντιστρόφως ανάλογο αυτού του χρόνου – θα λάβει **μικρότερη** τιμή.

Η τιμή $\psi(n,p)$ αποτελεί λοιπόν την **μέγιστη** τιμή της επιτάχυνσης για την εκτέλεση μιας **παράλληλης** εφαρμογής μεγέθους n σε ένα σύνολο p επεξεργαστών



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Αποδοτικότητα

Ο **δείκτης απόδοσης** των παράλληλων εφαρμογών που είναι γνωστός ως **αποδοτικότητα** περιγράφεται από τη σχέση

$$\epsilon(n, p) = \frac{\psi(n, p)}{p} = \frac{\sigma(n) + \phi(n)}{p\sigma(n) + \phi(n) + pk(n, p)}$$

Αυτή η παράμετρος αποτελεί **ένα μέτρο της χρήσης των επεξεργαστών του συστήματος**, δηλαδή μας πληροφορεί σχετικά με το **πόσοι** επεξεργαστές **χρησιμοποιούνται** για την εκτέλεση της παράλληλης εφαρμογής και **πόσοι** παραμένουν **αδρανείς**.

Η τιμή του είναι ένα **ποσοστό**, συνήθως **μικρότερο** του 100%

Η ανάγκη που επέβαλε τον ορισμό και την χρήση αυτού του μεγέθους, στηρίζεται στην παρατήρηση, πως από τη στιγμή που η εκτέλεση του ίδιου παράλληλου αλγορίθμου σε περισσότερους επεξεργαστές, ενδέχεται να οδηγήσει σε μεγαλύτερη επιτάχυνση, χρειαζόμαστε ένα μέτρο που να χαρακτηρίζει την ποιότητα της παραλληλοποίησης, αφού στην εξίσωση της επιτάχυνσης δεν εμφανίζεται το πλήθος των επεξεργαστών.



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

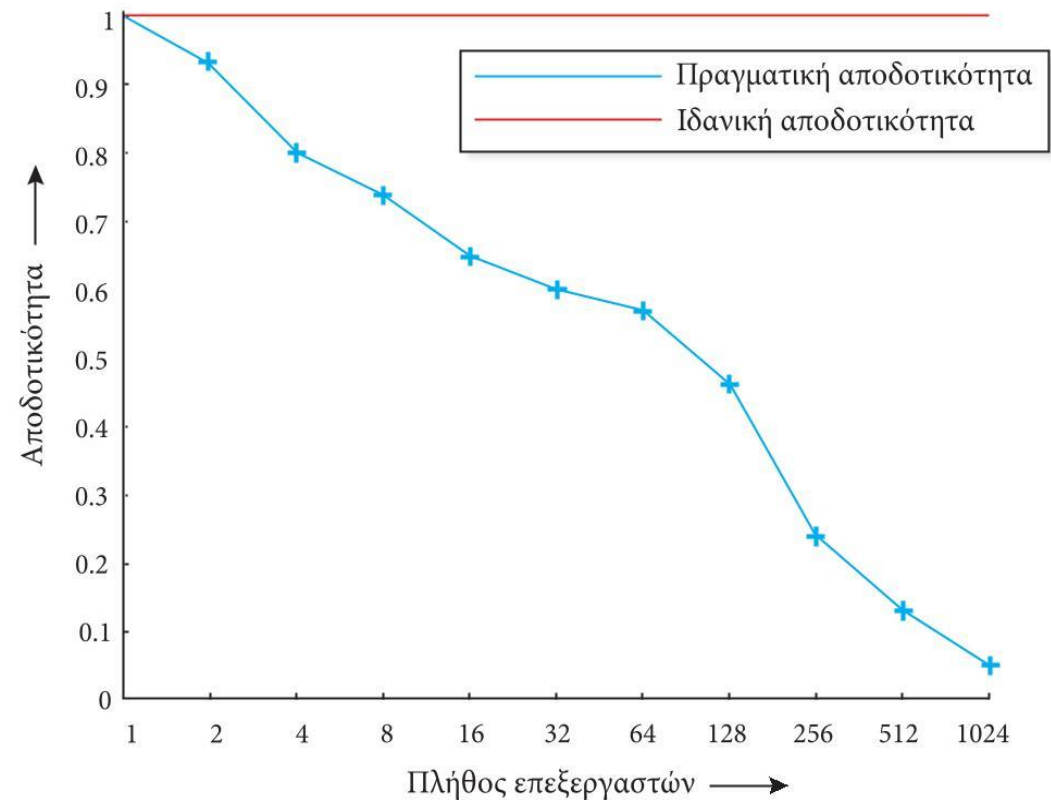
Αποδοτικότητα

Η **αύξηση** της τιμής του πλήθους των επεξεργαστών p , οδηγεί σε **μείωση** της τιμής της αποδοτικότητας.

Πράγματι, εάν **προσθέσουμε** και άλλους επεξεργαστές στο σύστημα, θα **ελαττωθεί** ο χρόνος εκτέλεσης των παράλληλων υπολογισμών $\phi(n)/p$, αλλά ταυτόχρονα θα **αυξηθεί** ο χρόνος που αφιερώνεται στην **επικοινωνία** των επεξεργαστών του συστήματος.

Από κάποιο σημείο και μετά, ο **συνολικός χρόνος εκτέλεσης** της εφαρμογής, **αυξάνεται** με την προσθήκη νέων επεξεργαστών και επομένως, η τιμή του μεγέθους της αποδοτικότητας θα **ελαττώνεται**.

Η **μέγιστη τιμή** της **αποδοτικότητας**, είναι **ανεξάρτητη** του πλήθους των επεξεργαστών p και ίση με τη **μονάδα**, αφού η **μέγιστη** τιμή της επιτάχυνσης $\psi(n,p)$ που αντιστοιχεί στην **ιδανική** περίπτωση, είναι ίση **με το πλήθος p** των διαθέσιμων επεξεργαστών.



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Χρόνος εκτέλεσης

Ο **χρόνος εκτέλεσης** T μιας της **παράλληλης** εφαρμογής, γενικά εκφράζεται ως συνάρτηση

- του **μεγέθους** N του προβλήματος
- του **πλήθους** P των διαθέσιμων **επεξεργαστών**
- του **πλήθους** M των **διεργασιών** που **εκτελούνται** στο σύστημα
- πολλών **άλλων παραμέτρων** που εξαρτώνται από την **φύση** της εφαρμογής

Ορίζεται ως το **χρονικό διάστημα** που οριοθετείται από τις χρονικές στιγμές **έναρξης της λειτουργίας του πρώτου επεξεργαστή** t_1 και **τερματισμού της λειτουργίας του τελευταίου επεξεργαστή** t_2 αν και στις εφαρμογές **καταμερισμού χρόνου**, αυτός ο ορισμός **δεν είναι** και ο καλύτερος δυνατός.

Προκειμένου να ορίσουμε τον χρόνο εκτέλεσης με **πιο συγκεκριμένο** τρόπο, θα πρέπει να λάβουμε υπόψη πως ο **καθένας** από τους **επεξεργαστές** κατά τη διάρκεια της εκτέλεσης, ενδέχεται να περιέλθει σε **τρεις διαφορετικές** καταστάσεις:

- σε **κατάσταση εκτέλεσης (comp)**, κατά τη διάρκεια της οποίας πραγματοποιεί τους κατάλληλους υπολογισμούς
- σε **κατάσταση επικοινωνίας (comm)**, κατά τη διάρκεια της οποίας επικοινωνεί με τους υπόλοιπους επεξεργαστές
- σε **κατάσταση αδράνειας (idle)**, αναμένοντας την ικανοποίησή κάποιας συνθήκης



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Χρόνος εκτέλεσης

Στηριζόμενοι σε αυτόν τον τρόπο συμπεριφοράς των επεξεργαστών, μπορούμε να ορίσουμε τον **συνολικό χρόνο** εκτέλεσης της εφαρμογής, είτε ως το **άθροισμα** των χρόνων για αυτές τις **τρεις καταστάσεις λειτουργίας** και για έναν **αυθαίρετο** επεξεργαστή k , έτσι ώστε να μπορούμε να γράψουμε

$$T = T_{\text{comp}}^k + T_{\text{comm}}^k + T_{\text{idle}}^k$$

είτε ως το **άθροισμα** όλων αυτών των χρόνων **για όλους** τους διαθέσιμους επεξεργαστές **διαιρεμένο** με το **πλήθος** P των **επεξεργαστών**, δηλαδή ως

$$T = \frac{1}{P} (T_{\text{comp}} + T_{\text{comm}} + T_{\text{idle}}) = \frac{1}{P} \left(\sum_{k=0}^{P-1} T_{\text{comp}}^k + \sum_{k=0}^{P-1} T_{\text{comm}}^k + \sum_{k=0}^{P-1} T_{\text{idle}}^k \right)$$

Στην πράξη αυτός ο δεύτερος ορισμός είναι **πιο εξυπηρετικός**, διότι είναι **πιο εύκολο** να μετρήσουμε τον **συνολικό** χρόνο **εκτέλεσης**, **επικοινωνίας** και **αδράνειας** για το **σύνολο** της εφαρμογής, **παρά** για τον κάθε επεξεργαστή **ξεχωριστά**.



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Σχόλια περί της μοντελοποίησης παράλληλων εφαρμογών

Η κατασκευή **μαθηματικών εκφράσεων** για τους τρεις παραπάνω χρόνους, ως **συνάρτηση** των παραμέτρων του προβλήματος, εξαρτάται **από την φύση** του προβλήματος και από τις **υφιστάμενες** συνθήκες. **Ειδικότερα:**

Ο **χρόνος υπολογισμού** μπορεί να εξαρτάται από το **πλήθος** των επεξεργαστών, το οποίο μπορεί να **μεταβάλλεται** σε συνάρτηση με το χρόνο, από τα **χαρακτηριστικά** της λειτουργίας τους, καθώς και από την **ποσότητα** της διαθέσιμης μνήμης.

Ο **χρόνος επικοινωνίας** εξαρτάται από το εάν η **αποστολή** και **λήψη** μηνυμάτων πραγματοποιείται ανάμεσα σε διεργασίες που εκτελούνται στον **ίδιο** ή σε **διαφορετικούς** επεξεργαστές.

Το κόστος της **ανταλλαγής μηνύματος** ανάμεσα σε διεργασίες που εκτελούνται σε **διαφορετικούς** επεξεργαστές, δίνεται από την έκφραση $T_{msg} = t_s + t_w L$ όπου t_s ο χρόνος **έναρξης** της επικοινωνίας και t_w ο χρόνος που απαιτείται για τη **μεταφορά** μιας λέξης, το μήκος L της οποίας συνήθως είναι ίσο με **4 bytes**.

Αυτό το κόστος είναι ανεξάρτητο τόσο από την **θέση** του επεξεργαστή μέσα στο δίκτυο, όσο και από το **πλήθος** των υπολογιστικών κόμβων που συμμετέχουν στη διαδικασία της επικοινωνίας, την **χρονική στιγμή αποστολής** του μηνύματος.

Η πράξη έχει δείξει, πως ο χρόνος που υπολογίζεται είναι **πολύ διαφορετικός** από τον παραπάνω **ιδανικό** χρόνο



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Σχόλια περί της μοντελοποίησης παράλληλων εφαρμογών

Παράγοντας που επηρεάζει την απόδοση, είναι ο ανταγωνισμός επεξεργαστών που επιθυμούν να στείλουν δεδομένα την ίδια χρονική στιγμή, με το κοινό εύρος ζώνης να διαμοιράζεται σε αυτούς.

Εάν υπάρχουν S τέτοιοι επεξεργαστές, ο χρόνος μετάδοσης του μηνύματος είναι $T_{msg} = t_s + t_w LS$ αφού το ενεργό εύρος ζώνης για κάθε επεξεργαστή, είναι ίσο με το $1/S$ του πραγματικού εύρους ζώνης.

Αυτή η έκφραση δεν περιλαμβάνει πρόσθετα κόστη που οφείλονται στις συγκρούσεις και αναμεταδόσεις των μηνυμάτων.

Ο χρόνος αδράνειας μπορεί να εκμηδενιστεί καταφεύγοντας σε τεχνικές εξισορρόπησης φορτίου και σε χρονική επικάλυψη των διαδικασιών υπολογισμού και επικοινωνίας η οποία πραγματοποιείται

- είτε αναθέτοντας στον ίδιο επεξεργαστή την εκτέλεση περισσότερων από μία διεργασίες, οπότε εάν κάποια από αυτές αναστείλει τη λειτουργία της, τότε ο επεξεργαστής εκτελεί κάποια άλλη διεργασία
- είτε σχεδιάζοντας την εφαρμογή έτσι ώστε τα αιτήματα λήψης νέων δεδομένων να παρεμβάλλονται ανάμεσα στα χρονικά διαστήματα των υπολογισμών, προκειμένου ο επεξεργαστής να μην περιέρχεται ποτέ σε κατάσταση αδράνειας.



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Νόμος του Amdahl

Προκύπτει με επαναδιατύπωση της εξίσωσης ορισμού της επιτάχυνσης και έχει τη μορφή

$$\psi(n, p) \leq \frac{1}{f(n) + \frac{1-f(n)}{p}}$$

με το κλάσμα $f(n)$ των υπολογισμών που εκτελούνται με **σειριακό τρόπο** με τιμές στην **περιοχή** $[0,1]$ να ορίζεται ως

$$f(n) = \frac{\sigma(n)}{\sigma(n) + \phi(n)}$$

Αυτός ο νόμος μας επιτρέπει να υπολογίσουμε ένα **άνω όριο** για το μέγεθος της **επιτάχυνσης** που σχετίζεται με τις **παράλληλες διαδικασίες επίλυσης** ενός προβλήματος συγκεκριμένου **μεγέθους** n , χρησιμοποιώντας ένα συγκεκριμένο **πλήθος** επεξεργαστών p , ως συνάρτηση του **ποσοστού της παραλληλίας** και του κλάσματος των **σειριακών** υπολογισμών.

Η ύπαρξη **άνω φράγματος** για τις **παράλληλες** διαδικασίες επιτρέπει την **αύξηση** της συνολικής απόδοσης του αλγορίθμου μέσω **βελτιώσεων** που σχετίζονται και με το **σειριακό** τμήμα των υπολογισμών.

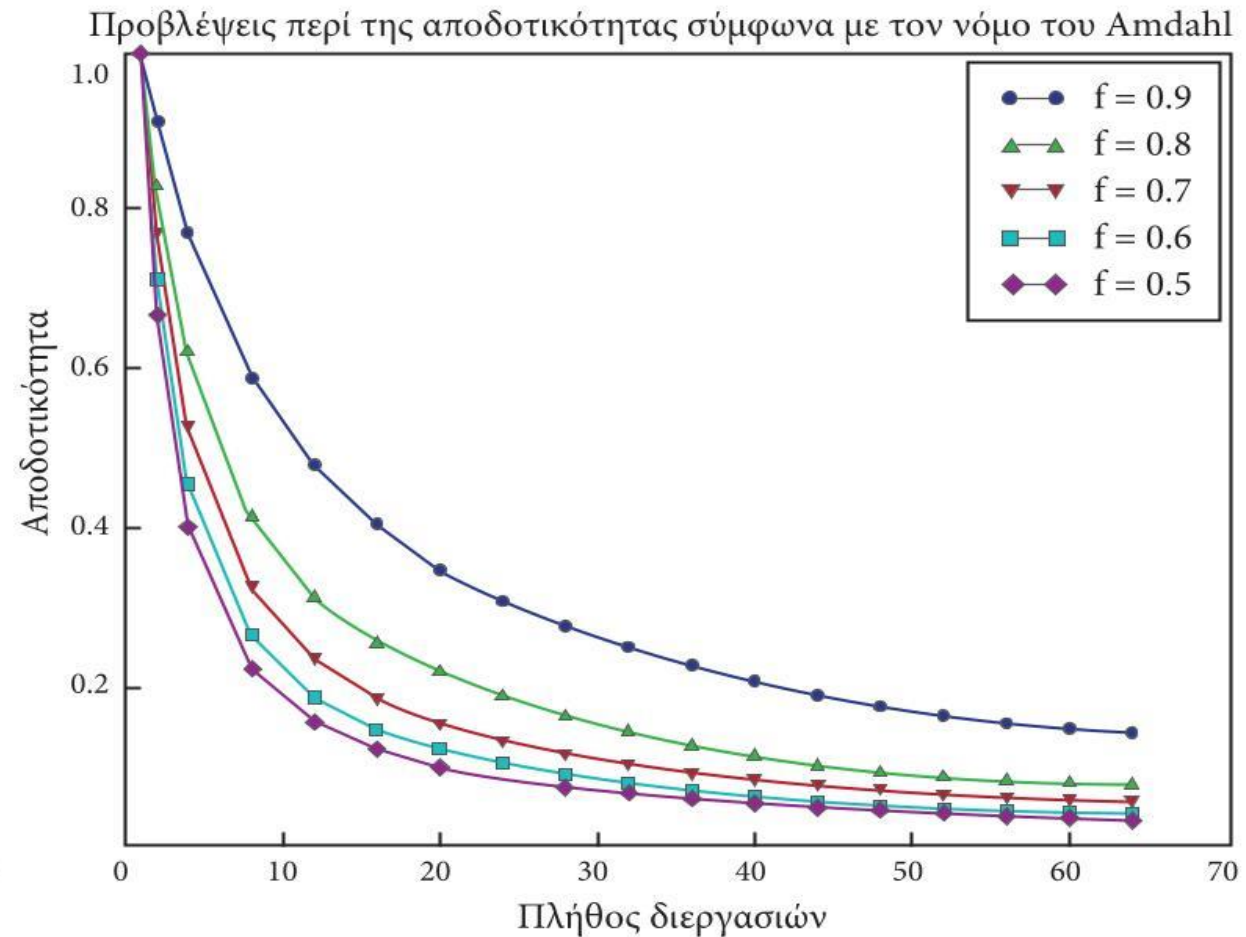
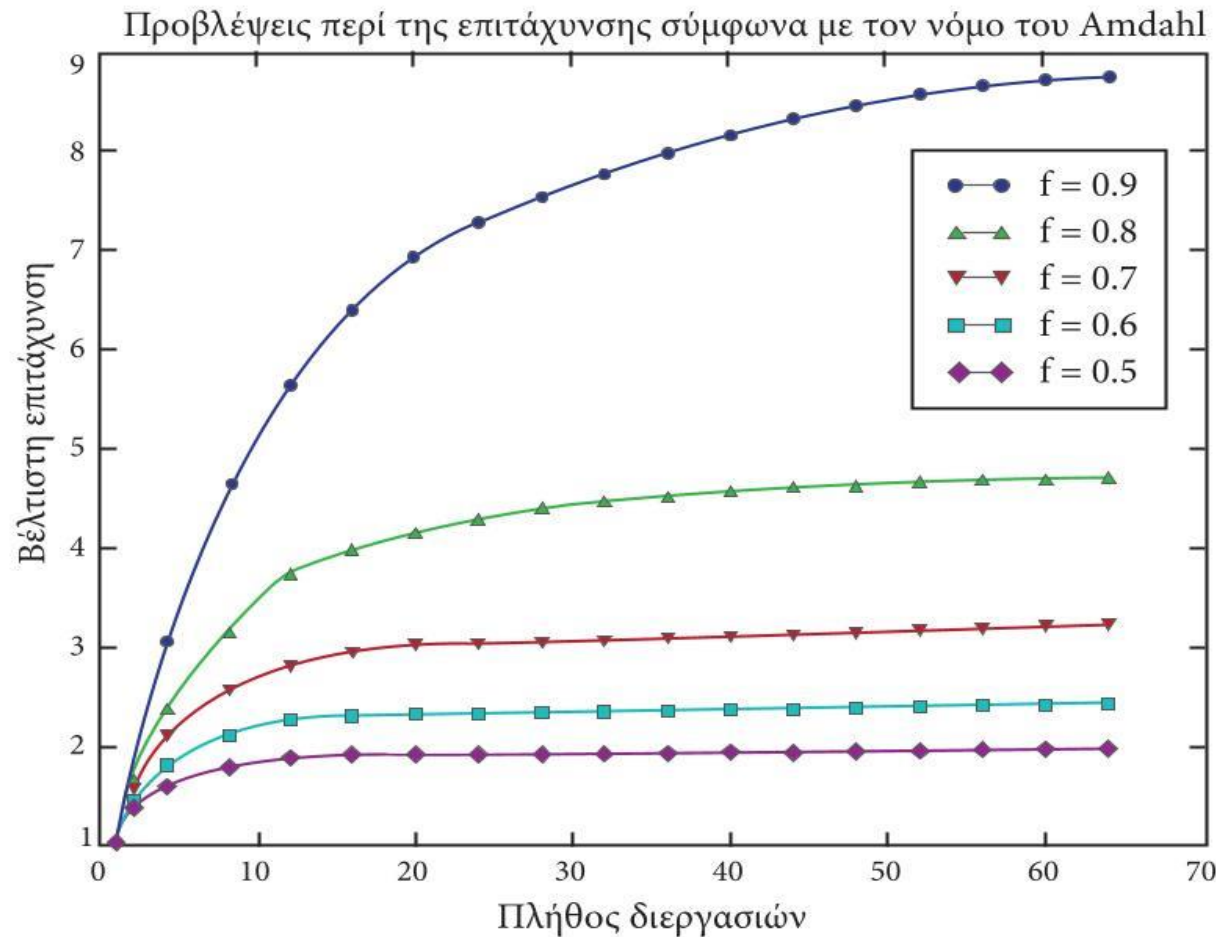
Κανένα μοντέλο παράλληλου προγραμματισμού, όσο **ευφυές** και αν είναι, **δεν είναι σε θέση** να ξεπεράσει αυτό το όριο. Η **αύξηση** της απόδοσης μέσω **βελτιώσεων** τμημάτων υπολογισμού, **μειώνεται** όσο προστίθενται βελτιώσεις (**νόμος του φθίνοντος κέρδους**). Στο όριο των **άπειρων** επεξεργαστών η **μέγιστη** δυνατή επιτάχυνση είναι ίση με $1/f$.



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Νόμος του Amdahl

Η μεταβολή της **επιτάχυνσης** και της **αποδοτικότητας** σύμφωνα με τις προβλέψεις του νόμου του Amdahl



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Νόμος των Gustafson - Barsis

Ο νόμος του Amdahl στηρίζεται στην παραδοχή πως η παραλληλοποίηση μιας εφαρμογής στοχεύει στην επίλυση ενός προβλήματος συγκεκριμένου μεγέθους σε όσο το δυνατό μικρότερο χρονικό διάστημα.

Ωστόσο, σε ορισμένες περιπτώσεις, ο στόχος μας δεν είναι η ελάττωση του χρονικού διαστήματος εκτέλεσης της εφαρμογής, αλλά η αύξηση της ακρίβειας της λύσης που μπορούμε να επιτύχουμε μέσα σε κάποιο συγκεκριμένο χρονικό διάστημα.

Ο νόμος του Amdahl δεν είναι ιδιαίτερα χρήσιμος σε προβλήματα υψηλής απόδοσης, αφού πραγματεύεται μόνο προβλήματα σταθερού μεγέθους, ενώ συχνά ενδιαφερόμαστε να επιλύσουμε μεγαλύτερα προβλήματα, στα οποία η αύξηση του μεγέθους οδηγεί συχνά σε μικρότερο σειριακό κλάσμα εκτέλεσης.

Ενώ ο νόμος του Amdahl σχετίζεται με το πόσο πιο γρήγορα μπορεί να εκτελεστεί μία συγκεκριμένη ποσότητα υπολογισμών σε N επεξεργαστές, κάποιες φορές ενδιαφερόμαστε για το πόσο περισσότερο υπολογιστικό έργο μπορούμε να εκτελέσουμε σε ένα συγκεκριμένο χρονικό διάστημα, χρησιμοποιώντας N επεξεργαστές.



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Νόμος των Gustafson - Barsis

Έστω πως ο **χρόνος εκτέλεσης** είναι **σταθερός**, ενώ το **μέγεθος** του προβλήματος **αυξάνεται** με την **αύξηση** της τιμής του p . Το ποσοστό του χρόνου για την εκτέλεση **παράλληλων** υπολογισμών που εκτελούν **εγγενώς σειριακές** πράξεις είναι

$$s = \frac{\sigma(n)}{\sigma(n) + \phi(n)/p}$$

οπότε, ο χρόνος εκτέλεσης των **παράλληλων υπολογισμών** που εκτελούν **εγγενώς παράλληλες** πράξεις, θα είναι

$$1 - s = 1 - \frac{\sigma(n)}{\sigma(n) + \phi(n)/p} = \frac{\phi(n)/p}{\sigma(n) + \phi(n)/p}$$

Επιλύοντας ως προς $\sigma(n)$ και $\phi(n)$ και αντικαθιστώντας στην **εξίσωση ορισμού της επιτάχυνσης** λαμβάνουμε

$$\psi(n, p) \leq \frac{\sigma(n) + \phi(n)}{\sigma(n) + \phi(n)/p} = \frac{[\sigma(n) + \phi(n)/p][s + (1 - s)p]}{\sigma(n) + \phi(n)/p}$$

από όπου τελικά παίρνουμε

$$\psi(n, p) \leq p + (1 - p)s \quad \textbf{(νόμος των Gustafson - Barsis)}$$



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Νόμος των Gustafson - Barsis

Ο νόμος των Gustafson – Barsis προσδιορίζει τη μέγιστη επιτάχυνση μιας παράλληλης εφαρμογής που επιλύει ένα πρόβλημα μεγέθους n με p επεξεργαστές με κλάσμα χρόνου εκτέλεσης σειριακών υπολογισμών, ίσο με s .

Σε αντίθεση με τον νόμο του Amdahl που ορίζει την επιτάχυνση θεωρώντας έναν σειριακό υπολογισμό και προβλέποντας πόσο γρήγορα αυτός θα εκτελεστεί σε ένα παράλληλο σύστημα, ο νόμος των Gustafson – Barsis λειτουργεί αντίστροφα.

Δηλαδή, θεωρεί έναν παράλληλο υπολογισμό και υπολογίζει τον χρόνο της εκτέλεσής του σε περίπτωση που αυτός εκτελεσθεί με σειριακό τρόπο.

Η επιτάχυνση που προβλέπεται από τον νόμο των Gustafson - Barsis χαρακτηρίζεται ως κλιμακωμένη επιτάχυνση, επειδή εάν χρησιμοποιήσουμε ως σημείο εκκίνησης, όχι τον σειριακό αλλά τον παράλληλο υπολογισμό, επιτρέπει τη θεώρηση του μεγέθους του προβλήματος ως μία αύξουσα συνάρτηση του πλήθους των επεξεργαστών.

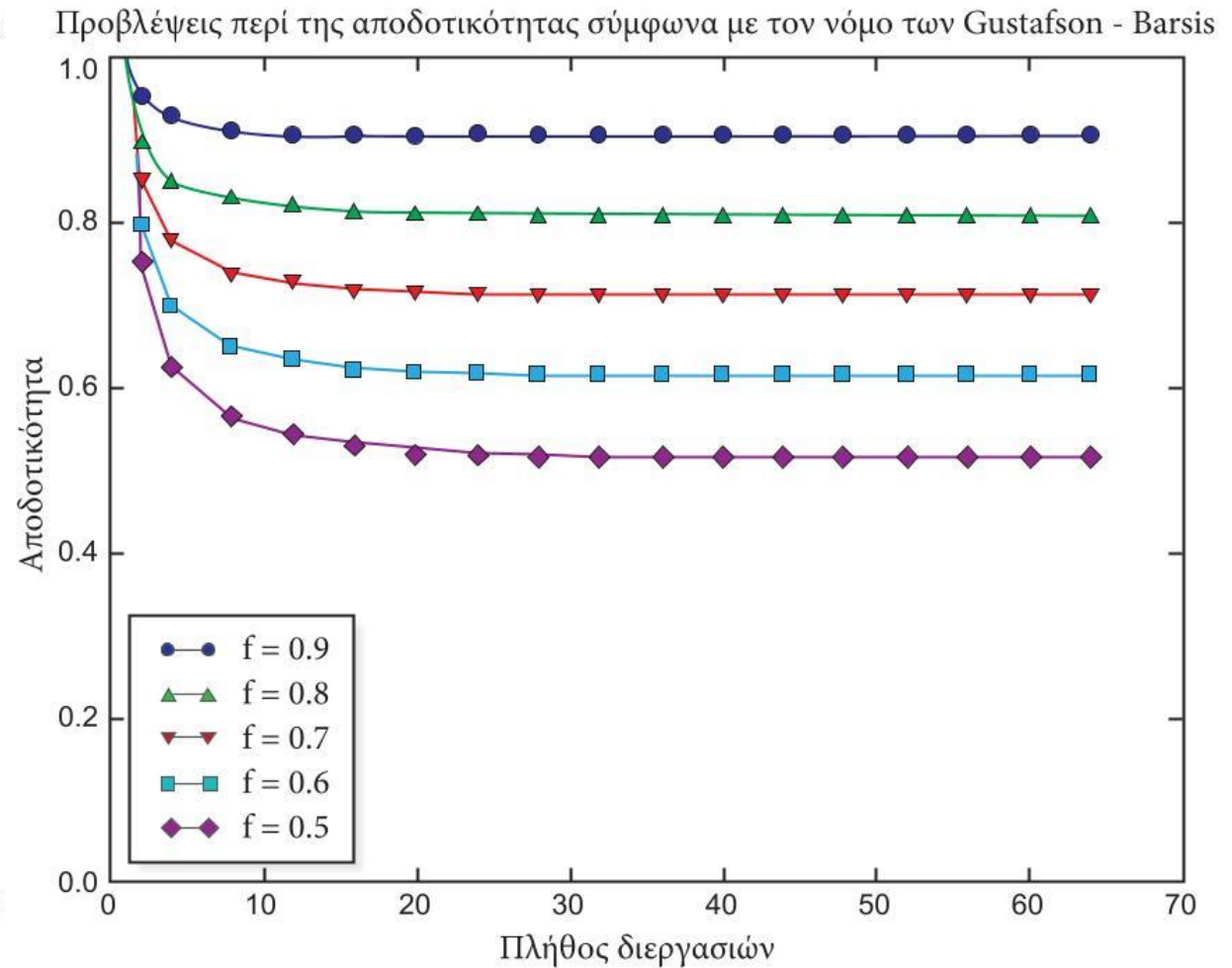
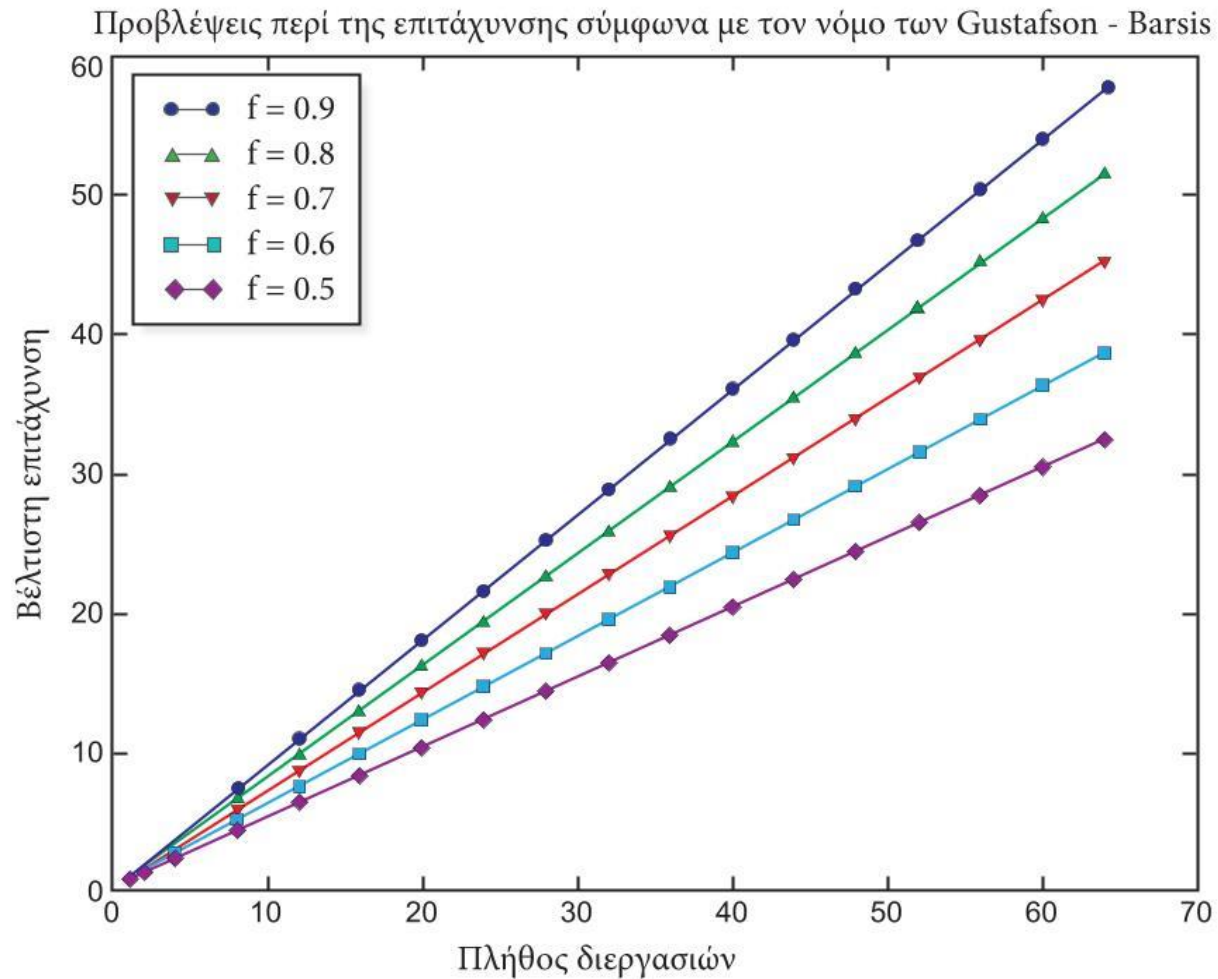
Οι προβλέψεις στις οποίες οδηγεί αυτός ο νόμος αναφέρουν μεγαλύτερες τιμές επιτάχυνσης σε σχέση με αυτές που προκύπτουν από τον νόμο του Amdahl.

Στην πραγματικότητα οι τιμές της επιτάχυνσης που προβλέπονται από αμφότερους τους νόμους είναι μεγαλύτερες από τις πραγματικές αφού δεν λαμβάνεται υπόψη η επιβάρυνση που περιγράφεται από τη συνάρτηση $k(n,p)$.



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Νόμος των Gustafson - Barsis



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Η μετρική των Karp - Flatt

Το **θεωρητικό άνω όριο** της **επιτάχυνσης** που προβλέπεται από τον **νόμο του Amdahl** είναι **υπερεκτιμημένο**, διότι δεν λαμβάνεται υπόψη το **κόστος της επικοινωνίας** μεταξύ των επεξεργαστών.

Απαιτείται λοιπόν ένα **μέτρο**, που να επιτρέπει τον **πειραματικό** προσδιορισμό του **κλάσματος της εκτέλεσης σειριακού κώδικα** στον παράλληλο αλγόριθμο, έτσι ώστε να **αποφανθούμε** για το εάν το **άνω όριο** της επιτάχυνσης οφείλεται στην ύπαρξη **εγγενώς σειριακού κώδικα** ή στην **επιβάρυνση** που προκύπτει από την **εκτέλεση** του **παράλληλου** κώδικα.

Αυτό το **πειραματικά προσδιοριζόμενο κλάσμα σειριακής εκτέλεσης** ενός παράλληλου υπολογισμού **e** ορίζεται ως

$$e = \frac{\sigma(n) + k(n, p)}{T(n, 1)}$$

όπου **T(n,1) = σ(n) + φ(n)** είναι ο χρόνος εκτέλεσης του προβλήματος σε έναν απλό επεξεργαστή.

Η παράμετρος **e** λαμβάνει υπόψη την **επιβάρυνση** που προκύπτει από την εκτέλεση του **παράλληλου** κώδικα, ενώ είναι σε θέση να ανιχνεύσει **πρόσθετες πηγές** επιβάρυνσης που **δεν χρησιμοποιούνται** στους προηγούμενους νόμους όπως

- ο **χρόνος προετοιμασίας** μιας διεργασίας προς εκτέλεση
- ο χρόνος που σπαταλάται ως αποτέλεσμα **διαδικασιών συγχρονισμού**
- η απουσία των **συνθηκών εξισορρόπησης** του **υπολογιστικού φόρτου**
- η **πάσης φύσεως επιβάρυνση** που σχετίζεται με την **αρχιτεκτονική** του συστήματος



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Η μετρική των Karp - Flatt

Η τιμή του e μας επιτρέπει να εξαγάγουμε **συμπεράσματα** σχετικά με τις **συνθήκες εκτέλεσης** της εφαρμογής:

- εάν η τιμή του **δεν μεταβάλλεται** με το p , η **επιβάρυνση** λόγω **εκτέλεσης** της παράλληλης εφαρμογής είναι αμελητέα
- εάν η τιμή του **αυξάνεται με το p** , αυτή η επιβάρυνση **κυριαρχεί** έναντι αυτής που οφείλεται στην **παραλληλοποίηση**, κάτι που έχει ως αποτέλεσμα την **πολύ φτωχή απόδοση** της εφαρμογής.

Ορίζοντας τον **συνολικό χρόνο εκτέλεσης** της **παράλληλης** εφαρμογής από την εξίσωση

$$T(n, p) = \sigma(n) + \frac{\phi(n)}{p} + k(n, p)$$

και **συνδυάζοντας** τις δύο παραπάνω εξισώσεις, μπορούμε να γράψουμε ότι

$$T(n, p) = T(n, 1)e + \frac{T(n, 1)(1 - e)}{p}$$

και **τελικά** καταλήγουμε στη σχέση

θυμηθείτε ότι $\psi(n, p) = T(n, 1) / T(n, p)$

$$e = \frac{\frac{1}{\psi(n, p)} - \frac{1}{p}}{1 - \frac{1}{p}}$$

(μετρική των Karp – Flatt)



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Η μετρική της ισαποδοτικότητας

Επαναδιατυπώνοντας την **εξίσωση ορισμού της επιτάχυνσης** με τη μορφή

$$\psi(n, p) \leq \frac{p[\sigma(n) + \phi(n)]}{\sigma(n) + \phi(n) + (p-1)\sigma(n) + pk(n, p)}$$

και ορίζοντας το **συνολικό χρονικό διάστημα** που απαιτείται για την εκτέλεση από **όλες** τις διεργασίες **όλων των διαδικασιών** που **δεν** σχετίζονται με **σειριακούς** υπολογισμούς ως

$$T_0(n, p) = (p-1)\sigma(n) + pk(n, p)$$

όπου **$pk(n, p)$** ο χρόνος που αφιερώνεται από τους **p επεξεργαστές** για την πραγματοποίηση των **διαδικασιών επικοινωνίας** και **$(p-1)\sigma(n)$** ο χρόνος πραγματοποίησης **σειριακών** υπολογισμών από τους **$(p-1)$ επεξεργαστές**, λαμβάνουμε

$$\psi(n, p) \leq \frac{p[\sigma(n) + \phi(n)]}{\sigma(n) + \phi(n) + T_0(n, p)}$$

Στην περίπτωση αυτή, η **αποδοτικότητα** του συστήματος θα δίδεται από τη σχέση

$$\epsilon(n, p) \leq \frac{\sigma(n) + \phi(n)}{\sigma(n) + \phi(n) + T_0(n, p)} \quad \text{ή ισοδύναμα} \quad \epsilon(n, p) \leq \frac{1}{1 + \frac{T_0(n, p)}{T(n, 1)}}$$

[αφού **$T(n, 1) = \sigma(n) + \phi(n)$**]



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Η μετρική της ισαποδοτικότητας

και **τελικά** οδηγούμαστε στην εξίσωση

$$T(n, 1) \geq \frac{\epsilon(n, p)}{1 - \epsilon(n, p)} T_0(n, p) \quad \text{ή ισοδύναμα} \quad T(n, 1) \geq CT_0(n, p)$$

που ορίζει την **μετρική της ισαποδοτικότητας**, όπου $C = \epsilon(n, p) / [1 - \epsilon(n, p)]$. Εάν λοιπόν επιθυμούμε να διατηρήσουμε **το ίδιο επίπεδο αποδοτικότητας** καθώς αυξάνει η τιμή του p , το μέγεθος n θα πρέπει να **μεταβάλλεται** με τον **παραπάνω** τρόπο.

Χρησιμοποιώντας την παραπάνω έκφραση, μπορούμε να προσδιορίσουμε τις τιμές **του πλήθους των επεξεργαστών** του συστήματος οι οποίοι διασφαλίζουν την **επίτευξη** ενός **συγκεκριμένου επιπέδου αποδοτικότητας**.

Επειδή η **επιβάρυνση** που οφείλεται στην **παραλληλοποίηση** του προβλήματος, **αυξάνεται** με την **αύξηση** του πλήθους των επεξεργαστών, ο **μόνος τρόπος** για να διατηρήσουμε **την ίδια** τιμή αποδοτικότητας, είναι να **αυξήσουμε το μέγεθος του προβλήματος**, το οποίο **περιορίζεται** από την **ποσότητα της κύριας μνήμης** που διαθέτουμε.

Θεωρώντας ένα **σύστημα** που χαρακτηρίζεται από την συνθήκη ισαποδοτικότητας $n \geq f(p)$ και συμβολίζοντας με $M(n)$ την ποσότητα της **μνήμης** που απαιτείται για ένα **πρόβλημα μεγέθους n** , η ανισότητα $M^{-1}(n) \geq f(p)$ μας επιτρέπει να εξαγάγουμε **συμπεράσματα** σχετικά πώς θα πρέπει να **αυξήσουμε** τη μνήμη, για να διατηρήσουμε **σταθερό επίπεδο αποδοτικότητας**.



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Μοντέλα παράλληλου προγραμματισμού

Στην πράξη υφίστανται **δύο προσεγγίσεις** για την υλοποίηση ενός **εργαλείου κατασκευής** παράλληλων εφαρμογών:

- η υλοποίηση μιας **παράλληλης βιβλιοθήκης**
- η κατασκευή ενός **μεταγλωττιστή** με δυνατότητες **παραλληλοποίησης σειριακού κώδικα**.

Στην **πρώτη** προσέγγιση, η **παράλληλη βιβλιοθήκη** (π.χ. **pthread**s, **TBB**, **MPI**) προσφέρει **ομάδες συναρτήσεων** που θα χρησιμοποιηθούν από τον προγραμματιστή για την υλοποίηση της παράλληλης εφαρμογής.

Επειδή η βιβλιοθήκη **δεν βρίσκεται ενσωματωμένη** στην γλώσσα προγραμματισμού, ο μεταγλωττιστής **δεν γνωρίζει απολύτως τίποτε** όσον αφορά στη **σημασιολογία** των συναρτήσεων που περιλαμβάνονται σε αυτή.

Αν και προφανώς **είναι σε θέση να ελέγξει** την **συντακτική τους ορθότητα** και τον **σωστό τρόπο χρήσης τους**, ωστόσο, αυτές οι συναρτήσεις δεν αποτελούν για αυτόν, παρά **μαύρα κουτιά** με **άγνωστο** περιεχόμενο.

Ωστόσο, η υλοποίηση της παράλληλης βιβλιοθήκης **εκτός της γλώσσας προγραμματισμού**, την καθιστά **ανεξάρτητη** από αυτή, υπό την έννοια, πως η **βελτίωση** ή η **επέκτασή** της δεν επηρεάζει **σε τίποτε** τη **δομή** της γλώσσας προγραμματισμού.



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Μοντέλα παράλληλου προγραμματισμού

Στην **δεύτερη** προσέγγιση, η **παράλληλη βιβλιοθήκη** είναι **ενσωματωμένη** στη γλώσσα προγραμματισμού. Δηλαδή,

- είτε **τροποποιούμε** την σύνταξη και την **σημασιολογία** μιας **υφιστάμενης γλώσσας** προγραμματισμού, η οποία **επεκτείνεται** έτσι ώστε να υποστηρίζει την υλοποίηση **παραλλήλων** εφαρμογών (π.χ. **OpenMP**)
- είτε κατασκευάζουμε **από την αρχή** μία **παράλληλη γλώσσα** προγραμματισμού

Αυτή η προσέγγιση απαιτεί την **τροποποίηση** του πηγαίου κώδικα του **μεταγλωττιστή** της υποκείμενης γλώσσας, έτσι ώστε αυτός να είναι σε θέση **να υποστηρίξει τα νέα χαρακτηριστικά**, γεγονός που **απαιτεί** την **διαθεσιμότητα** του πηγαίου κώδικα.

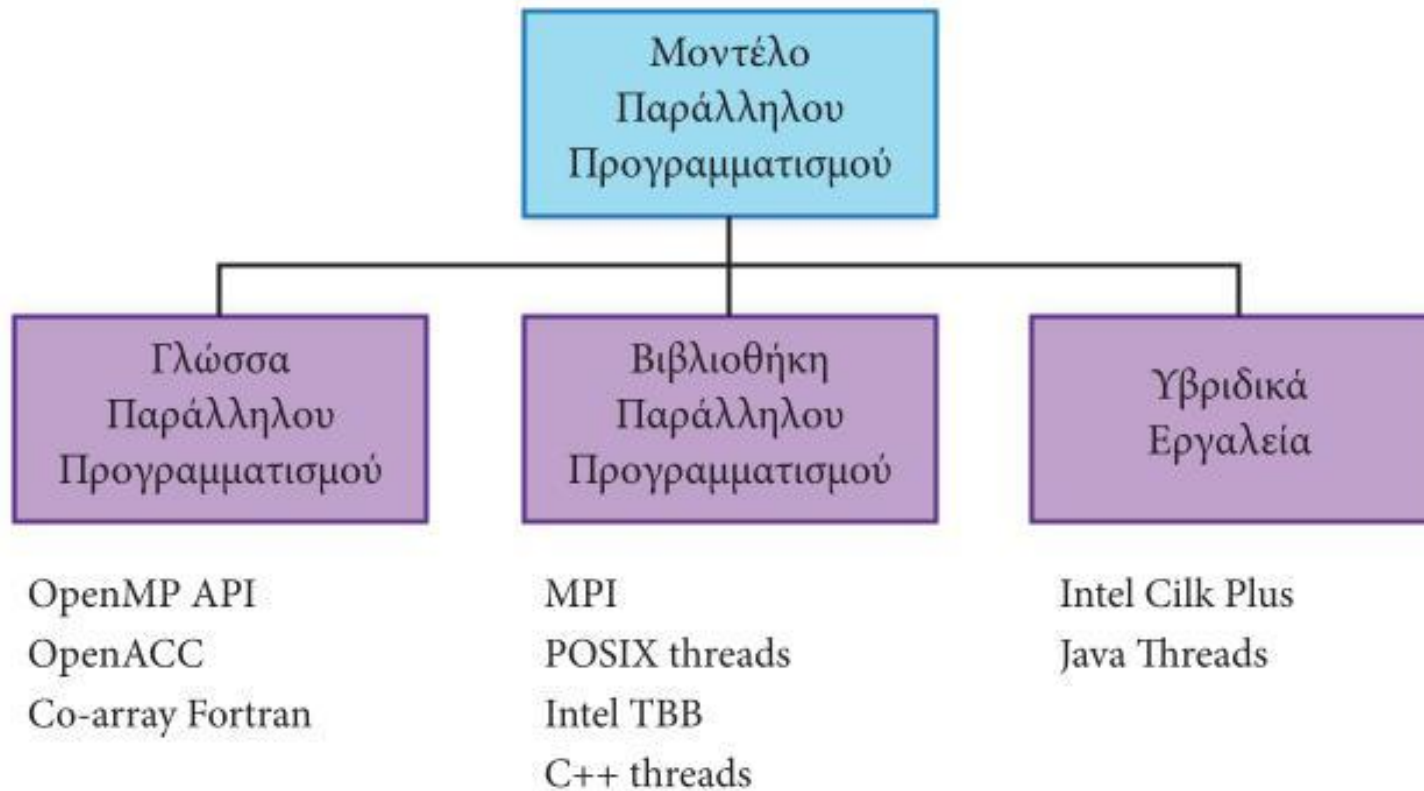
Το **πλεονέκτημα** αυτής της προσέγγισης, είναι πως ο μεταγλωττιστής είναι **ενήμερος** για **όλες τις δομές** που υλοποιούνται στα πλαίσια της ανάπτυξης των παραλλήλων εφαρμογών, τις οποίες και **λαμβάνει υπόψη** κατά την **λεκτική, συντακτική** και **σημασιολογική ανάλυση** του πηγαίου κώδικα.

Τέλος μπορούμε να καταφύγουμε σε μία **υβριδική υλοποίηση**, στην οποία κάποια τμήματα του **μοντέλου** παράλληλου προγραμματισμού **ενσωματώνονται** στην γλώσσα, ενώ κάποια άλλα τμήματα, θα προσφερθούν ως μία βιβλιοθήκη συναρτήσεων, η οποία αναπτύσσεται **εκτός της γλώσσας** (π.χ. Intel Cilk Plus και Java threads).



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Μοντέλα παράλληλου προγραμματισμού



Οι τρόποι υλοποίησης **μοντέλων παράλληλου προγραμματισμού**

Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Μοντέλα παράλληλου προγραμματισμού

Ένα **μοντέλο παράλληλου προγραμματισμού** αποτελεί έναν **τύπο αφαίρεσης** που **απεικονίζει** τις πάσης φύσεως **διαδικασίες** μίας παράλληλης εφαρμογής, στην **δομή** και **οργάνωση** του υποκείμενου παράλληλου συστήματος στο οποίο αυτή εκτελείται.

Αυτή η απεικόνιση **δεν περιορίζεται** σε **ένα** επίπεδο αφαίρεσης, αλλά εκτείνεται σε **πολλά επίπεδα**, που περιλαμβάνουν

- την ίδια την **παράλληλη εφαρμογή** όσον αφορά στις **ιδιότητες** και στα **χαρακτηριστικά** της
- την **γλώσσα προγραμματισμού** στην οποία αυτή έχει υλοποιηθεί
- τον **μεταγλωττιστή** που χρησιμοποιείται για την δημιουργία του εκτελέσιμου αρχείου
- την **επικοινωνία μεταξύ των διεργασιών** δια της χρήσεως του **υποκείμενου δικτυακού εξοπλισμού**
- το **σύστημα / εισόδου εξόδου** που χρησιμοποιείται για **ανάγνωση** και **αποθήκευση** δεδομένων

Κατά καιρούς έχουν εμφανιστεί **πολλά προγραμματιστικά μοντέλα** τα οποία μπορούμε να χρησιμοποιήσουμε για την ανάπτυξη των παράλληλων εφαρμογών μας, τα πιο σημαντικά εκ των οποίων παρουσιάζονται στη συνέχεια.

Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Μοντέλα παράλληλου προγραμματισμού

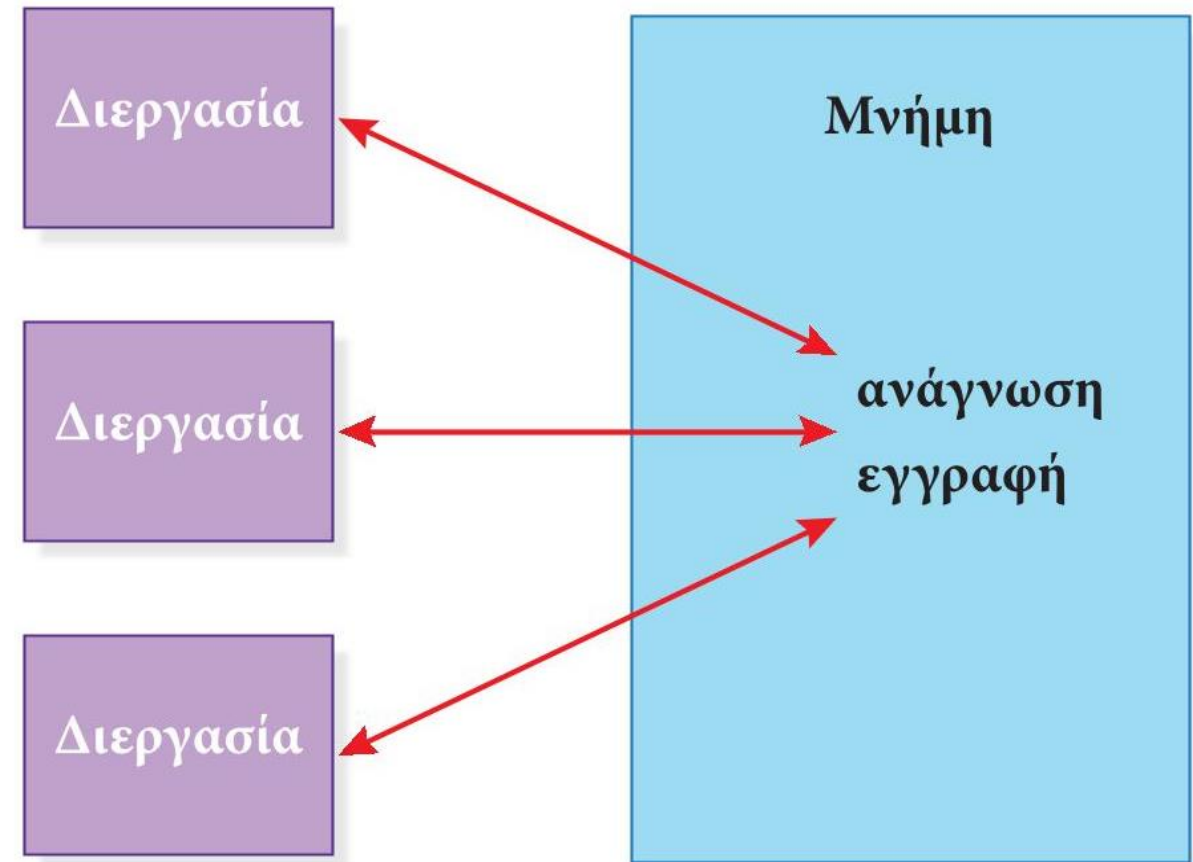
Το μοντέλο κοινόχρηστης μνήμης

Σε αυτό το μοντέλο, οι διεργασίες της εφαρμογής **διαμοιράζονται** έναν **κοινό χώρο διευθύνσεων μνήμης**, ο οποίος επιτρέπει την πραγματοποίηση διαδικασιών **ανάγνωσης** και **εγγραφής** με **ασύγχρονο** τρόπο

Ο **συγχρονισμός** των διεργασιών της εφαρμογής γίνεται με διαδικασίες **κλειδώματος**, χρησιμοποιώντας **σημαφóρους**.

Το **βασικό μειονέκτημα** της προσέγγισης, είναι η **αρκετά μεγάλη δυσκολία** που χαρακτηρίζει τον βαθμό **κατανόησης** και **χρήσης** της έννοιας της **τοπικότητας** των δεδομένων.

Στις υλοποιήσεις αυτού του τύπου, οι **μεταγλωττιστές** του συστήματος **μεταφράζουν** τις μεταβλητές της εφαρμογής του χρήστη σε **πραγματικές διευθύνσεις μνήμης**, έτσι ώστε να τις χρησιμοποιήσουν **μέσα** από την εφαρμογή



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Μοντέλα παράλληλου προγραμματισμού

Το μοντέλο των νημάτων

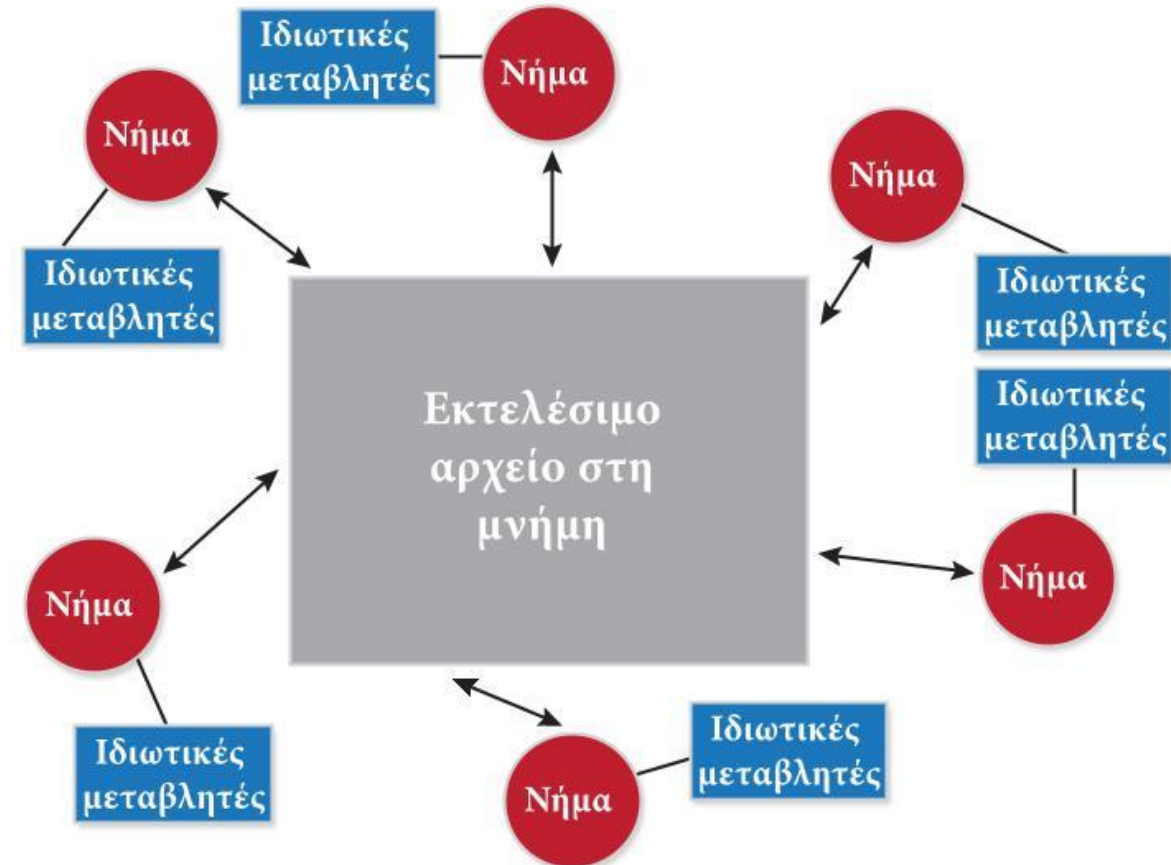
Σε αυτό το μοντέλο η **κάθε** διεργασία αποτελείται από **πολλά νήματα**, τα οποία εκτελούνται **ταυτόχρονα**.

Η **εκτέλεση** του αρχείου της εφαρμογής θα δημιουργήσει το **κατάλληλο πλήθος νημάτων**, το **καθένα** των οποίων θα έχει **τα δικά του** δεδομένα και θα χρησιμοποιεί τους **κοινόχρηστους πόρους** του συστήματος.

Η **επικοινωνία** των νημάτων μπορεί να λάβει χώρα χρησιμοποιώντας **κοινόχρηστη μνήμη** υπό καθεστώς **συγχρονισμού**.

Τα νήματα της εφαρμογής θα ολοκληρωθούν **πριν από το κεντρικό νήμα που τα δημιούργησε**, το οποίο θα παραμείνει στη μνήμη και θα τερματιστεί **τελευταίο**.

Η **υλοποίηση** ενός νήματος στηρίζεται είτε στην χρήση **βιβλιοθήκης συναρτήσεων** είτε στην χρήση μεταγλωττιστή με **πολυνηματικές** δυνατότητες.



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Μοντέλα παράλληλου προγραμματισμού

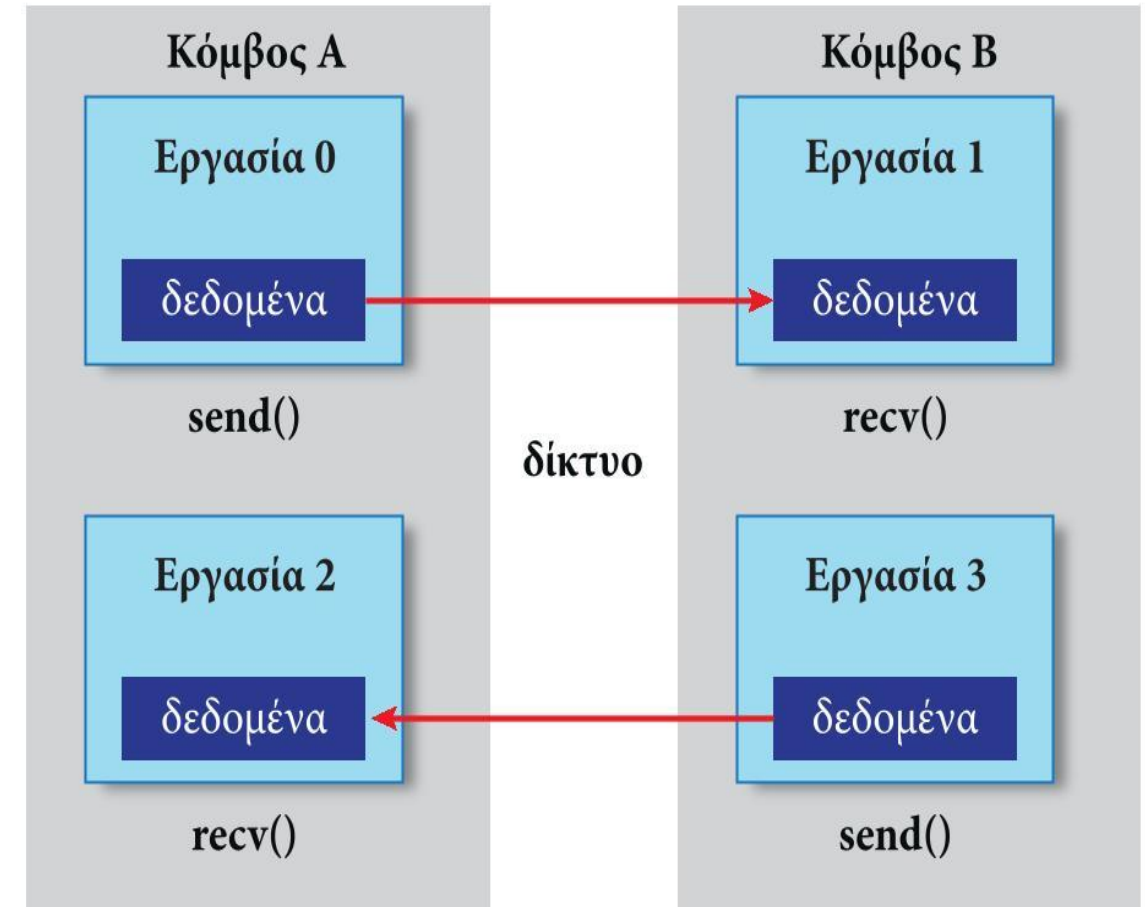
Το μοντέλο μεταβίβασης μηνυμάτων

Το **μοντέλο μεταβίβασης μηνυμάτων** επιτρέπει την ανάπτυξη εφαρμογών **κατανεμημένης μνήμης**.

Η **παράλληλη εφαρμογή** αποτελείται από ένα **σύνολο διεργασιών** που εκτελούνται στον **ίδιο** ή σε **διαφορετικούς** υπολογιστές, η καθεμία από τις οποίες χρησιμοποιεί την **δική της** περιοχή μνήμης και εφαρμόζει τους **δικούς της** υπολογισμούς πάνω στα **δικά της** δεδομένα.

Η **ανταλλαγή** των δεδομένων ανάμεσα στις διεργασίες, στηρίζεται στην **αποστολή** και **παραλαβή** κατάλληλα διαμορφωμένων μηνυμάτων, με την διακίνηση των δεδομένων να απαιτεί την **συνεργασία** των διεργασιών του συστήματος.

Η πιο γνωστή υλοποίηση αυτού του μοντέλου είναι η βιβλιοθήκη του **MPI**



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Μοντέλα παράλληλου προγραμματισμού

Το μοντέλο της εικονικής κοινόχρηστης μνήμης

Το μοντέλο αυτό υλοποιεί μία προγραμματιστική προσέγγιση χρήσης **κοινόχρηστης μνήμης** για την εκτέλεση εφαρμογών που εκτελούνται σε συστήματα **κατανεμημένης μνήμης**.

Στηρίζεται στην έννοια της **εικονικής κοινόχρηστης συσχετιστικής μνήμης** η οποία είναι γνωστή ως **χώρος πλειάδων** και είναι **προσπελάσιμη** από τις διεργασίες του συστήματος με διαδικασίες **εισόδου** και **εξόδου**.

Το παράλληλο μοντέλο αντικειμενοστραφούς προγραμματισμού

Επιτρέπει την ανάπτυξη **παράλληλων εφαρμογών** χρησιμοποιώντας τις έννοιες και τα εργαλεία του **αντικειμενοστραφούς προγραμματισμού**, όπως **αφηρημένα κοινόχρηστα αντικείμενα** που **αλληλεπιδρούν** μεταξύ τους.

Γνωστή γλώσσα προγραμματισμού που υποστηρίζει την δυνατότητα ανάπτυξης **αντικειμενοστραφών παράλληλων εφαρμογών**, είναι η **CC++ (Compositional C++)**, που προσφέρει **έξι επίπεδα αφαίρεσης** που περιγράφονται από ισάριθμες λέξεις κλειδιά και υποστηρίζουν την υλοποίηση παράλληλων εφαρμογών.



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Μοντέλα παράλληλου προγραμματισμού

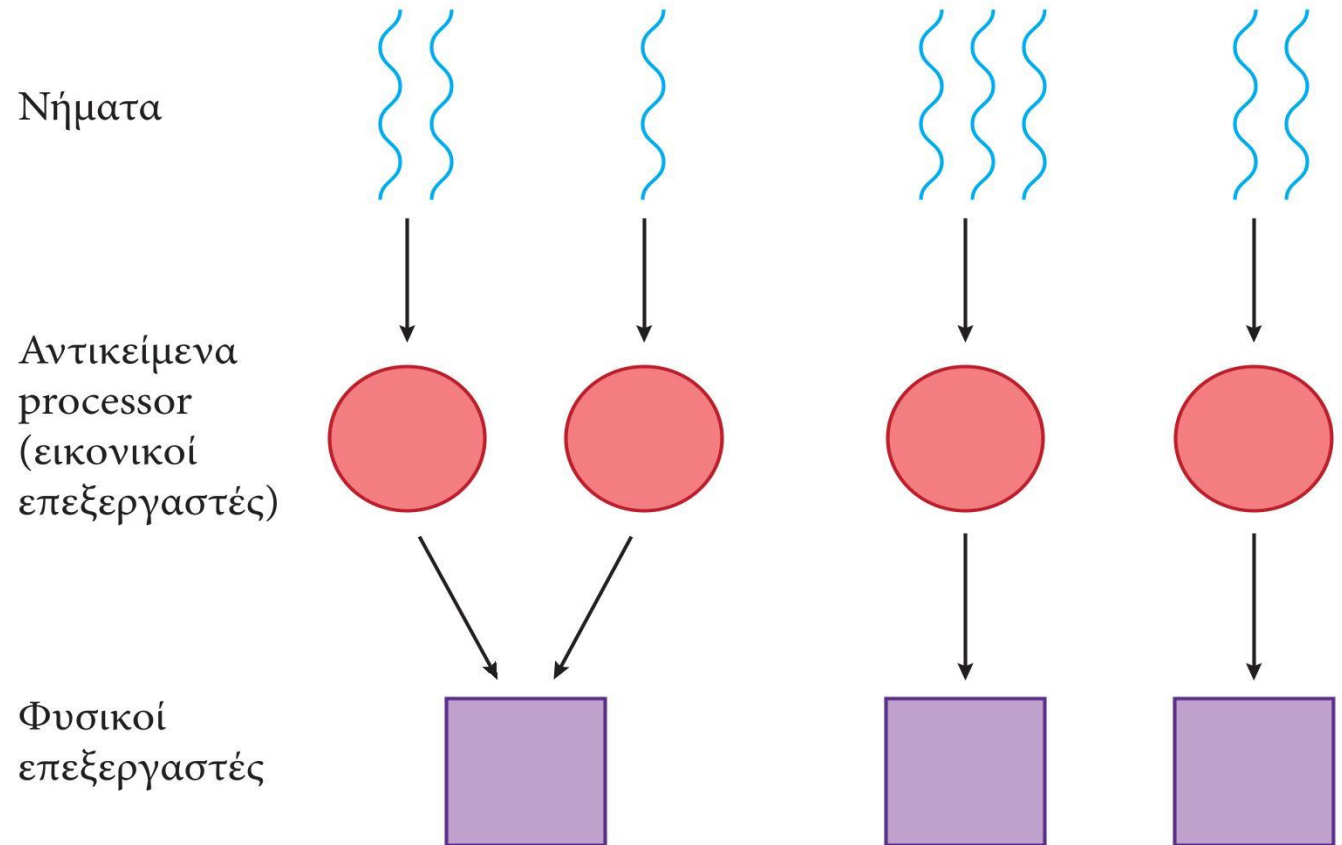
Το παράλληλο μοντέλο αντικειμενοστραφούς προγραμματισμού

Αυτά τα **επίπεδα αφαίρεσης** προσδιορίζονται από τις συμβολοσειρές

- **processor** (έλεγχος τοπικότητας)
- **global** (συνεργασία αντικειμένων)
- **thread** (μηχανισμός ταυτόχρονης εκτέλεσης)
- **sync** (συγχρονισμός νημάτων)
- **atomic** (ατομικές λειτουργίες)
- **ccvoid** (ανταλλαγή δεδομένων)

Τα **αντικειμενοστραφή** μοντέλα παράλληλων εφαρμογών, υποστηρίζουν **δύο** βασικές τεχνικές

1. Τα **ενεργά αντικείμενα**, το καθένα των οποίων διαθέτει το δικό του νήμα ελέγχου και εκτελείται σε οποιονδήποτε επεξεργαστή
2. Τις **κοινόχρηστες περιοχές**, που επιτρέπουν την κοινή χρήση ενός αντικειμένου από μία ομάδα νημάτων της εφαρμογής



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Μοντέλα παράλληλου προγραμματισμού

Το μοντέλο παράλληλων δεδομένων

Σε αυτό το μοντέλο, το **μεγαλύτερο ποσοστό** των παράλληλων διαδικασιών σχετίζεται με την πραγματοποίηση **υπολογισμών** σε **σύνολα δεδομένων**, τα οποία είναι **οργανωμένα** με κάποιο συγκεκριμένο τρόπο.

Η εφαρμογή αποτελείται από ένα **σύνολο διεργασιών** οι οποίες λειτουργούν **συλλογικά** επί της **ίδιας δομής δεδομένων**, με την **κάθε** διεργασία, να χρησιμοποιεί **τα δικά της** δεδομένα που ανήκουν σε **διαφορετική περιοχή** της παραπάνω δομής.

Ωστόσο, **όλες οι διεργασίες** εφαρμόζουν **την ίδια** διαδικασία στο σύνολο των δεδομένων που η **καθεμία** επεξεργάζεται.

Το μοντέλο χρησιμοποιείται τόσο στα συστήματα **κοινόχρηστης** μνήμης, όσο και στα συστήματα **κατανεμημένης** μνήμης.

Στην **πρώτη** περίπτωση, η προσπέλαση των δεδομένων γίνεται δια μέσου της **κοινόχρηστης** μνήμης, ενώ στη δεύτερη περίπτωση, η δομή δεδομένων **διασπάται σε μικρά** τμήματα που αποθηκεύονται στις **τοπικές μνήμες** των κόμβων.

Η χρήση του μοντέλου απαιτεί την **συγγραφή κώδικα** που είτε χρησιμοποιεί **συναρτήσεις παράλληλων βιβλιοθηκών** είτε καταφεύγει στη χρήση **οδηγιών παράλληλων μεταγλωττιστών**.

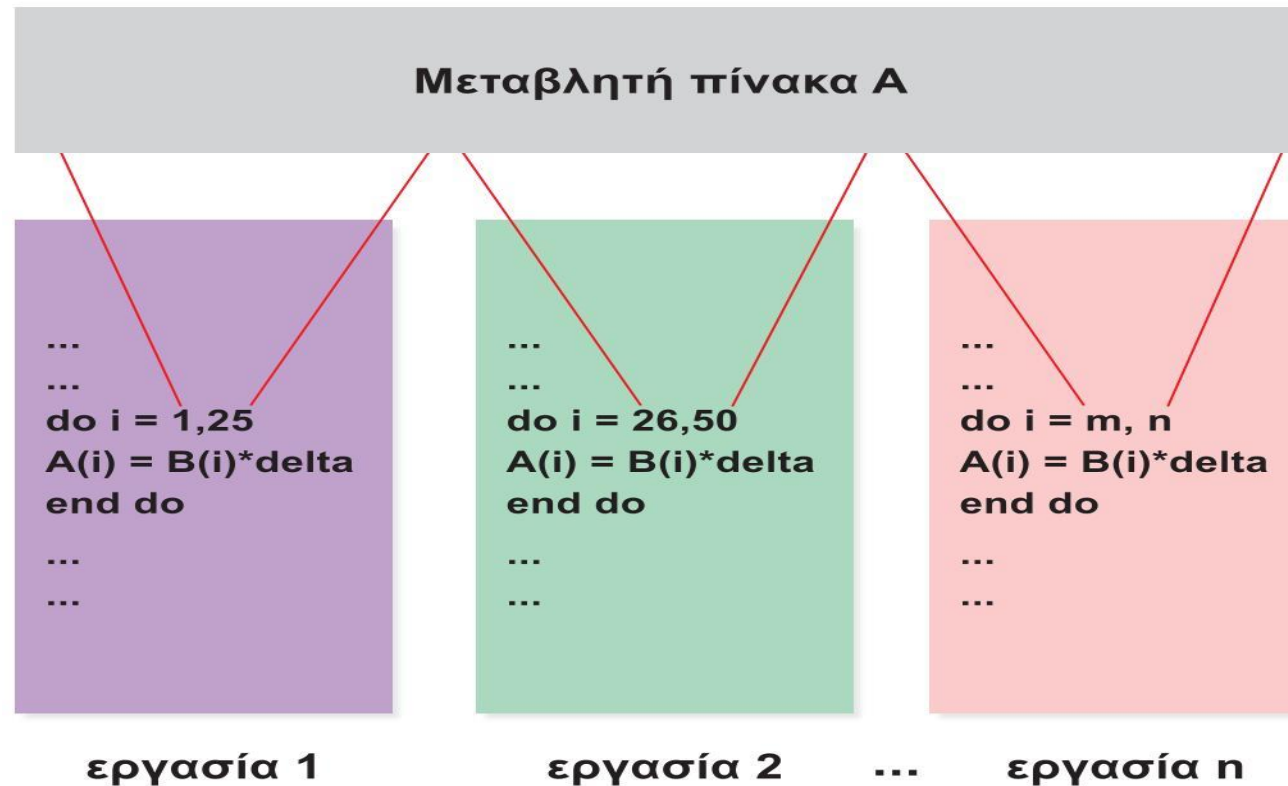
Γνωστή γλώσσα που επιτρέπει την ανάπτυξη τέτοιων εφαρμογών είναι η γλώσσα **HPF (High Performance Fortran)**.



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Μοντέλα παράλληλου προγραμματισμού

Το μοντέλο παράλληλων δεδομένων



Στο **μοντέλο παράλληλων δεδομένων**, το σύνολο των διεργασιών της εφαρμογής εφαρμόζει ταυτόχρονα **την ίδια διαδικασία σε διαφορετικά τμήματα** ενός **κοινόχρηστου** συνόλου δεδομένων.

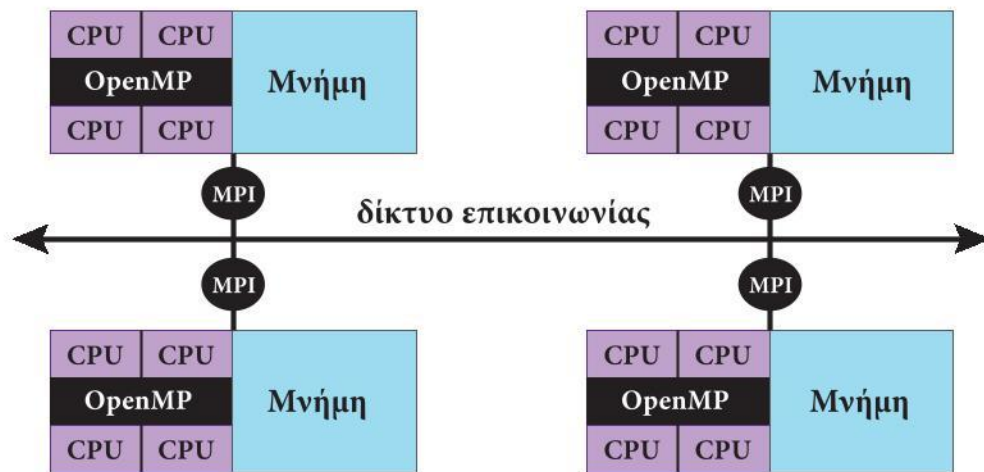
Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Μοντέλα παράλληλου προγραμματισμού

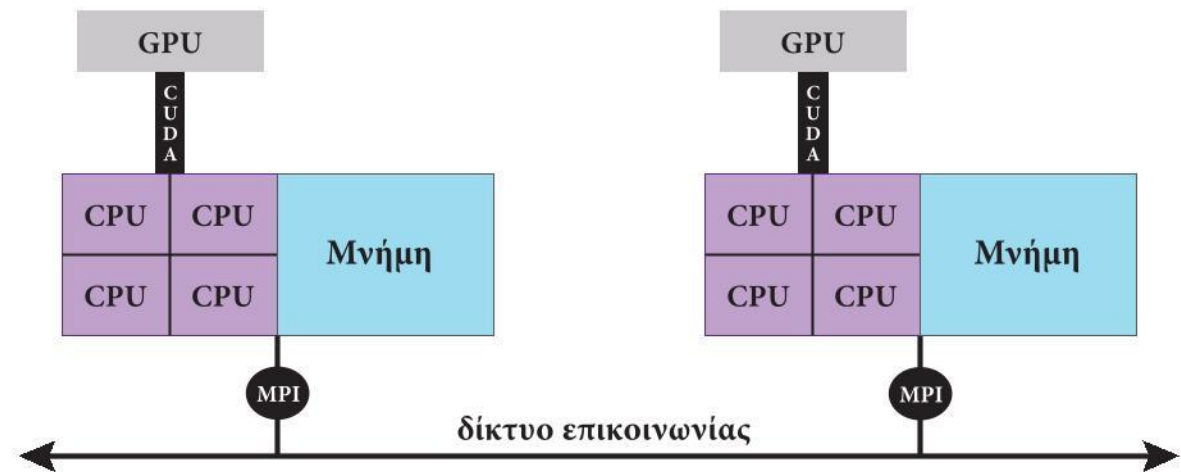
Υβριδικά μοντέλα παράλληλου προγραμματισμού

Προκύπτουν από τον **συνδυασμό** δύο ή περισσότερων από τα προηγούμενα μοντέλα, όπως είναι ο συνδυασμός

- του **μοντέλου μεταβίβασης μηνυμάτων** με το **μοντέλο των νημάτων** ή με το μοντέλο της **κοινόχρηστης μνήμης**
- του **μοντέλου μεταβίβασης μηνυμάτων** με το μοντέλο των **παράλληλων δεδομένων**



Συνδυασμός **MPI** με **OpenMP**



Συνδυασμός **MPI** με **CUDA**

Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Μοντέλα παράλληλου προγραμματισμού

Το μοντέλο SPMD (Single Program Multiple Data)

Αποτελεί **μοντέλο υψηλού επιπέδου** που προκύπτει από τον συνδυασμό **οποιασδήποτε** από τα προηγούμενα μοντέλα.

Η παράλληλη εφαρμογή αποτελείται από **διεργασίες** που **όλες** εκτελούν **το ίδιο** πρόγραμμα και εφαρμόζουν τις **ίδιες** ή **διαφορετικές** εντολές, πάνω στα **ίδια** ή σε **διαφορετικά** δεδομένα

Οι διεργασίες της εφαρμογής μπορούν να χρησιμοποιήσουν **διακλαδώσεις** και να εκτελέσουν **διαφορετικά** τμήματα κώδικα.

Η δομή μιας **παράλληλης εφαρμογής** που ακολουθεί αυτό το μοντέλο, χαρακτηρίζεται από τις επόμενες φάσεις:

- Την **αρχικοποίηση του προγράμματος** και την **ενεργοποίηση του μηχανισμού συγχρονισμού** διεργασιών ή νημάτων.
- Τον ορισμό για κάθε διεργασία ή νήμα ενός **μοναδικού αναγνωριστικού**
- Την **εκτέλεση** του προγράμματος που περιλαμβάνει την **ανάθεση υπολογιστικού έργου**
- Τον **τερματισμό** της λειτουργίας του προγράμματος

Ένα Πρόγραμμα Πολλά Δεδομένα (SPMD)



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Μοντέλα παράλληλου προγραμματισμού

Το μοντέλο MPMD (Multiple Program Multiple Data)

Αποτελεί **μοντέλο υψηλού επιπέδου** που προκύπτει από τον συνδυασμό οποιωνδήποτε από τα προηγούμενα μοντέλα.

Ωστόσο, στην προκειμένη περίπτωση, υπάρχει **η δυνατότητα εκτέλεσης πολλών προγραμμάτων** και επομένως η λειτουργία του απαιτεί την χρήση **πολλών εκτελέσιμων αρχείων** με τις διεργασίες να χρησιμοποιούν τα **ίδια** ή **διαφορετικά** δεδομένα.

Αυτό το μοντέλο προσφέρει τις **μεγαλύτερες δυνατότητες**, αφού υποστηρίζει **σχεδόν όλα** τα σενάρια παράλληλης εκτέλεσης.

Κλασικό παράδειγμα εφαρμογών που χρησιμοποιούν αυτό το μοντέλο είναι οι **εφαρμογές σε CUDA** στις οποίες παρά το γεγονός ότι το τελικό αποτέλεσμα της ανάπτυξης του κώδικα είναι **ένα απλό εκτελέσιμο αρχείο**, ωστόσο, στην πραγματικότητα, περιέχει **δύο** εκτελέσιμα αρχεία, ένα για την **CPU** και έναν για την **GPU**.

Πολλά Προγράμματα Πολλά Δεδομένα (MPMD)



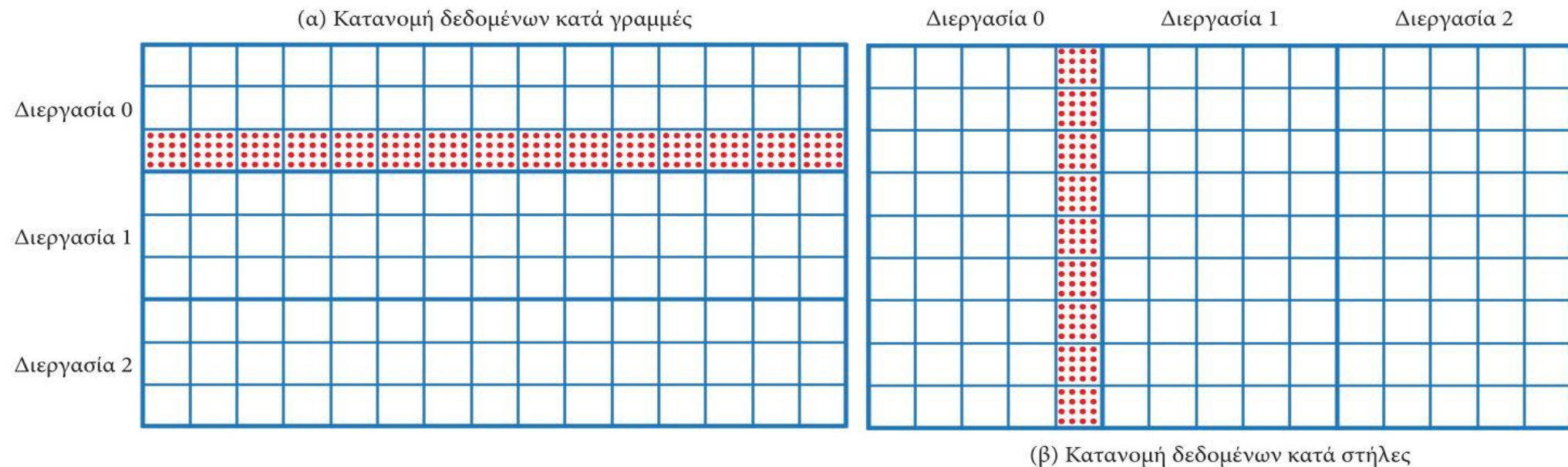
Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Τεχνικές παράλληλου προγραμματισμού

Παραλληλοποίηση εφαρμογών που χρησιμοποιούν πίνακες

Αποτελεί μία από τις πιο **απλές** και **ευρέως χρησιμοποιούμενες** περιπτώσεις παραλληλισμού.

Θεωρώντας τους **συνήθεις δισδιάστατους πίνακες** και υποθέτοντας πως σε κάθε **γραμμή** ή **στήλη** του πίνακα θα εφαρμοσθούν οι ίδιες μορφές επεξεργασίας, μπορούμε να προχωρήσουμε στην **παράλληλη εκτέλεση** των υπολογισμών, **διαμοιράζοντας** με τον κατάλληλο τρόπο τις **γραμμές** ή τις **στήλες** του πίνακα στις διαθέσιμες **διεργασίες** ή **νήματα**



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Τεχνικές παράλληλου προγραμματισμού

Παραλληλοποίηση εφαρμογών που χρησιμοποιούν πίνακες

Η απόφαση σχετικά με τον **τρόπο της διαμέρισης** εξαρτάται από το **πρόβλημα** και από την **γλώσσα προγραμματισμού**.

Για παράδειγμα, αν και στην C η διαμέριση γίνεται **κατά γραμμές**, εάν χρησιμοποιούμε **ανάστροφους** πίνακες είναι προτιμότερο να πραγματοποιηθεί **κατά στήλες**.

Ο **τύπος της διαμέρισης** ενδέχεται να επηρεάσει και ίσως με **δραματικό τρόπο**, την απόδοση της παράλληλης εφαρμογής.

Σε κάποιες περιπτώσεις ίσως χρειαστεί να καταφύγουμε στους κατάλληλους **μετασχηματισμούς βρόχων**.

Τυπικό παράδειγμα **παράλληλης εφαρμογής** που στηρίζεται στην διαμέριση ενός δισδιάστατου πίνακα κατά γραμμές είναι ο **παράλληλος πολλαπλασιασμός πινάκων**

Εναλλακτικά, μπορούμε να διαμερίσουμε τον πίνακα στον **κατάλληλο αριθμό υποπινάκων μικρότερων διαστάσεων** και να αποστείλουμε σε κάθε **διεργασία ή νήμα** τον κατάλληλο σε κάθε περίπτωση **υποπίνακα** (π.χ. αλγόριθμος του Fox).



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Τεχνικές παράλληλου προγραμματισμού

Παραλληλοποίηση εφαρμογών που χρησιμοποιούν πίνακες

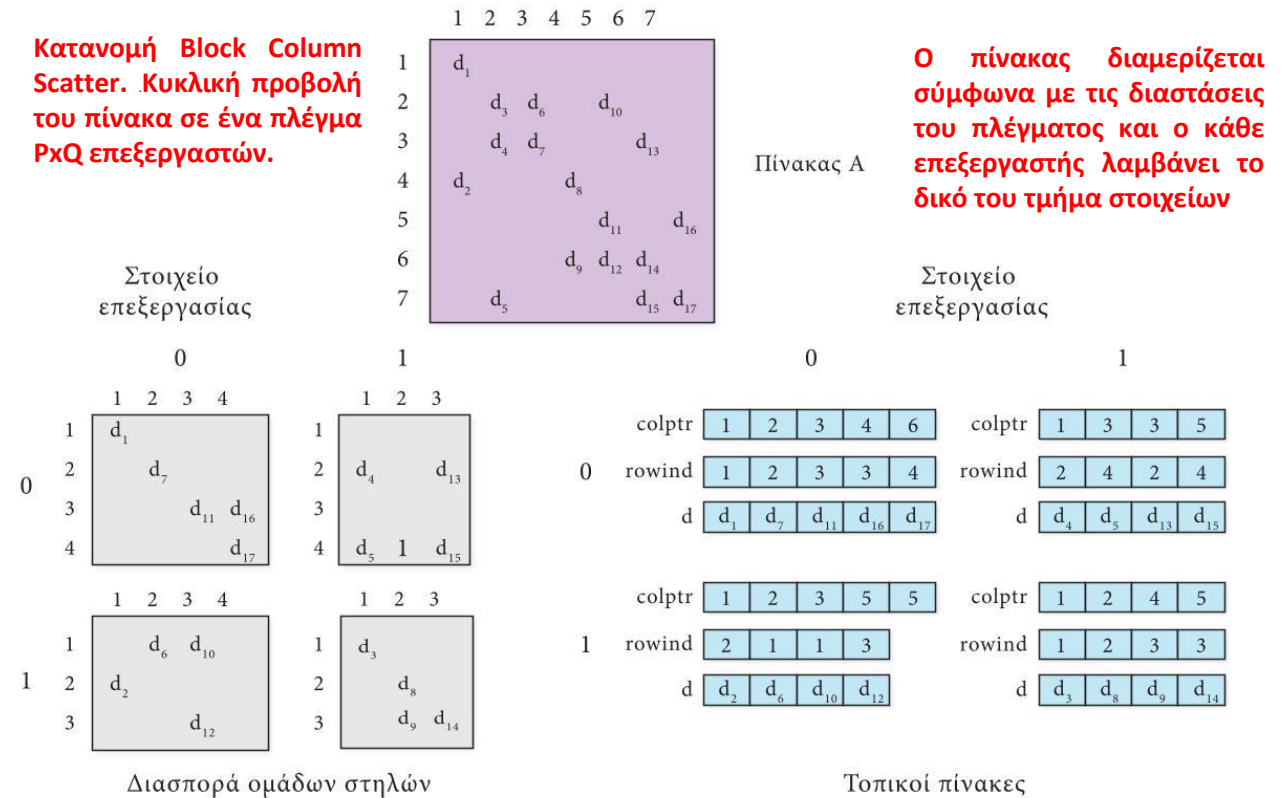
Στην ειδική περίπτωση των **αραιών πινάκων**, στους οποίους οι τιμές των περισσότερων στοιχείων τους είναι ίσες με το μηδέν, τα **μόνα στοιχεία** που αποθηκεύονται, είναι τα **N** το πλήθος **μη μηδενικά** στοιχεία. Αυτό γίνεται με δύο τρόπους:

1) μέσω χρήσης πίνακα διαστάσεων $3 \times N$, με το 1^ο και το 2^ο κελί της υπ' αριθμόν i στήλης να περιέχουν την **γραμμή** και την **στήλη** του **αραιού** πίνακα που περιέχει το **μη μηδενικό** στοιχείο και το 3^ο κελί να περιέχει την τιμή αυτού του στοιχείου

2) οι πληροφορίες αποθηκεύονται στους **κόμβους** μιας **συνδεδεμένης λίστας**.

Στην περίπτωση αυτή, η **κατανομή** των στοιχείων των **αραιών** πινάκων στις διαθέσιμες διεργασίες, **δεν** εφαρμόζεται στους **αραιούς** πίνακες αλλά στις **ισοδύναμες αναπαραστάσεις** τους, οι οποίες περιέχουν μόνο τα **μη μηδενικά** στοιχεία

Κατανομή Block Column Scatter. Κυκλική προβολή του πίνακα σε ένα πλέγμα $P \times Q$ επεξεργαστών.



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Τεχνικές παράλληλου προγραμματισμού

Παραλληλοποίηση απλών και ένθετων επαναληπτικών βρόχων

Το πιο γνωστό παράδειγμα **παραλληλοποίησης** αφορά στους **βρόχους επανάληψης** με το ζήτημα να έχει **τεράστια πρακτική αξία** αφού οι βρόχοι αποτελούν τα πιο **απαιτητικά** τμήματα κώδικα, από πλευράς **χρόνου εκτέλεσης** και **υπολογισμών**. Σε πολλές περιπτώσεις η **διαμέριση** και η **παραλληλοποίηση** του βρόχου γίνεται **αυτόματα** από τον **μεταγλωττιστή**.

Προϋπόθεση για την παραλληλοποίηση του βρόχου είναι η **ανεξαρτησία** των επαναλήψεων, κάτι που **ισχύει** για τον βρόχο

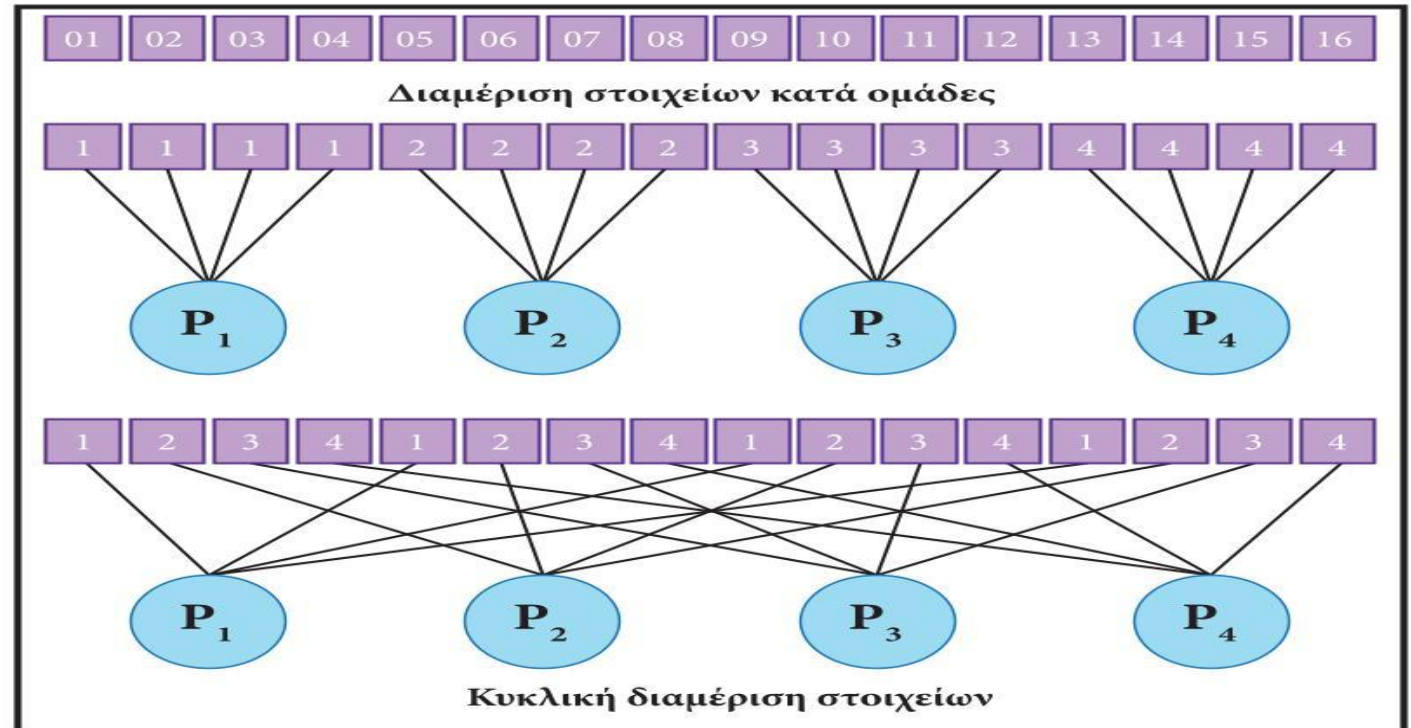
```
for (i=0;i<N;i++)  
    C[i] = A[i] + B[i];
```

όχι όμως και για τον βρόχο

```
for (i=2;i<N;i++)  
    A[i] = A[i-1] + A[i-2];
```



Οι δύο τρόποι
διαμέρισης των
επαναλήψεων
του πρώτου
βρόχου



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Τεχνικές παράλληλου προγραμματισμού

Παραλληλοποίηση απλών και ένθετων επαναληπτικών βρόχων

Στην περίπτωση **ένθετων βρόχων** βάθους $d = n$, η **αυτόματη** παραλληλοποίησή τους είναι **αρκετά πολύπλοκη** και **δεν επαρκούν** τα εργαλεία του μεταγλωττιστή που αναπαριστούν **δηλώσεις**, όπως είναι τα **αφηρημένα συντακτικά δέντρα** κ.λπ..

Αυτή η **αδυναμία** οφείλεται στο ότι γεγονός πως σε έναν **ένθετο** βρόχο ο παραλληλισμός **δεν αφορά σε δηλώσεις αλλά σε στιγμιότυπα δηλώσεων**, οι οποίες σχετίζονται με τις **επαναλήψεις** του βρόχου και στηρίζεται σε **πληροφορίες** όπως είναι ο **τρόπος προσπέλασης** της μνήμης και **ενημέρωσης** του **δείκτη** του βρόχου σε **κάθε** επανάληψη.

Η **παραλληλοποίηση** των **ένθετων** βρόχων μοιάζει **πάρα πολύ** με την διαδικασία της **σύνθεσης συστολικών συστοιχιών**.

Η **βασική** διαφορά ανάμεσα σε αυτές τις δύο μεθόδους, να έχει να κάνει με το γεγονός, πως στην πρώτη περίπτωση οι δηλώσεις των βρόχων **υφίστανται ξεχωριστή επεξεργασία η μία μετά την άλλη**, ενώ στην δεύτερη περίπτωση, το **σώμα** του βρόχου θεωρείται στο σύνολό του ως ένα **ενιαίο τμήμα** κώδικα.

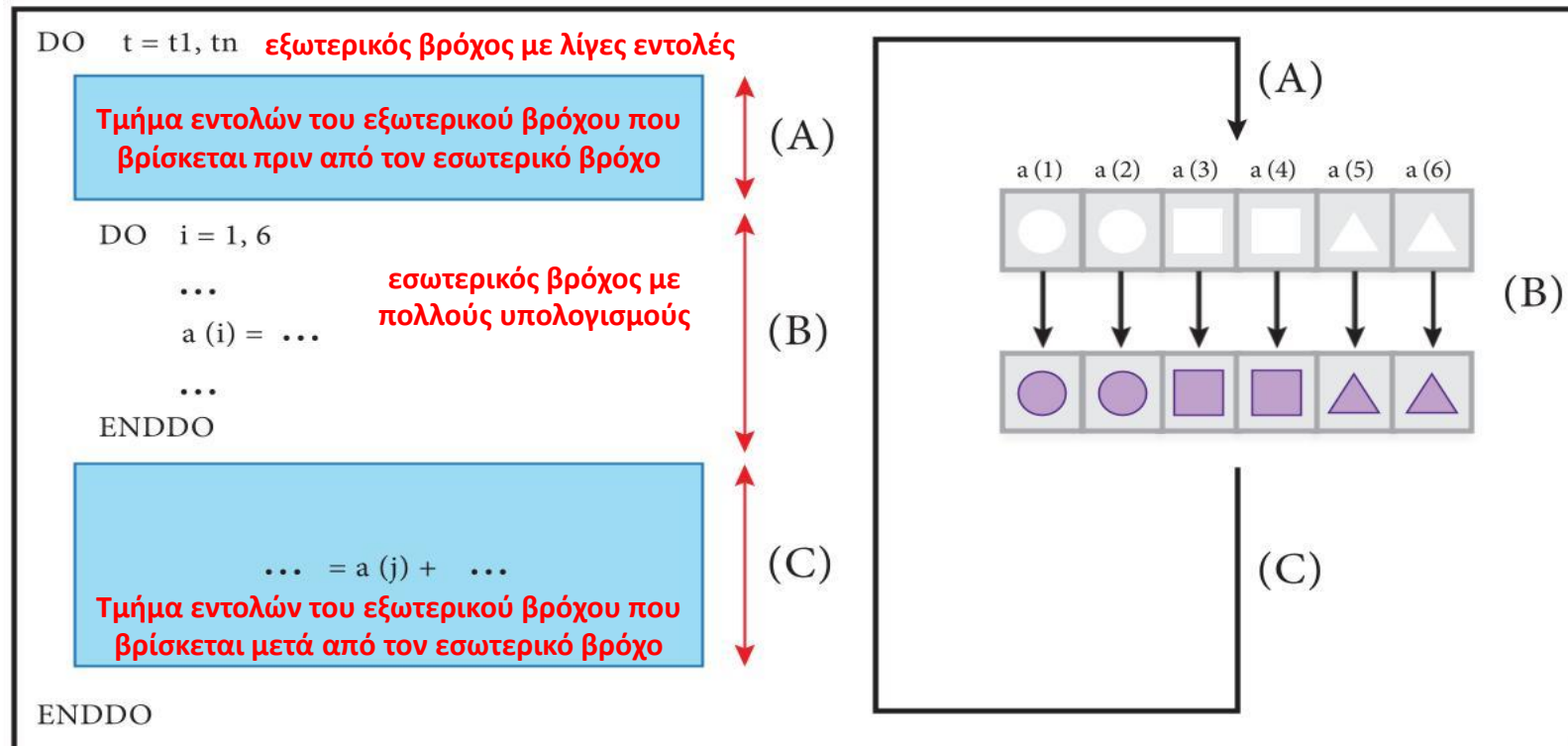


Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Τεχνικές παράλληλου προγραμματισμού

Παραλληλοποίηση απλών και ένθετων επαναληπτικών βρόχων

Παραλληλοποίηση διπλού βρόχου με έναν εξωτερικό και έναν εσωτερικό βρόχο



Παρατηρώντας πως τα στοιχεία του πίνακα **a** ενημερώνονται στο τμήμα **B** και χρησιμοποιούνται στο τμήμα **C** θα πρέπει να **ληφθεί μέριμνα**, ώστε οι **εντολές** της περιοχής **C** να χρησιμοποιούν τις **σωστές**, **ενημερωμένες** τιμές των στοιχείων του πίνακα **a**.

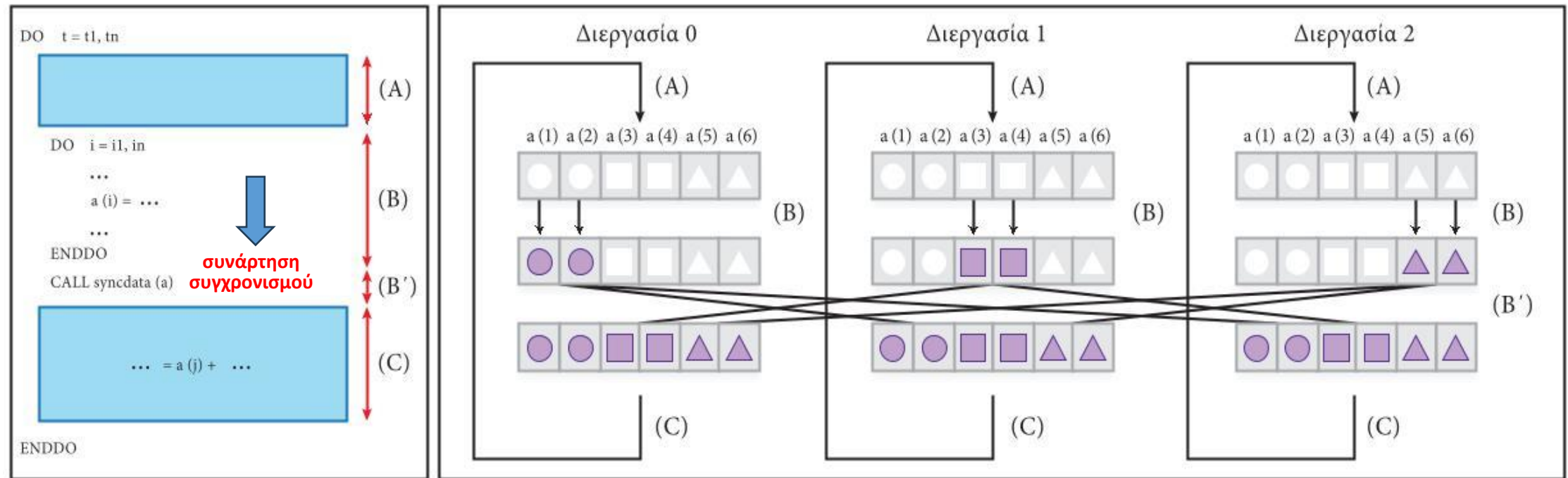
Σειριακή εκτέλεση **διπλού βρόχου** με έναν **εξωτερικό βρόχο** και έναν **εσωτερικό βρόχο**.

Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Τεχνικές παράλληλου προγραμματισμού

Παραλληλοποίηση απλών και ένθετων επαναληπτικών βρόχων

Παραλληλοποίηση διπλού βρόχου με έναν εξωτερικό και έναν εσωτερικό βρόχο



Η κάθε διεργασία θα εκτελέσει **μόνο το κατάλληλο πλήθος επαναλήψεων**

Η **συνάρτηση συγχρονισμού** ενημερώνει τις υπόλοιπες διεργασίες για τις νέες τιμές των **στοιχείων** του πίνακα

Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Τεχνικές παράλληλου προγραμματισμού

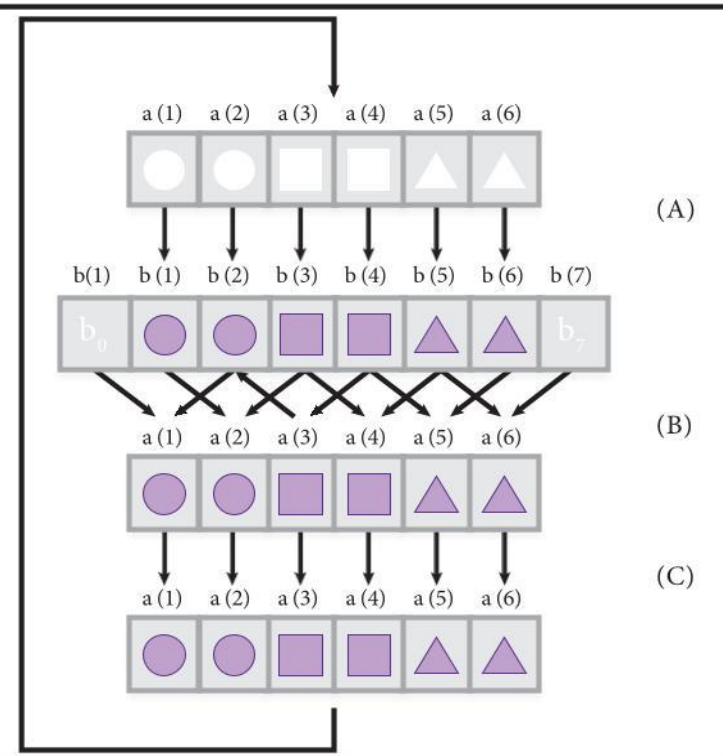
Παραλληλοποίηση απλών και ένθετων επαναληπτικών βρόχων

Παραλληλοποίηση διπλού βρόχου με έναν εξωτερικό και πολλούς εσωτερικούς βρόχους

Στην προκειμένη περίπτωση, η κλήση της συνάρτησης συγχρονισμού μετά από την ολοκλήρωση της λειτουργίας του κάθε βρόχου οδηγεί σε απαράδεκτα μεγάλη χρονική καθυστέρηση.

```
DO t = t1, tn
  DO i = 1, 6
    b(i) = b(i) + a(i)
  ENDDO
  DO i = 1, 6
    a(i) = b(i-1) + b(i+1)
  ENDDO
  DO i = 1, 6
    a(i) = a(1) + 1.0
  ENDDO
ENDDO
```

(A)
(B)
(C)



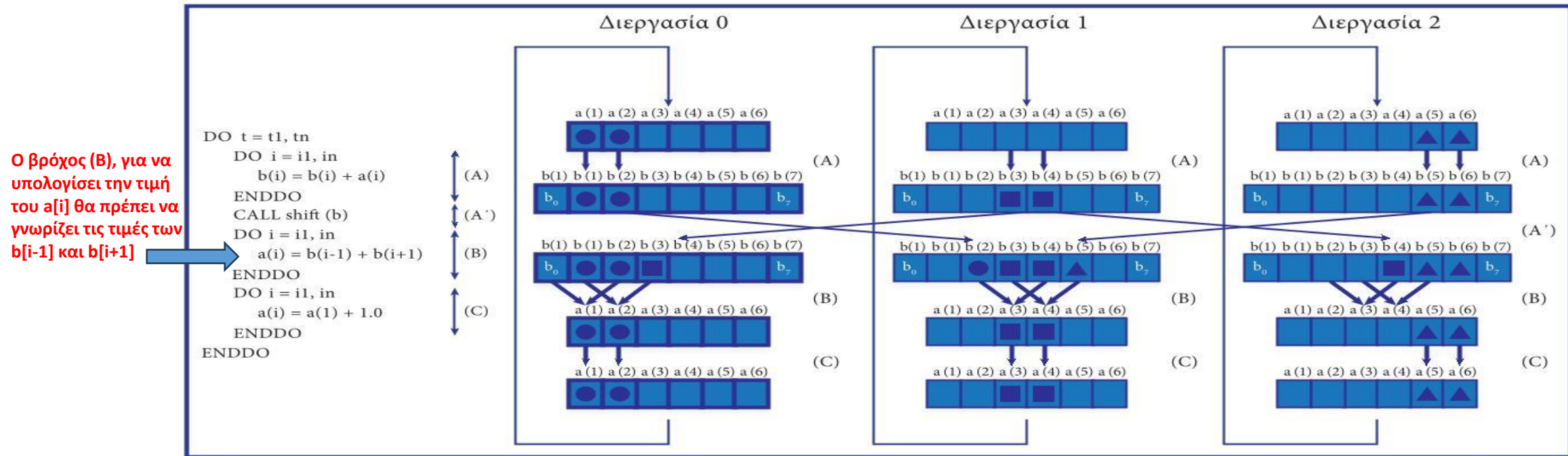
Σειριακή εκτέλεση διπλού βρόχου με έναν εξωτερικό βρόχο και τρεις εσωτερικούς βρόχους.

Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Τεχνικές παράλληλου προγραμματισμού

Παραλληλοποίηση απλών και ένθετων επαναληπτικών βρόχων

Παραλληλοποίηση διπλού βρόχου με έναν εξωτερικό και πολλούς εσωτερικούς βρόχους



Η κάθε διεργασία **δεν χρειάζεται** τις τιμές των a και b εκτός της δικής της περιοχή τιμών με εξαίρεση τον **βρόχο (B)**. Κατά συνέπεια θα πρέπει να **ανταλλάξουμε** τις τιμές των στοιχείων που βρίσκονται **στα όρια των περιοχών τιμών** στις οποίες **λειτουργεί** η κάθε διεργασία, μέσω κλήσεως **συνάρτησης μετατόπισης**.

Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Τεχνικές παράλληλου προγραμματισμού

Παραλληλοποίηση απλών και ένθετων επαναληπτικών βρόχων

Βρόχοι **DOALL** και βρόχοι **DOACROSS**

Υπάρχουν **δύο κατηγορίες βρόχων** όσον αφορά στον τρόπο με τον οποίο λαμβάνει χώρα η **παραλληλοποίησή** τους.

Βρόχοι τύπου DOALL: **δεν μεταφέρουν** εξαρτήσεις και κατά συνέπεια, **παραλληλοποιούνται πλήρως**, κατανέμοντας τις επαναλήψεις τους με αυθαίρετο τρόπο στους **διαθέσιμους επεξεργαστές** χωρίς να λαμβάνεται μέριμνα για την υλοποίηση μηχανισμών **συγχρονισμού**, αφού λόγω της **πλήρους απουσίας** εξαρτήσεων, αυτοί **δεν** είναι αναγκαίοι.

Βέβαια, εάν οι **βρόχοι τύπου DOALL**, βρίσκονται **μέσα** σε άλλους βρόχους οι οποίοι **μεταφέρουν εξαρτήσεις**, ή πριν ή μετά από δηλώσεις, οι οποίες εκτελούνται με **σειριακό τρόπο**, απαιτείται η χρήση τέτοιων μηχανισμών

Βρόχοι τύπου DOACROSS: **μεταφέρουν** εξαρτήσεις, αν και μέσω των κατάλληλων μεθόδων, μπορούν να **διαμοιραστούν** στους **παράλληλους επεξεργαστές**, με τίμημα την **μείωση της απόδοσης** κάτι που **δεν ισχύει** για τους βρόχους **DOALL**.

Άλλοι τύποι βρόχου είναι οι βρόχοι **DOSERIAL** που περιγράφουν τους **συνήθεις σειριακούς βρόχους**, οι βρόχοι **DOCONSIDER** καθώς και αυτοί που εκτελούνται **υπό καθεστώς εικασίας**, των οποίων οι αποφάσεις για την παραλληλοποίηση **δεν λαμβάνονται στατικά** κατά την φάση της **μεταγλώττισης**, αλλά **δυναμικά**, κατά φάση της **εκτέλεσής** τους.



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

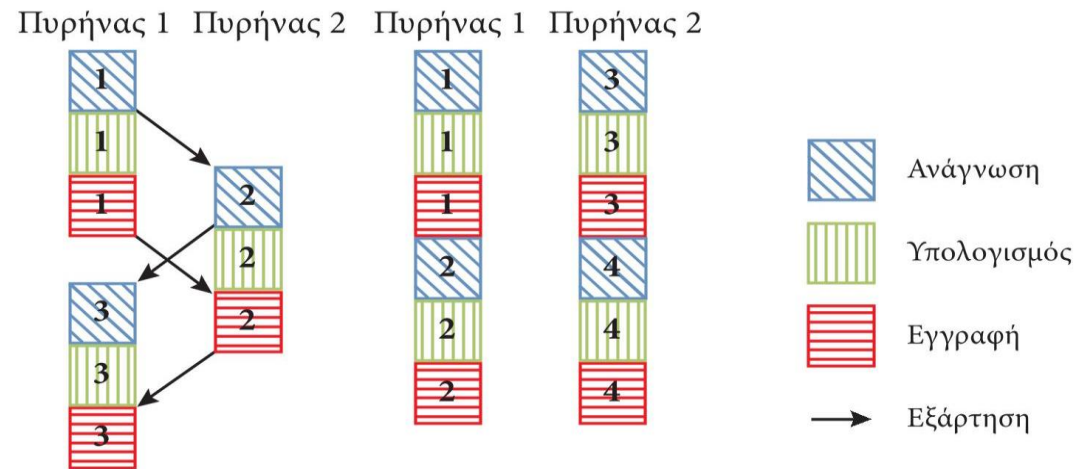
Τεχνικές παράλληλου προγραμματισμού

Παραλληλοποίηση απλών και ένθετων επαναληπτικών βρόχων

Βρόχοι **DOALL** και βρόχοι **DOACROSS**

```
while( !eof(input) ) {  
    buffer = read(input);  
    result = compute(buffer);  
    write(output, result);  
}
```

(α) Κώδικας βρόχου



(β) Βρόχος DOACROSS (γ) Βρόχος DOALL

Στους βρόχους **τύπου DOALL**, οι τρεις φάσεις εκτελούνται **διαδοχικά** για την κάθε επανάληψη, με τις επαναλήψεις του βρόχου να εκτελούνται **η μία μετά την άλλη** και με τον κάθε επεξεργαστή να εκτελεί **την δική του** ομάδα επαναλήψεων, ενώ στους βρόχους τύπου **DOACROSS**, η ύπαρξη **εξαρτήσεων (εισόδου / εξόδου)** που **μεταφέρονται** από τον βρόχο οδηγεί σε **χρονικές υστερήσεις** και σε **μεγαλύτερη χρονική διάρκεια εκτέλεσης**, λόγω των διαδικασιών **συγχρονισμού**

Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Τεχνικές παράλληλου προγραμματισμού

Παραλληλοποίηση απλών και ένθετων επαναληπτικών βρόχων

Βαθμός παραλληλισμού και βαθμός επιμερισμού υπολογιστικού έργου

Στους βρόχους **τύπου DOALL**, ο **βαθμός του παραλληλισμού** είναι ίσος με το **συνολικό πλήθος των επαναλήψεων**, αφού όλες οι επαναλήψεις είναι **ανεξάρτητες** μεταξύ τους και **δυνητικά** μπορούν να εκτελεστούν **παράλληλα**.

Εάν θεωρήσουμε **ξεχωριστά** τον κάθε βρόχο μίας ένθετης δομής, ο **επιμερισμός** των επαναλήψεων κατά την διεύθυνση ενός βρόχου **τύπου DOALL**, προτιμάται έναντι **οποιουδήποτε άλλου** διαφορετικού επιμερισμού, αφού **δεν απαιτείται** η πραγματοποίηση μέσα στο βρόχο καμίας διαδικασίας **επικοινωνίας** ή **συγχρονισμού**.

Στους βρόχους **τύπου DOACROSS**, ο βαθμός του παραλληλισμού **περιορίζεται** από τις διαδικασίες συγχρονισμού τύπου χρονικής φραγής, που είναι αναγκαίες για την σωστή λειτουργία του βρόχου (στην **χειρότερη δυνατή** περίπτωση, ο **βαθμός του παραλληλισμού** είναι ίσος με τη **μονάδα**, κάτι που ουσιαστικά σημαίνει πως **δεν υφίσταται** καθόλου παραλληλισμός).

Σε κάθε περίπτωση, ο στόχος είναι η επίτευξη του **μέγιστου δυνατού βαθμού παραλληλισμού**, κάτι που γίνεται μεταφέροντας τον **μέγιστο δυνατό όγκο υπολογισμών**, οι οποίοι επιδέχονται **παραλληλοποίηση**, σε βρόχους οι οποίοι βρίσκονται σε **όσο το δυνατόν υψηλότερα επίπεδα** της ένθετης επαναληπτικής δομής (και στην ιδανική περίπτωση, στον **πλέον εξωτερικό** βρόχο).



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Τεχνικές παράλληλου προγραμματισμού

Παραλληλοποίηση απλών και ένθετων επαναληπτικών βρόχων

Βαθμός παραλληλισμού και βαθμός επιμερισμού υπολογιστικού έργου

Επειδή ανάμεσα στις επαναλήψεις ενός **παράλληλου** βρόχου δεν απαιτείται συγχρονισμός, η τοποθέτηση των παράλληλων βρόχων στα **υψηλότερα δυνατά επίπεδα**, οδηγεί στην **μείωση** της συχνότητας **εφαρμογής** των διαδικασιών αυτού του τύπου.

Ο **στόχος** των τεχνικών παραλληλοποίησης, είναι η πραγματοποίηση από κάθε βρόχο όσο το δυνατόν λιγότερων προσπελάσεων στην **κοινόχρηστη μεταβλητή πίνακα**, η οποία οδηγεί στην **μείωση** της πιθανότητας εμφάνισης **συγκρουόμενων** μεταξύ τους **κατανομών δεδομένων** και **υπολογισμών** στους υποκείμενους επεξεργαστές.

Η **επεξεργασία** ενός ένθετου βρόχου έτσι ώστε αυτός να **παραλληλοποιηθεί** στον **μέγιστο δυνατό βαθμό**, στηρίζεται στην εφαρμογή επί του βρόχου των **κατάλληλων** σε κάθε περίπτωση **μετασχηματισμών**.

Ο **βαθμός επιμερισμού υπολογιστικού έργου**, ορίζεται ως **η ποσότητα των υπολογισμών** που πραγματοποιούνται στα πλαίσια της εκτέλεσης της **κάθε** επανάληψης του παράλληλου βρόχου.

Εάν υπάρχει βρόχος **τύπου DOALL** που αποτελεί τμήμα ενός **μεγαλύτερου τέλει ένθετου βρόχου**, ο **βαθμός του παραλληλισμού** για αυτόν υπαγορεύεται **από την θέση του** μέσα σε αυτή την ένθετη δομή, ενώ η **μεταφορά** του βρόχου σε υψηλότερα επίπεδα προκαλεί την **αύξηση** του **βαθμού επιμερισμού του υπολογιστικού έργου**.



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Τεχνικές παράλληλου προγραμματισμού

Παραλληλισμός μέσω διοχέτευσης

Ο **παραλληλισμός μέσω ταυτοχρονισμού** (**χωρικός** παραλληλισμός) στηρίζεται στην **ταυτόχρονη εκτέλεση** υπολογισμών σε σύνολα επεξεργαστών **κατανεμημένων** στην επιφάνεια ενός ολοκληρωμένου κυκλώματος ή σε έναν μεγαλύτερο χώρο.

Αντίθετα, ο **παραλληλισμός μέσω διοχέτευσης** (**χρονικός** παραλληλισμός), υλοποιείται μέσω **ενός συνόλου βαθμίδων** που λειτουργούν ως **φίλτρα** λαμβάνοντας **εισόδους** από **διαφορετικές δομές δεδομένων** και είτε δημιουργούν την **δική τους** έξοδο, είτε **μεταβιβάζουν** αυτά τα δεδομένα στις **επόμενες βαθμίδες**, όπως συμβαίνει σε έναν **συστολικό επεξεργαστή**.

Η **αρχή λειτουργίας** του μοντέλου είναι αυτή που χαρακτηρίζει τον **ομώνυμο μηχανισμό** που χρησιμοποιείται σε έναν **επεξεργαστή** για την **επιτάχυνση** της εκτέλεσης των εντολών, ενώ η **επικοινωνία δεδομένων** πραγματοποιείται ανάμεσα σε **γειτονικές** βαθμίδες, με τον **ρυθμό πραγματοποιήσεως** των υπολογισμών να **αντισταθμίζει** το **εύρος ζώνης** της επικοινωνίας.

Κυρίαρχο ρόλο στο μοντέλο κατέχει η έννοια του **ρεύματος δεδομένων** που δημιουργείται μέσω της **υποδιαίρεσης** των δεδομένων σε **ομάδες δεδομένων**, τα οποία **ανταλλάσσονται** ανάμεσα στις βαθμίδες της διοχέτευσης.



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Τεχνικές παράλληλου προγραμματισμού

Παραλληλισμός μέσω διοχέτευσης

Πλεονεκτήματα που χαρακτηρίζουν αυτόν τον τύπο παράλληλων υπολογισμών, περιλαμβάνουν:

- την δυνατότητα **επικοινωνίας** ενός κόμβου μόνο με τον **προηγούμενο** και με τον **επόμενο** κόμβο
- την δυνατότητα της **χρονικής επικάλυψης** υπολογισμών
- την δυνατότητα επίτευξης **υψηλού βαθμού επιμερισμού** υπολογισμών
- την δυνατότητα **ασύγχρονης λειτουργίας** διά της χρήσεως των κατάλληλων εντολών **send** και **recv**
- την **ανεξαρτησία** του συστήματος από το **μέγεθος** των δεδομένων

Σε περιπτώσεις κατά τις οποίες **δεν υφίσταται χρονική επικάλυψη** ανάμεσα στις διαδικασίες **υπολογισμού** και **επικοινωνίας**, η **επιβάρυνση λόγω επικοινωνίας** αποτελεί μία **χρονική υστέρηση**, η οποία συνίσταται από

- το **χρονικό διάστημα** που αφιερώνεται στην κλήση των διαδικασιών **send** και **recv**
- το **χρονικό διάστημα ενεργοποίησης** του συστήματος **κατανεμημένης** μνήμης
- το **χρονικό διάστημα μεταφοράς των δεδομένων** από και προς τις **ενδιάμεσες** μνήμες του συστήματος.

Στην αντίθετη περίπτωση αυτή η επιβάρυνση δεν είναι τίποτε άλλο, από το **χρονικό διάστημα αποστολής** του μηνύματος.



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Τεχνικές παράλληλου προγραμματισμού

Παραλληλισμός μέσω διοχέτευσης

Εάν το σύστημα αποτελείται από έναν **κεντρικό επεξεργαστή** και από ένα σύνολο **κόμβων** που **παραλαμβάνουν** δεδομένα από αυτόν, πραγματοποιούν **υπολογισμούς** και **επιστρέφουν** τα δεδομένα πίσω σε αυτόν, μπορούμε να **μοντελοποιήσουμε** την **επιβάρυνση λόγω επικοινωνίας**, τόσο για την περίπτωση της **επικάλυψης** όσο και για την περίπτωση της **μη επικάλυψης**.

Η επιβάρυνση που σχετίζεται με τον κεντρικό υπολογιστή είναι $\sigma_h + \tau_h \times s$, όπου σ_h είναι η **χρονική υστέρηση** που σχετίζεται με την διαδικασία **αρχικοποίησης** του μηνύματος, s είναι το **μέγεθος** του μηνύματος σε bytes, ενώ τ_h είναι ο **χρόνος μετάδοσης** ενός **απλού byte** για τα μηνύματα που δημιουργούνται στον **κεντρικό επεξεργαστή**.

Η αντίστοιχη **επιβάρυνση** που σχετίζεται με τους **κόμβους** είναι αντίστοιχα $\sigma_n + \tau_n \times s$, με τα μεγέθη που εμφανίζονται σε αυτή την έκφραση, να είναι τα ίδια με πριν, αλλά αυτή τη φορά να αναφέρονται στους **κόμβους**.

Εάν συμβολίσουμε με σ'_h , τ'_h , σ'_n , και τ'_n τα **αντίστοιχα** μεγέθη, για την περίπτωση της **επικάλυψης**, ο **χρόνος αποστολής** ενός μηνύματος μεγέθους s bytes από τον **κεντρικό επεξεργαστή** προς έναν **κόμβο**, αποδεικνύεται πως έχει τη μορφή

$$(\sigma_h + \tau_h s) + (\sigma'_h + \tau'_h \times s) + \tau_n \times s$$

με τον **πρώτο** όρο να είναι ο **χρόνος κλήσης** της **send** και της **αντιγραφής** του μηνύματος, τον δεύτερο όρο να αναφέρεται στην **επιβάρυνση λόγω μεταφοράς δεδομένων** και τον **τρίτο όρο** να περιγράφει τον **χρόνο** που απαιτείται, προκειμένου ο παραλήπτης να **αντιγράψει** το μήνυμα στον χώρο της **διεργασίας παραλήπτη**



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Τεχνικές παράλληλου προγραμματισμού

Παραλληλισμός μέσω διοχέτευσης

Το **ρεύμα δεδομένων** ορίζεται ως ένα **διατεταγμένο ζεύγος** της μορφής (μ, δ) , όπου μ είναι ο **χρόνος άφιξης** του πρώτου μπλοκ δεδομένων και δ είναι ο χρόνος που μεσολαβεί **ανάμεσα σε δυο διαδοχικές αφίξεις** τέτοιων μπλοκ.

Ο κάθε κόμβος της διοχέτευσης, μπορεί να μοντελοποιηθεί ως μία **υπολογιστική μονάδα**, η οποία δέχεται m ρεύματα δεδομένων εισόδου, εφαρμόζει επί αυτών τους κατάλληλους σε κάθε περίπτωση **υπολογισμούς** και δημιουργεί στην έξοδο του, **ένα** ρεύμα δεδομένων εξόδου

Εάν έχουμε m ρεύματα **δεδομένων εισόδου** $x_i = (\mu_i, \delta_i)$ ($1 \leq i \leq m$), μία υπολογιστική μονάδα μπορεί να περιγραφεί ως μία **συνάρτηση** της μορφής $y = f(x_1, x_2, \dots, x_m)$, ενώ ένας **αλγόριθμος παραλλήλων δεδομένων** που χρησιμοποιεί την τεχνική της **διοχέτευσης**, στηρίζεται στη χρήση ενός **δικτύου τέτοιων μονάδων**, οι οποίες συνδέονται μεταξύ τους με **ρεύματα δεδομένων**.

Η συνάρτηση f για το καθένα από τα n μπλοκ δεδομένων που δέχεται στο **καθένα** από τα ρεύματα εισόδου της, θα πρέπει να είναι σε θέση να πραγματοποιήσει την **αλληλουχία διαδικασιών**

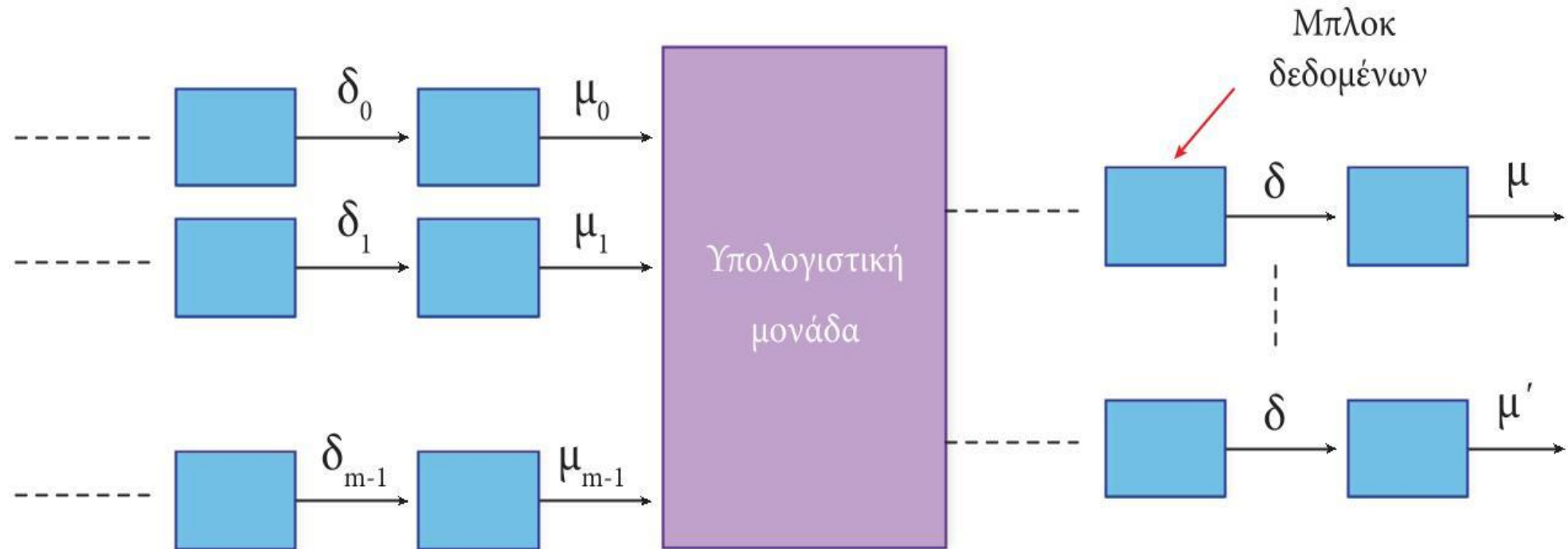
παραλαβή εισόδου $\rightarrow$ υπολογισμός αποτελέσματος $\rightarrow$ αποστολή εξόδου



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Τεχνικές παράλληλου προγραμματισμού

Παραλληλισμός μέσω διοχέτευσης

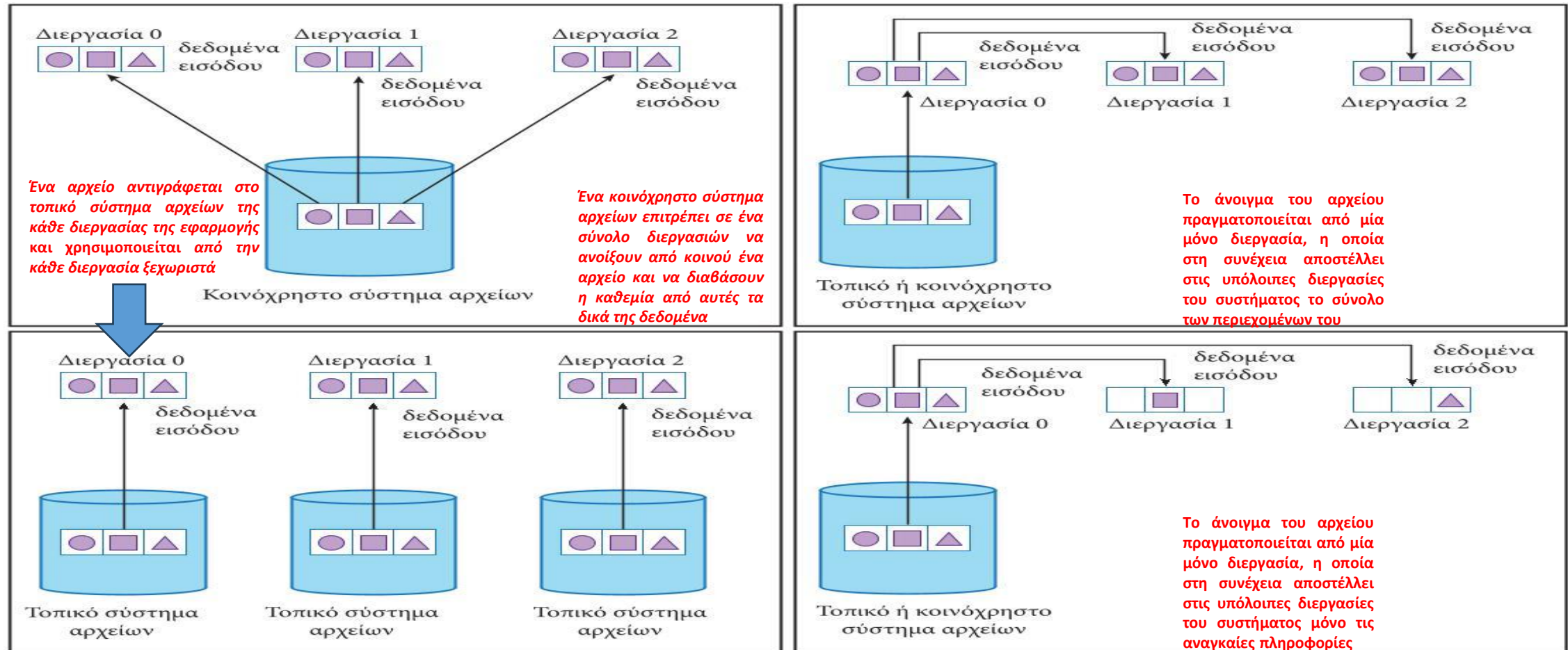


Το μοντέλο της **υπολογιστικής μονάδας** που χρησιμοποιείται σε μία παράλληλη αρχιτεκτονική **διοχέτευσης**.

Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Τεχνικές παράλληλου προγραμματισμού

Παραλληλοποίηση διαδικασιών εισόδου - εξόδου



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Αναγνώριση και απαλοιφή εξαρτήσεων

Υπό **ποιες προϋποθέσεις** δύο προγράμματα που υφίστανται στον **ίδιο χώρο επαναλήψεων** ενός βρόχου αλλά χαρακτηρίζονται από **διαφορετική σειρά εκτέλεσης**, είναι **ισοδύναμα**;

Αυτό το πρόβλημα αποδεικνύεται πως είναι **μη διαγνώσιμο** και επομένως θα πρέπει να καταφύγουμε σε μία **συντηρητική** απάντηση, διατυπώνοντας **συνθήκες ισοδυναμίας** οι οποίες θα είναι **ικανές** αλλά όχι **αναγκαίες**.

Αυτή η διαδικασία είναι **άρρηκτα συνδεδεμένη** με το πολύ σημαντικό ζήτημα της **ανάλυσης εξαρτήσεων**.

Στην γενική περίπτωση, οι εξαρτήσεις υφίστανται, τόσο ανάμεσα **στις επιμέρους ατομικές λειτουργίες** μιας εφαρμογής όσο και ανάμεσα στα **βήματα ενός αλγορίθμου** και είναι σημαντικές, όταν υφίσταται η **δυνατότητα παράλληλης** εκτέλεσης τμημάτων του αλγορίθμου, ταυτόχρονα και **ανεξάρτητα** το ένα από το άλλο.

Το **πλήθος** και ο **τύπος** των εξαρτήσεων εξαρτώνται από την **γλώσσα προγραμματισμού** και τη **μορφή του αλγορίθμου**, ενώ παράγοντες που υπαγορεύουν τον **βαθμό** στον οποίο αυτές μπορούν να απομακρυνθούν, περιλαμβάνουν μεταξύ άλλων, τον **τύπο του μεταγλωττιστή**, τον **τρόπο της μεταγλώττισης**, καθώς και την **χρονική στιγμή της εκτέλεσης**.



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Οι συνθήκες του Bernstein

Εξετάζουν εάν δύο **διαδοχικά τμήματα** ενός προγράμματος μπορούν να εκτελεστούν **παράλληλα**, οδηγώντας **στα ίδια αποτελέσματα** με εκείνα που θα προέκυπταν, εάν η εφαρμογή εκτελούνταν με **σειριακό** τρόπο, δηλαδή είναι **ισοδύναμα**.

Σύμφωνα με αυτές τις συνθήκες, δύο προγράμματα είναι **ισοδύναμα** μεταξύ τους, εάν τα **στιγμιότυπα** των **δηλώσεων** που είναι **εξαρτημένα μεταξύ τους** εκτελούνται με την **ίδια σειρά** και στα δύο αυτά προγράμματα

Ο Bernstein θεώρησε **δύο μοντέλα** συστήματος **μνήμης / επεξεργαστή** που διαφοροποιούνται στο ότι το ένα **χρησιμοποιεί κρυφή μνήμη** ενώ το άλλο **όχι** και όρισε τέσσερα σύνολα διευθύνσεων:

- τις διευθύνσεις W_i στις οποίες πραγματοποιείται **μόνο ανάγνωση**
- τις διευθύνσεις X_i στις οποίες πραγματοποιείται **μόνο εγγραφή**
- τις διευθύνσεις Y_i στις οποίες πραγματοποιείται **πρώτα ανάγνωση** και στη συνέχεια **εγγραφή**
- τις διευθύνσεις Z_i στις οποίες πραγματοποιείται **πρώτα εγγραφή** και στη συνέχεια **ανάγνωση**

με τον δείκτη i να προσδιορίζει την διεργασία P_i η οποία πραγματοποιεί τις παραπάνω διαδικασίες.



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Οι συνθήκες του Bernstein

Περιοριζόμενοι στο **απλό μοντέλο** που **δεν** περιλαμβάνει την χρήση **κρυφής μνήμης** οι **τρεις** συνθήκες του **Bernstein**, η ισχύς των οποίων διασφαλίζει πως η **παράλληλη εκτέλεση** δύο διεργασιών P_1 και P_2 θα οδηγήσει **στα ίδια** αποτελέσματα με αυτά που θα προέκυπταν εάν ο κώδικας εκτελούνταν με **σειριακό** τρόπο είναι οι

$$(W_1 \cup Y_1 \cup Z_1) \cap (X_2 \cup Y_2 \cup Z_2) = \emptyset$$

$$(X_1 \cup Y_1 \cup Z_1) \cap (W_2 \cup Y_2 \cup Z_2) = \emptyset$$

$$(X_1 \cap X_2 \cap (W_2 \cup Y_2)) = \emptyset$$

Από τις **τρεις παραπάνω** συνθήκες

- η **πρώτη** διασφαλίζει πως η διεργασία P_1 δεν θα αντιληφθεί **την παραμικρή διαφορά** όσον αφορά στο περιεχόμενο της μνήμης είτε εκτελεστεί **πριν από** την P_2 είτε **παράλληλα με** αυτή και πως η διεργασία P_2 δεν θα **τροποποιήσει** το περιεχόμενο των διευθύνσεων μνήμης που χρησιμοποιούνται από την διεργασία P_1
- η **δεύτερη** διασφαλίζει πως η διεργασία P_2 δεν **απαιτεί** για την εκτέλεσή της αποτελέσματα υπολογισμών που πραγματοποιούνται από την P_1 αφού πλέον **δεν είναι υποχρεωτικό** η P_1 να εκτελεστεί **πριν από** αυτή
- η **τρίτη** διασφαλίζει πως μία **τρίτη** διεργασία P_3 που εκτελείται **μετά από** τις P_1 και P_2 θα αντιλαμβάνεται **πάντοτε** το **ίδιο** περιεχόμενο μνήμης, **ανεξάρτητα από το εάν** οι διεργασίες P_1 και P_2 έχουν εκτελεστεί με **ακολουθιακό** ή **παράλληλο** τρόπο



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Αναγνώριση και απαλοιφή εξαρτήσεων

Στην περίπτωση **εκδήλωσης** δύο **διακριτών** συμβάντων **A** και **B**, θα λέμε πως το συμβάν **B** **εξαρτάται** από το συμβάν **A**, εάν το **A** θα πρέπει να εμφανιστεί **πριν από** το **B**, ή ισοδύναμα, εάν το **B** **δεν μπορεί να εκτελεστεί** πριν από την ολοκλήρωση του **A**.

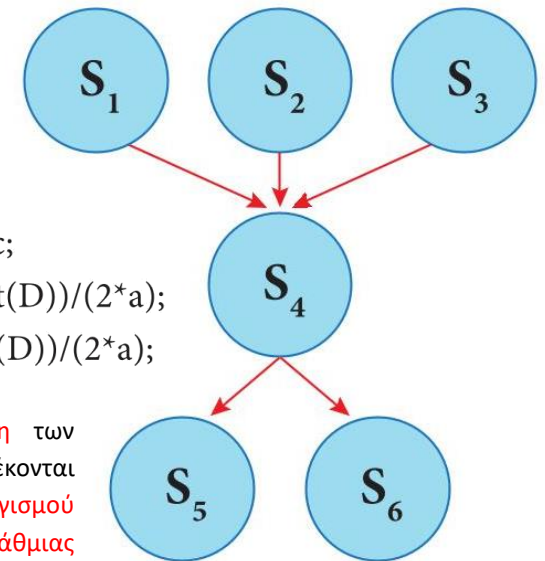
Εάν τα **συμβάντα** αναφέρονται στην εκτέλεση **δύο δηλώσεων** κώδικα **S₁** και **S₂**, θα λέμε πως ανάμεσά τους υφίσταται **εξάρτηση** και ειδικότερα πως η δήλωση **S₂** εξαρτάται από την δήλωση **S₁**, εάν η δήλωση **S₁** εμφανίζεται στον πηγαίο κώδικα **πριν από** την δήλωση **S₂** και κατά συνέπεια, όπως υπαγορεύεται από την **σειριακή φύση** του κώδικα, εκτελείται πρώτη.



Η **δομή του γράφου** καταδεικνύει το γεγονός πως η εντολή **S₄** που υπολογίζει την **διακρίνουσα** η τιμή της οποίας δίδεται από την εξίσωση **$D = b^2 - 4ac$** απαιτεί την **αρχικοποίηση** των παραμέτρων **a**, **b** και **c** και κατά συνέπεια θα πρέπει να εκτελεστεί **μετά από** τις εντολές **S₁**, **S₂** και **S₃** που πραγματοποιούν αυτές τις αρχικψ οποιήσεις, ενώ στη συνέχεια, χρησιμοποιείται **η ίδια** για τον υπολογισμό των ριζών **x₁** και **x₂** και κατά συνέπεια, θα πρέπει να εκτελεστεί **πριν από** τις εντολές **S₅** και **S₆** που υπολογίζουν τις τιμές αυτών των ριζών.

$S_1 \rightarrow a=1;$
 $S_2 \rightarrow b=5;$
 $S_3 \rightarrow c=6;$
 $S_4 \rightarrow D=b*b-4*a*c;$
 $S_5 \rightarrow x_1=(-b + \text{sqrt}(D))/(2*a);$
 $S_6 \rightarrow x_2=(-b - \text{sqrt}(D))/(2*a);$

Διαγραμματική απεικόνιση των εξαρτήσεων που εμπλέκονται στην διαδικασία **υπολογισμού των ριζών** μιας **δευτεροβάθμιας εξίσωσης**.



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Εξαρτήσεις ελέγχου και εξαρτήσεις δεδομένων

Στην πράξη υφίστανται δύο βασικοί τύποι εξαρτήσεων, οι **εξαρτήσεις ελέγχου** και οι **εξαρτήσεις δεδομένων**.

Οι **εξαρτήσεις ελέγχου** σχετίζονται με την κατάσταση κατά την οποία η εκτέλεση μιας εντολής **αναμένει την αποτίμηση μιας συνθήκης διακλάδωσης ελέγχου** ή την **ολοκλήρωση ενός βρόχου**. Με άλλα λόγια, ανάμεσα σε δύο δηλώσεις S_1 και S_2 υφίσταται **εξάρτηση ελέγχου**, εάν η δήλωση S_1 υπαγορεύει εάν **θα εκτελεστεί ή όχι** η δήλωση S_2 .

(για παράδειγμα, στον κώδικα **`if (x!=0) y=1/x;`** η εντολή **`y = 1/x`** θα εκτελεστεί μόνο εάν είναι **`x ≠ 0`**)

Οι **εξαρτήσεις δεδομένων** σχετίζονται με τις περιπτώσεις στις οποίες **δεν είναι δυνατή η ταυτόχρονη** εκτέλεση δύο δηλώσεων κώδικα, λόγω **συγκρούσεων** που εκδηλώνονται κατά την **απόπειρα πρόσβασης στην ίδια** θέση μνήμης.

Ειδικότερα, μία **εξάρτηση δεδομένων** εμφανίζεται όταν ικανοποιούνται **ταυτόχρονα** οι επόμενες τρεις συνθήκες:

- υπάρχουν **δύο δηλώσεις** οι οποίες αναφέρονται **στην ίδια** θέση μνήμης
- **τουλάχιστον η μία** από αυτές τις δηλώσεις παραπέμπει σε διαδικασία **εγγραφής** σε αυτή τη θέση
- **ανάμεσα** στις δύο αυτές δηλώσεις **δεν υπάρχει καμία άλλη αναφορά** σε αυτή τη θέση μνήμης

Η **εξάρτηση** των δεδομένων σε αυτή την περίπτωση, έγκειται στο γεγονός, πως ο **υπολογισμός** της τιμής μίας μεταβλητής **δεν είναι δυνατός** εάν δεν καταστεί **γνωστή** και **διαθέσιμη** η τιμή κάποιας άλλης μεταβλητής, από την οποία αυτή εξαρτάται.



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Τα τέσσερα είδη εξαρτήσεων

Εξαρτήσεις ροής ($S_1 \delta_f S_2$): η εντολή S_1 πραγματοποιεί **εγγραφή**, ενώ η εντολή S_2 πραγματοποιεί **ανάγνωση**. Αυτή η εξάρτηση οδηγεί στον **κίνδυνο ανάγνωσης ύστερα από εγγραφή** (Read After Write Hazard, **RAW**).

Αντι-εξαρτήσεις ($S_1 \delta_a S_2$): η εντολή S_1 πραγματοποιεί **ανάγνωση**, ενώ η εντολή S_2 πραγματοποιεί **εγγραφή**. Αυτή η εξάρτηση οδηγεί στον **κίνδυνο εγγραφής ύστερα από ανάγνωση** (Write After Read Hazard, **WAR**).

Εξαρτήσεις εξόδου ($S_1 \delta_o S_2$): αμφότερες οι εντολές S_1 και S_2 προσπελαίνουν **την ίδια** θέση μνήμης για την πραγματοποίηση **εγγραφής**. Αυτή η εξάρτηση οδηγεί στον **κίνδυνο εγγραφής ύστερα από εγγραφή** (Write After Write Hazard, **WAW**).

Εξαρτήσεις εισόδου ($S_1 \delta_i S_2$): αμφότερες οι εντολές S_1 και S_2 προσπελαίνουν **την ίδια** θέση μνήμης για την πραγματοποίηση **ανάγνωσης**. Αυτή η εξάρτηση, η οποία γενικά **δεν λαμβάνεται υπόψη**, χαρακτηρίζεται ως εξάρτηση **RAR** (Read After Read).



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Τα τέσσερα είδη εξαρτήσεων

Θεωρώντας την επόμενη **ακολουθία** δηλώσεων κώδικα

$$S_1 : a = b$$

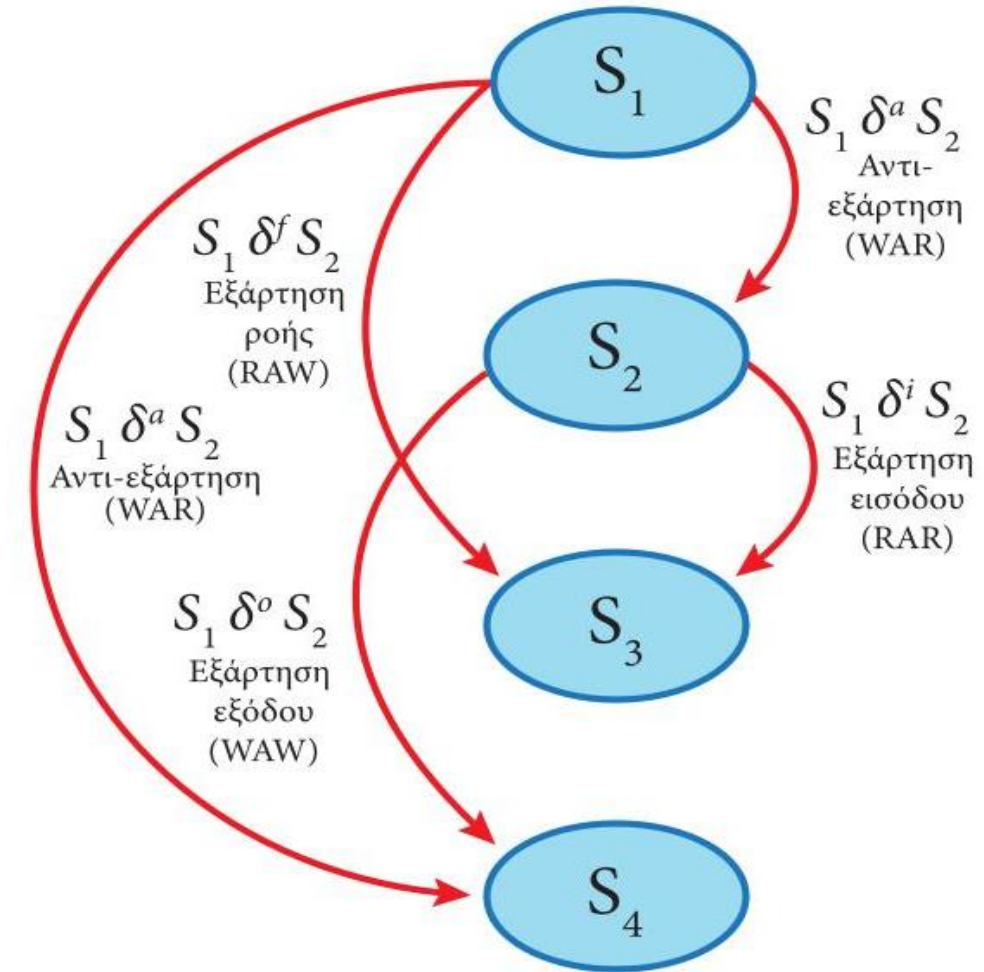
$$S_2 : b = c + d$$

$$S_3 : e = a + d$$

$$S_4 : b = f$$

οι **εξαρτήσεις** που υφίστανται ανάμεσά τους, περιγράφονται από τον **κατευθυνόμενο ακυκλικό γράφο** του επισυναπτόμενου σχήματος που ονομάζεται **γράφος εξάρτησης**.

Χρησιμοποιώντας τέτοιους γράφους, οι **μεταγλωττιστές** που πραγματοποιούν **αυτόματη παραλληλοποίηση κώδικα**, έχουν την δυνατότητα να **ανιχνεύσουν** αυτόματα **ανεξάρτητα** μεταξύ τους τμήματα επαναληπτικών βρόχων τα οποία μπορούν να εκτελεστούν **παράλληλα**, ή να **αναφέρουν τον λόγο** για τον οποίο αυτή η παραλληλοποίηση **δεν είναι** επιτρεπτή.



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Γράφοι εξάρτησης

Ένας **γράφος εξάρτησης** αποτελεί μία **δομή γράφου**, στην οποία οι **κορυφές** αναπαριστούν **δηλώσεις** του προγράμματος, ενώ τα **βέλη** αναπαριστούν **εξαρτήσεις** που μπορούν να ανήκουν σε **οποιοδήποτε** από τους **τέσσερις** τύπους εξάρτησης.

Στην περίπτωση κατά την οποία οι **κορυφές** του γράφου περιγράφουν **μεταβλητές**, ενώ η ύπαρξη ενός **βέλους** που ξεκινά από την κορυφή **x** και καταλήγει στην κορυφή **y**, υποδηλώνει πως η τιμή της μεταβλητής **y** θα υπολογιστεί σε **μεταγενέστερη** χρονική στιγμή σε σχέση με την τιμή της μεταβλητής **x**, αφού σε αυτόν τον υπολογισμό, χρησιμοποιείται η τιμή της **x**, ο γράφος εξάρτησης, περιγράφεται από το όνομα **γράφος προέγησης**.

Εάν η δομή του γράφου είναι **περίπλοκη** και **δυσνόητη**, μπορούμε να σχεδιάσουμε **μόνο** τα αναγκαία βέλη ή να εφαρμόσουμε **μετασχηματισμούς** που συνίστανται στην **απομάκρυνση τόξων** ή στην **κατάργηση** κυκλικών διαδρομών.

Οι πιο **σημαντικοί** από αυτούς των **μετασχηματισμούς** ενός γράφου εξάρτησης είναι οι εξής:

- **Μετονομασία μεταβλητής** που συνίσταται στην χρήση **διαφορετικού ονόματος** για **κάθε στιγμιότυπο** της μεταβλητής.
- **Επέκταση μεταβλητής** που συνίσταται στη **μετατροπή** μίας **βαθμωτής** μεταβλητής σε **πίνακα**
- **Διαίρεση κορυφών** που σχετίζεται με την διαδικασία της **απομάκρυνσης κυκλικών διαδρομών**
- **Αντικατάσταση προς τα εμπρός** που σχετίζεται με την διαδικασία της **απαλοιφής τόξων**



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Το πρόβλημα της ευθυγράμμισης δεδομένων

Η **ευθυγράμμιση δεδομένων** ορίζεται ως ο **διαμοιρασμός** των **παράλληλων** δομών δεδομένων στους επεξεργαστές του συστήματος έτσι ώστε τα στοιχεία που χρησιμοποιούνται σε έναν υπολογισμό από έναν επεξεργαστή να βρίσκονται στη μνήμη αυτού του ίδιου επεξεργαστή, έτσι ώστε να μην απαιτείται η παραμικρή επικοινωνία δεδομένων.

Η **ευθυγράμμιση δεδομένων** μπορεί να πραγματοποιηθεί είτε ως τμήμα της γενικότερης διαδικασίας κατανομής των δεδομένων τους επεξεργαστές του συστήματος, είτε σε **μεταγενέστερο** στάδιο μέσω **μετασχηματισμών** και **απεικονίσεων**.

Θεωρώντας τη δήλωση $a[i][j]=b[i][j]$ ενός **παράλληλου** βρόχου, αυτή εκτελείται **χωρίς την παραμικρή επικοινωνία** ανάμεσα στους επεξεργαστές εάν τα **αντίστοιχα** στοιχεία των πινάκων **A** και **B**, βρίσκονται αποθηκευμένα στην **τοπική μνήμη** του επεξεργαστή που θα αναλάβει να εκτελέσει την παραπάνω δήλωση.

Ωστόσο αυτό το **σχήμα αποθήκευσης** δεν είναι **κατάλληλο** για την εκτέλεση της δήλωσης $a[i][j]=a[i][j]+b[k][i]$. Ακόμη και αν αποθηκεύσουμε στην μνήμη του ίδιου επεξεργαστή τα στοιχεία $a[i][j]$ και $b[j][i]$, τότε παρά το γεγονός πως τώρα απαιτείται η χρήση **μιας και μοναδικής γραμμής**, ωστόσο **και πάλι** απαιτείται η επικοινωνία μεταξύ των επεξεργαστών και κατά συνέπεια, η χρήση του **κατάλληλου συστήματος** που να επιτρέπει μία τέτοια επικοινωνία, όπως είναι μία **δομή υπερκύβου**.

Στην προκειμένη περίπτωση προκειμένου **να μην έχουμε επικοινωνία δεδομένων** θα πρέπει ο **κάθε** επεξεργαστής να περιέχει στην μνήμη του **όχι ένα απλό στοιχείο** αλλά **μία ολόκληρη γραμμή** πίνακα.



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Ευθυγράμμιση δεδομένων με τεχνικές ανάθεσης

(Knobe, Lukas & Steele): Τα **γειτονικά** μεταξύ τους ως προς **κάποια διάσταση** στοιχεία πινάκων, αποθηκεύονται στις μνήμες επεξεργαστών που αντιστοιχούν σε **γειτονικές θέσεις δομής υπερκύβου**, οπότε η **ολίσθηση δεδομένων** κατά την διεύθυνση **οποιοδήποτε** άξονα του πλέγματος, είναι **πολύ πιο γρήγορη** από κάθε διαδικασία **μεταβίβασης μηνυμάτων**.

Ο υπολογισμός μιας διεύθυνσης μνήμης απαιτεί **την γνώση του cpuld**, καθώς και της **απόστασης** του θεωρούμενου στοιχείου μνήμης από την **πρώτη διεύθυνση** της τοπικής μνήμης του αντίστοιχου επεξεργαστή, ενώ εάν αυτή η απόσταση είναι **σταθερή** για **όλους** τους επεξεργαστές, τότε το μόνο που απαιτείται, είναι **η γνώση του cpuld**.

Σε αυτή την τεχνική **το κάθε cpuld** θεωρείται ως μία **ακολουθία από bits**, διαχωρισμένη σε **κελιά** του κατάλληλου μήκους που θα φιλοξενήσουν **τις τιμές των δεικτών του πίνακα** χρησιμοποιώντας τον **κατάλληλο αριθμό** από bits.

Εάν π.χ. το **cpuld** έχει μήκος **16 bits**, ενώ ο πίνακας έχει διαστάσεις **32 x 64 x 32**, μπορούμε να **προσδιορίσουμε** τις διευθύνσεις μνήμης των στοιχείων του πίνακα στις **αντίστοιχες διευθύνσεις** του χώρου, χρησιμοποιώντας αντίστοιχα **5, 6 και 5 bits** και **διαχωρίζοντας** με τον τρόπο αυτό, τα **16 bits** του **cpuld** σε **τρία** κελιά, που ορίζονται από τα bits **0 - 4, 5 - 10 και 11 - 15**.

Παράδειγμα: έστω το στοιχείο πίνακα **A[9][20][5]**. Η **κωδικοποίηση** των αριθμών **9, 20 και 5** στο **δυαδικό** σύστημα με **5, 6 και 5 bits** αντίστοιχα, μας δίνει **01001, 010100 και 00101**. Η **συνένωση** των bits αυτών των τριών συμβολοσειρών μας οδηγεί στον δυαδικό αριθμό **0100101010000101** μήκους **16 bits**, που αποτελεί το **cpuld** του επεξεργαστή στην τοπική μνήμη του οποίου θα αποθηκευτεί η τιμή του στοιχείου. Ως **σύμβαση**, το **πρώτο** κελί του πίνακα, εκχωρείται στον επεξεργαστή με **cpuld = 0**.



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Ευθυγράμμιση δεδομένων με τεχνικές ανάθεσης

Η **συσχέτιση** της **κάθε διάστασης** του πίνακα με το **κατάλληλο τμήμα** του `cruld` του επεξεργαστή, στη μνήμη του οποίου θα αποθηκευτεί η τιμή που υπολογίζεται με βάση την τιμή του δείκτη για αυτή τη διάσταση ονομάζεται **στρατηγική ανάθεσης**.

Αυτή η στρατηγική υλοποιείται μέσω **γραμμικών συναρτήσεων ανάθεσης** που **απεικονίζουν** την τιμή **X** ενός δείκτη του πίνακα, στην τιμή **$aX + B$** (με το **a** να εκφράζει **βήμα μεταβολής** και το **b** να εκφράζει **μετατόπιση**), ενώ η **πλήρης** περιγραφή της διαδικασίας, απαιτεί τον ορισμό **μίας τέτοιας** συνάρτησης ανάθεσης για την **κάθε διάσταση** του πίνακα.

Εάν λαμβάνει χώρα αποθήκευση της τιμής **ενός και μοναδικού κελιού** του πίνακα στη μνήμη **του κάθε** επεξεργαστή, μπορούμε να **οργανώσουμε** τους επεξεργαστές σε κατάλληλες **τοπολογίες**.

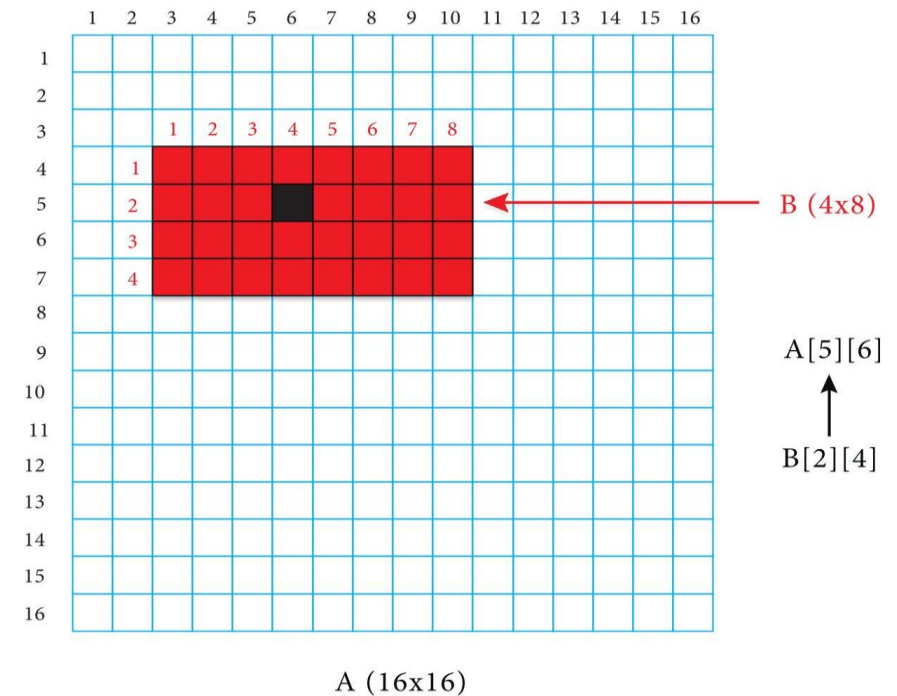
Στο διπλανό σχήμα, οι πίνακες **A** και **B**, έχουν διαστάσεις **16 x 16** και **4 x 8** ενώ ο πίνακας **B** έχει **αντιγραφεί** σε **υποπίνακα** του πίνακα **A**.



Οι επεξεργαστές είναι **οργανωμένοι** σε **πλέγμα** διαστάσεων **16 x 16**.

Η **συνάρτηση ανάθεσης** που προσδιορίζει τον επεξεργαστή **(x,y)** στην μνήμη του οποίου θα αποθηκευτεί το στοιχείο **$A(i,j)$** είναι η $x = i-1, y = j-1$.

Η **συνάρτηση ανάθεσης** που προσδιορίζει τον επεξεργαστή **(x,y)** στην μνήμη του οποίου θα αποθηκευτεί το στοιχείο **$B(i,j)$** έτσι ώστε η αντιγραφή να πραγματοποιηθεί **χωρίς επικοινωνία δεδομένων**, είναι η $x = i+2, y = j+1$.



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Ευθυγράμμιση δεδομένων με μετασχηματισμούς και απεικονίσεις

(Li & Chen) Στηρίζεται στην χρήση συναρτήσεων ευθυγράμμισης που επιτρέπουν την ενσωμάτωση του χώρου των επαναλήψεων **πολλαπλών πινάκων**, σε έναν **ενιαίο** χώρο στιγμιοτύπων.

Αυτή η ευθυγράμμιση αποτελεί μία **απεικόνιση** ανάμεσα σε δύο **χώρους στιγμιοτύπων** η οποία μετασχηματίζει **ζεύγη αναφορών σε πίνακες**, έτσι ώστε να **ελαχιστοποιείται** το **κόστος επικοινωνίας** μεταξύ των επεξεργαστών.

Το βασικό εργαλείο της τεχνικής των **Li & Chen**, είναι το **σχήμα αναφοράς πρόσβασης στη μνήμη** που έχει τη μορφή

$$\lceil a(i_1, i_2, \dots, i_m) \leftarrow b(j_1, j_2, \dots, j_n) : \gamma \rceil$$

Αυτό το σχήμα περιγράφει μία **διαδικασία ανάθεσης τιμής σε στοιχείο πίνακα**, της μορφής

$$a(i_1, i_2, \dots, i_m) = b(j_1, j_2, \dots, j_n)$$

η οποία πραγματοποιείται **υπό την προϋπόθεση** πως η συνθήκη **γ** είναι αληθής. Έτσι, η **ανάθεση τιμής υπό συνθήκη**

$$\text{if } (k \neq 0) \ a[i][j][k] = b[i][j][k-1]$$

χρησιμοποιώντας το παραπάνω **σχήμα αναφοράς**, συμβολίζεται ως

$$\lceil a(i, j, k) \leftarrow a(i, j, k-1) : k \neq 0 \rceil$$



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Ευθυγράμμιση δεδομένων με μετασχηματισμούς και απεικονίσεις

Το **σχήμα αναφοράς** στη μνήμη χαρακτηρίζεται ως **ομοιόμορφο** κατά τη **διεύθυνση** p , εάν ισχύει η συνθήκη $j_p = i_p + c$, όπου c μία μικρή θετική σταθερά, η οποία είναι ανεξάρτητη από τα όρια του χώρου στιγμιοτύπων.

Χρησιμοποιώντας το χαρακτηριστικό της ομοιομορφίας και θεωρώντας για τους σκοπούς της περιγραφής μας, έναν **κοινό χώρο στιγμιοτύπων n διαστάσεων**, οπότε θα είναι

$$[A(i_1, i_2, \dots, i_n) \leftarrow B(j_1, j_2, \dots, j_n) : \gamma]$$

μπορούμε να ορίσουμε **τρεις τύπους αναφοράς** στη μνήμη και πιο συγκεκριμένα

- την αναφορά στην **τοπική μνήμη** εάν είναι $c = 0$ για όλες τις διευθύνσεις και επομένως, είναι $j_p = i_p$ για **κάθε** διεύθυνση
- την αναφορά σε θέση μνήμης **γειτονικού επεξεργαστή**, εάν είναι $j_k = i_k + c_k$ ($1 \leq k \leq n$)
- την τυχαία αναφορά στην μνήμη **οποιουδήποτε επεξεργαστή** για **ανομοιόμορφο** σχήμα αναφοράς προς **κάθε** διεύθυνση

Οι τρεις παραπάνω τύποι πρόσβασης, αντιστοιχούν σε **ισάριθμους τύπους επικοινωνίας δεδομένων**, με την **τοπική** αναφορά να είναι **τάξεις μεγέθους** πιο γρήγορη από την επικοινωνία δεδομένων μεταξύ **γειτονικών επεξεργαστών**, η οποία είναι **πιο αποτελεσματική** σε σχέση με την **γενική περίπτωση επικοινωνίας** ανάμεσα σε δύο **οποιουδήποτε επεξεργαστές**.

Σε αμφότερες τις δύο τελευταίες περιπτώσεις, η επικοινωνία στηρίζεται στον **μηχανισμό μεταβίβασης μηνυμάτων**.



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Ευθυγράμμιση δεδομένων με μετασχηματισμούς και απεικονίσεις

Η **διαστατικότητα** του **χώρου στιγμιotypών** ενός πίνακα, **ταυτίζεται** με την **διαστατικότητα** του πίνακα και υπαγορεύει το **πλήθος των συνιστωσών** του χώρου στιγμιotypών. Θεωρώντας, για παράδειγμα, ένα **σχήμα αναφοράς** της μορφής

$$[A[i] \leftarrow B[i][i + 1] : \gamma]$$

ο **χώρος στιγμιotypών** του πίνακα **A** έχει διαστατικότητα **n = 1** και διαθέτει **μία και μοναδική** συνιστώσα **dom(A,1)** που σχετίζεται με την έκφραση **i**, ενώ ο **χώρος στιγμιotypών** του πίνακα **B** έχει διαστατικότητα **n = 2** και διαθέτει **δύο** συνιστώσες **dom(B,1)** και **dom(B,2)** που σχετίζονται με τις εκφράσεις **i** και **i + 1**.

Η **ευθυγράμμιση** των **χώρων στιγμιotypών**, πραγματοποιείται με τη βοήθεια των επόμενων τεσσάρων απεικονίσεων:

- **Μετάθεση**: ορίζεται ως $g(i_1, i_2, \dots, i_n) = g(i_{q_1}, i_{q_2}, \dots, i_{q_n})$ όπου $(q_1, q_2, \dots, q_n)$ είναι μετάθεση των στοιχείων της πλειάδας $(1, 2, \dots, n)$.
- **Εμβύθιση**: ορίζεται ως **Βύθιση** $g(i_1, i_2, \dots, i_m) = g(i_{q_1}, i_{q_2}, \dots, i_{q_n})$ όπου $(q_1, q_2, \dots, q_n)$ είναι μετάθεση των στοιχείων της πλειάδας $(1, 2, \dots, n)$ και με τις εκφράσεις i_k ($m \leq k \leq n$) να (μπορεί να) περιέχουν τα $i_1, i_2, \dots, i_m$.
- **Ολίσθηση**: ορίζεται ως $g(i) = i - c$, όπου **c** μία **ακέραια** σταθερά.
- **Ανάκλαση**: ορίζεται ως $g(i) = -i$.

Η **μετάθεση** και η **εμβύθιση** σχετίζονται με μετασχηματισμούς ανάμεσα σε **διαφορετικές** συνιστώσες ενός χώρου στιγμιotypών, ενώ η **ολίσθηση** και η **ανάκλαση** σχετίζονται με μετασχηματισμούς στο εσωτερικό μιας απλής συνιστώσας.

Οι **δύο πρώτες** διαδικασίες, **αυξάνουν** τον βαθμό ομοιομορφίας των σχημάτων αναφοράς στη μνήμη, ενώ οι δύο τελευταίες **μειώνουν περαιτέρω** το κόστος αναφοράς, **μειώνοντας** της τιμές των σταθερών **c** που εκφράζουν **μετατοπίσεις**.



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Ευθυγράμμιση δεδομένων με μετασχηματισμούς και απεικονίσεις

Θεωρώντας δύο πίνακες **A** και **B**, με χώρους $\text{dom}(A)$ και $\text{dom}(B)$ και δύο συνιστώσες **p** και **q** που συμβολίζονται ως $\text{dom}(A,p)$ και $\text{dom}(B,q)$, αυτές σχετίζονται με **σχέση συνάφειας** εάν $i_q = i_p + c$ για σταθερά **c** με **μικρή** τιμή.

Αυτές οι συνιστώσες **προτιμώνται** από τον αλγόριθμο αφού η ευθυγράμμισή τους θα οδηγήσει σε **αύξηση της ομοιομορφίας** των σχημάτων αναφοράς στη μνήμη.

Η **ευθυγράμμιση** στηρίζεται στην κατασκευή ενός **γράφου** στον οποίο οι τιμές των **βαρών** των ακμών του, εκφράζουν τον **βαθμό προτίμησης** του αλγορίθμου, όσον αφορά στην **επιλογή** του ζεύγους των τμημάτων που συνδέονται με αυτό το τόξο:

- το βάρος της κάθε ακμής που συνδέει **δυο χρονικές ως προς τη φύση τους** συνιστώσες λαμβάνει **άπειρη** τιμή
- το βάρος της κάθε **ανταγωνιστικής** ακμής είναι $\epsilon < 1$
- το βάρος της κάθε **μη ανταγωνιστικής** ακμής είναι **ίσο με τη μονάδα**

Δύο ακμές χαρακτηρίζονται ως **ανταγωνιστικές**, όταν αμφότερες σχετίζονται με **το ίδιο σχήμα αναφοράς** στη μνήμη και καταλήγουν στην **ίδια κορυφή** και **μη ανταγωνιστικές**, στην αντίθετη περίπτωση

Εάν ανάμεσα σε δύο κορυφές ορίζονται **περισσότερες από μία** ακμές, αυτή η ομάδα ακμών, μπορεί να αντικατασταθεί με μία **απλή** ακμή, το βάρος της οποίας είναι ίσο με το **άθροισμα των βαρών** των ακμών της ομάδας.



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Ευθυγράμμιση δεδομένων με μετασχηματισμούς και απεικονίσεις

Έστω n το **μέγιστο πλήθος** των κορυφών σε μία στήλη του **γράφου** G , το οποίο εκφράζει την **μέγιστη διαστατικότητα** όλων των χώρων στιγμιοτύπων που πρόκειται να ευθυγραμμιστούν.

Το σύνολο V των **κορυφών** του **γράφου** G , διαμερίζεται σε n **ξένα μεταξύ τους** υποσύνολα $V_1, V_2, \dots, V_n$, υπό τον **περιορισμό**, πως η τοποθέτηση στο **ίδιο υποσύνολο** δύο κορυφών που ανήκουν στην **ίδια στήλη** του γράφου, **δεν είναι** επιτρεπτή.

Δύο **άξονες** που σχετίζονται με τον χώρο στιγμιοτύπων του **ιδίου** πίνακα, **δεν επιτρέπεται** να ανήκουν στο **ίδιο** σύνολο της διαμέρισης, αφού η διαδικασία ευθυγράμμισης πραγματοποιείται ανάμεσα σε **άξονες χώρων στιγμιοτύπων** και προφανώς δεν είναι δυνατόν να εφαρμοσθεί ανάμεσα σε άξονες που ανήκουν στον ίδιο χώρο.

Η **διαμέριση** του **γράφου** G θα γίνει με τέτοιο τρόπο, ώστε **να ελαχιστοποιηθεί το συνολικό βάρος των τόξων που βρίσκονται ανάμεσα σε εκείνες τις κορυφές που ανήκουν σε διαφορετικά υποσύνολα**.

Εάν ο χώρος στιγμιοτύπων E χαρακτηρίζεται από την **μέγιστη διαστατικότητα** n , κάποιος άλλος χώρος στιγμιοτύπων D με την ίδια τιμή διαστατικότητας μπορεί να υποστεί τον μετασχηματισμό της **μετάθεσης** με τον E (αφού αυτή η πράξη εφαρμόζεται ανάμεσα σε χώρους με την **ίδια διαστατικότητα**), ενώ εάν έχει **μικρότερη τιμή διαστατικότητας** μπορεί να υποστεί την πράξη της **εμβύθισης** στον χώρο E . Οι **άλλοι δύο** μετασχηματισμοί εφαρμόζονται εάν **κάτι τέτοιο** απαιτείται.

Με τον τρόπο αυτό, ο χώρος στιγμιοτύπων εντός του οποίου θα απεικονιστούν όλοι οι υπόλοιποι χώροι θα καταστεί ο κοινός χώρος στιγμιοτύπων για όλους τους βρόχους, κάτι που οδηγεί και στο ζητούμενο.



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Ευθυγράμμιση δεδομένων με μετασχηματισμούς και απεικονίσεις

```
dom T = [0...n];  
dom D = [1...n] x [1...(n + 1)];  
dom D1 = [1...n];  
for (k: T) {  
  if (k > 0) then  
    apivot(k) = max {|a(i, k, k - 1)| | k <= i <= n};  
  if (k > 0) then  
    ipivot(k) = max {i | k <= i <= n : |a(i, k, k - 1)| = apivot(k)};  
  if (k > 0) then  
    forall (i : D1)  
      fac(i, k) = if (i <= k) then 1.0;  
                  else if (i = ipivot(k)) then  
                    a(k, k, k - 1)/a(ipivot(k), k, k - 1);  
                  else a(i, k, k - 1)/a(ipivot(k), k, k - 1);  
  forall ((i, j) : D)  
    a(i, j, k) = if (k = 0) then A0[i, j];  
                else if (i < k) then a(i, j, k - 1);  
                else if (i = k) then a(ipivot(k), j, k - 1);  
                else if (i = ipivot(k)) then a(k, j, k - 1) - a(i, j, k - 1) * fac(i, k);  
                else a(i, j, k - 1) - a(ipivot(k), j, k - 1) * fac(i, k);}
```

n = 3

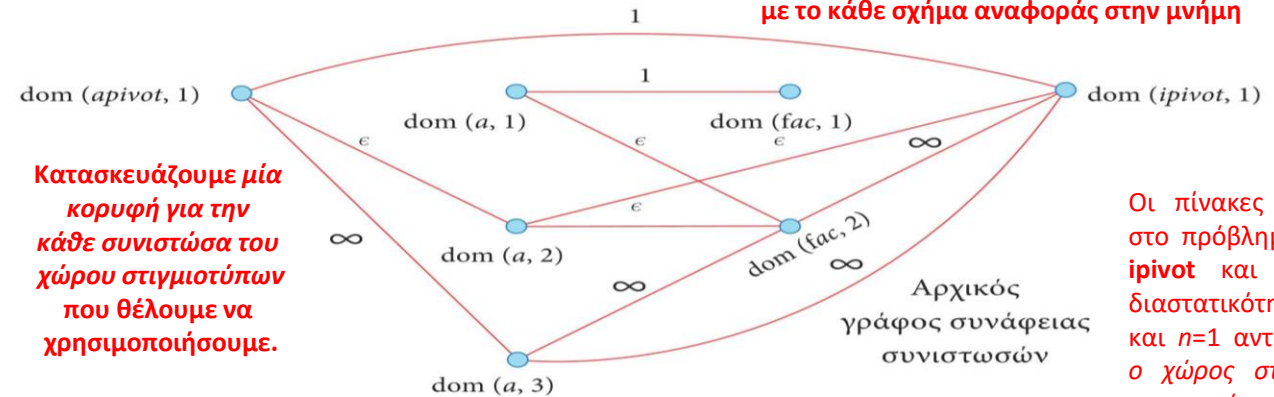
dom (a)	dom (fac)	dom (ipivot)	dom (apivot)
dom (a, 1) dom (a, 2) dom (a, 3)	dom (fac, 1) dom (fac, 2)	dom (ipivot, 1)	dom (apivot, 1)

Οι κορυφές του γράφου οργανώνονται σε στήλες,
με την κάθε στήλη να περιέχει συνιστώσες που ανήκουν στον ίδιο χώρο

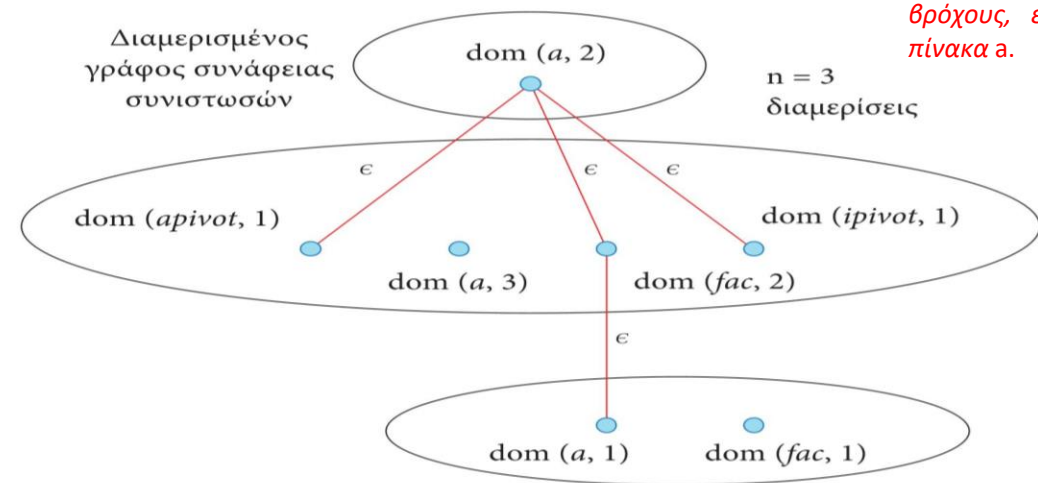
Αλγόριθμος απαλοιφής του Gauss

Κατασκευάζουμε μία
κορυφή για την
κάθε συνιστώσα του
χώρου στιγμιοτύπων
που θέλουμε να
χρησιμοποιήσουμε.

Τα τόξα που συνδέουν δύο συνιστώσες σχετίζονται
με το κάθε σχήμα αναφοράς στην μνήμη



Οι πίνακες που εμπλέκονται
στο πρόβλημα είναι οι **a**, **fac**,
ipivot και **apivot** με τιμές
διαστατικότητας n=3, n=2, n=1
και n=1 αντίστοιχα. Επομένως
ο χώρος στιγμιοτύπων εντός
του οποίου θα απεικονιστούν
όλοι οι υπόλοιποι και θα
καταστεί ο κοινός χώρος
στιγμιοτύπων για όλους του
βρόχους, είναι ο χώρος του
πίνακα **a**.



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Ευθυγράμμιση δεδομένων με μετασχηματισμούς και απεικονίσεις

Ο **μετασχηματισμένος** κώδικας του προβλήματος και οι εφαρμοζόμενες **απεικονίσεις** ευθυγράμμισης.

```
dom  $T = [0...n]$ ;
dom  $D = [1...n] \times [1...(\pi + 1)]$ ;
for ( $k : T$ ) {
  forall (( $i, j$ ) :  $D$ )
    if (( $k > 0$ ) and ( $i = 1$ ) and ( $j = k$ )) then
       $apivot(i, j, k) = \max \{ |a(i, k, k - 1)| \mid k \leq i \leq n \}$ ;
  forall (( $i, j$ ) :  $D$ )
    if (( $k > 0$ ) and ( $i = 1$ ) and ( $j = k$ )) then
       $ipivot(i, j, k) = \max \{ i \mid k \leq i \leq n : |a(i, k, k - 1)| = apivot(1, k, k) \}$ ;
  forall (( $i, j$ ) :  $D$ )
    if (( $k > 0$ ) and ( $j = k$ )) then
       $fac(i, j, k) = \text{if } (i \leq k) \text{ then } 1.0;$ 
      else if ( $i = ipivot(1, k, k)$ ) then
         $a(k, k, k - 1) / a(ipivot(1, k, k), k, k - 1);$ 
      else  $a(i, k, k - 1) / a(ipivot(1, k, k), k, k - 1);$ 
```

$$g_1 : \text{dom}(fac) \longrightarrow \text{dom}(a), \quad g_1(i, k) = (i, k, k)$$
$$g_2 : \text{dom}(ipivot) \longrightarrow \text{dom}(a), \quad g_2(k) = (0, k, k)$$
$$g_3 : \text{dom}(ipivot) \longrightarrow \text{dom}(a), \quad g_3(k) = (0, k, k)$$

```
forall (( $i, j$ ) :  $D$ )
   $a(i, j, k) = \text{if } (k = 0) \text{ then } A0[i, j];$ 
  else if ( $i < k$ ) then  $a(i, j, k - 1);$ 
  else if ( $i = k$ ) then  $a(ipivot(1, k, k), j, k - 1);$ 
  else if ( $i = ipivot(1, k, k)$ ) then
     $a(k, j, k - 1) - a(i, j, k - 1) * fac(i, k, k);$ 
  else  $a(i, j, k - 1) - a(ipivot(1, k, k), j, k - 1) * fac(i, k, k); \}$ 
```



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Ευθυγράμμιση δεδομένων με τη βοήθεια υπερεπιπέδων

(Huang & Sadayappan) Σε αυτή την τεχνική, αρχικά πραγματοποιείται η **μοντελοποίηση** του χώρου των **επαναλήψεων** και των **δεδομένων** και στη συνέχεια η κατασκευή των **κατάλληλων συσχετίσεων** που **απεικονίζουν** τον έναν χώρο στον άλλο.

Η **σωστή εφαρμογή** της τεχνικής απαιτεί

- την ύπαρξη **ακριβώς ενός αντιγράφου** για **κάθε στοιχείο** του πίνακα στην **τοπική μνήμη** του επεξεργαστή που το περιέχει
- την πραγματοποίηση **όλων των διαδικασιών ενημέρωσης** της τιμής του στοιχείου από τον επεξεργαστή που το περιέχει
- την ύπαρξη μιας **συγκεκριμένης** και **αμετάβλητης** κατανομής δεδομένων.

έτσι ώστε η **κατανομή** των δεδομένων να είναι **ανεξάρτητη** από την **αρχιτεκτονική** του υποκείμενου φυσικού συστήματος.

Η **πλήρης απαλοιφή** της επικοινωνίας δεδομένων, απαιτεί

- να **μην υπάρχουν** δύο στοιχεία δεδομένων που να ανήκουν **στο ίδιο σύνολο** μιας διαμέρισης δεδομένων τα οποία να προσπελούνται από επαναλήψεις που να ανήκουν σε **διαφορετικά σύνολα** μιας διαμέρισης επαναλήψεων
- να **μην υπάρχουν** δύο στοιχεία δεδομένων που να ανήκουν σε **διαφορετικά σύνολα** μιας διαμέρισης δεδομένων, τα οποία να προσπελούνται από επαναλήψεις που ανήκουν στο **ίδιο σύνολο** μιας διαμέρισης επαναλήψεων

Η τεχνική διαμέρισης των **Huang & Sadayappan**, στηρίζεται στην χρήση **υπερεπιπέδων**



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Ευθυγράμμιση δεδομένων με τη βοήθεια υπερεπιπέδων

Ειδικότερα, το κάθε μέλος των διαμερίσεων ορίζεται ως η γενικευμένη ένωση ενός πεπερασμένου πλήθους υπερεπιπέδων επαναλήψεων ή δεδομένων, τα οποία, θα πρέπει να είναι παράλληλα το ένα ως προς το άλλο.

Τα στοιχεία αυτών των διαμερίσεων αποτελούν σύνολα επαναλήψεων και δεδομένων, αντίστοιχα, της μορφής

$$P_I = \bigcup_{i=1}^k \{I \mid H_i I = c_i\} \quad \text{και} \quad P_D = \bigcup_{i=1}^k \{D \mid G_i D = c_i\}$$

όπου k ένας θετικός ακέραιος. Οι Huang & Sadayappan αποδεικνύουν ότι

Μία διαμέριση επαναλήψεων $\{P_{I_1}, P_{I_2}, \dots\}$ και μία διαμέριση δεδομένων $\{P_{D_1}, P_{D_2}, \dots\}$ που στηρίζονται στην χρήση υπερεπιπέδων, επιτρέπουν την εκτέλεση της παράλληλης εφαρμογής χωρίς την πραγματοποίηση της παραμικρής επικοινωνίας μεταξύ των επεξεργασιών, εάν ισχύει η συνθήκη

$$\forall I, I' \in IS, \forall D, D' \in DS, \text{access}_1(I) = D \wedge \text{access}_2(I') = D' : \\ (\exists P_{I_j} : I \in P_{I_j} \wedge I' \in P_{I_j} \Leftrightarrow \exists P_{D_k} : D \in P_{D_k} \wedge D' \in P_{D_k})$$

όπου IS και DS οι χώροι επαναλήψεων και δεδομένων και access_1 και access_2 δύο οποιεσδήποτε συναρτήσεις προσπέλασης.



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Ευθυγράμμιση δεδομένων με τη βοήθεια υπερεπιπέδων

Εάν χρησιμοποιούνται **πολλοί** πίνακες δεδομένων, το σύνολο **DS** αποτελεί την **ένωση** όλων των **χώρων δεδομένων** από όλους τους πίνακες, ενώ ένα στοιχείο της **διαμέρισης δεδομένων** περιέχει **υπερεπίπεδα** που σχετίζονται με όλους τους πίνακες.

Στους **πολλαπλούς ένθετους βρόχους**, ο χώρος επαναλήψεων **IS** είναι η **ένωση** των χώρων επαναλήψεων **όλων των ένθετων βρόχων**, ενώ ένα στοιχείο της **διαμέρισης επαναλήψεων**, περιέχει υπερεπίπεδα που σχετίζονται **με όλους τους βρόχους**.

Οι **Huang & Sadayappan** θεωρούν αφινικές συναρτήσεις πρόσβασης $\text{access}(I) = AI + \alpha$ που επιτρέπουν την προσπέλαση ενός στοιχείου κάποιου πίνακα που σχετίζεται με ένα **υπερεπίπεδο δεδομένων** της μορφής $\{ D \mid GD = c_i \}$.

Εάν η επανάληψη **I** προσπελαύνει ένα **στοιχείο** του πίνακα μέσω της συνάρτησης $\text{access}(I)$ και ο **δείκτης** του στοιχείου βρίσκεται στο υπερεπίπεδο δεδομένων $\{ D \mid GD = c_i \}$, θα ισχύει ότι $G(AI + \alpha) = c_i$ ή ισοδύναμα $(GAI = c_i - G\alpha)$ και κατά συνέπεια, αυτή η επανάληψη ανήκει στο **υπερεπίπεδο επαναλήψεων** $\{ I \mid (GAI = c_i - G\alpha) \}$.

Αντίστροφα, εάν θεωρήσουμε ένα υπερεπίπεδο επαναλήψεων $\{ I \mid HI = c_i \}$ έτσι ώστε να είναι $H = GA$ για κάποιο **G**, ο **δείκτης** του στοιχείου πίνακα που προσπελαύνεται κατά την επανάληψη **I** που ανήκει σε αυτό το υπερεπίπεδο μέσω της συνάρτησης $\text{access}(I)$, θα είναι $D = AI + \alpha$ και συνδυάζοντας όλα τα παραπάνω, προκύπτει ότι $GD = G(AI + \alpha) = (GAI + G\alpha) = c_i + G\alpha$ αφού είναι $GAI = HI = c_i$. Κατά συνέπεια, το στοιχείο δεδομένων **D** που προσπελαύνεται κατά την επανάληψη **I** που βρίσκεται στο υπερεπίπεδο $\{ I \mid HI = c_i \}$ ανήκει στο υπερεπίπεδο δεδομένων $\{ D \mid GD = c_i + G\alpha \}$.

Επομένως, ανάμεσα στους **χώρους επαναλήψεων** και **δεδομένων**, υφίσταται μία **αντιστοιχία 1-προς-1**, η οποία μας επιτρέπει να **μεταβούμε** από τον έναν στον άλλο.



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Η οργάνωση των υπολογισμών στις ομοιόμορφες αναδρομικές εξισώσεις

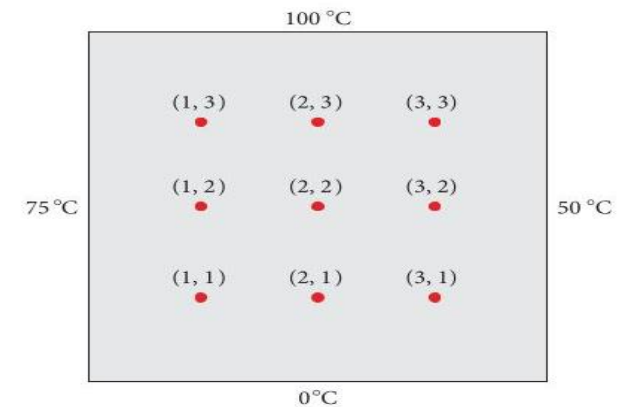
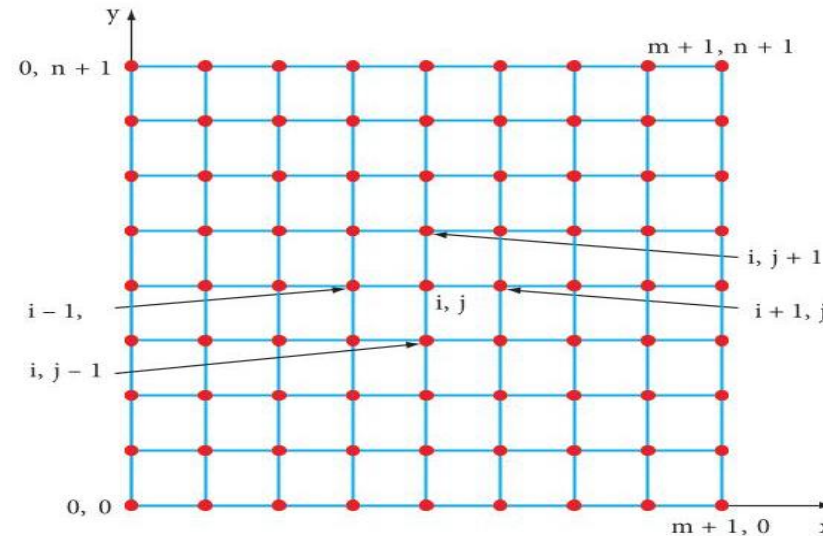
Το ζήτημα της **οργάνωσης των υπολογισμών** που σχετίζονται με τα συστήματα **ομοιόμορφων αναδρομικών εξισώσεων**, μελετήθηκε για πρώτη φορά το 1967 από τους **Karp, Miller και Winograd** (KMW).

Το **μοντέλο των KMW**, είναι αρκετά **ισχυρό**, για να περιγράψει ένα **ευρύ φάσμα αλγορίθμων** από τον χώρο των **μαθηματικών** και της **πληροφορικής** και αρκετά **απλό**, ώστε να προσφέρει μία **επακριβή και πλήρη περιγραφή** μιας τέτοιας οργάνωσης.

Έστω η **μερική διαφορική εξίσωση** του Laplace

$$\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} = f(x, y)$$

που περιγράφει την **κατανομή** της θερμοκρασίας $T(x, y)$, σε κάθε σημείο (x, y) της επιφάνειας μιας **λεπτής τετράγωνης πλάκας** πάχους Δz (που είναι **πολύ μικρότερο** σε σχέση με τις διαστάσεις της πλάκας), η οποία είναι **μονωμένη** με τέτοιο τρόπο, ώστε η θερμότητα να ρέει **μόνο κατά μήκος των διευθύνσεων x και y**



Η **αριθμητική** της επίλυση σύμφωνα με την **μέθοδο των πεπερασμένων διαφορών**, συνίσταται στον ορισμό ενός **πλέγματος σημείων** πάνω στην **επιφάνεια** της **θερμαινόμενης πλάκας** και στον **αριθμητικό υπολογισμό** της τιμής της **θερμοκρασίας**, στο **κάθε σημείο** αυτού του πλέγματος

Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Η οργάνωση των υπολογισμών στις ομοιόμορφες αναδρομικές εξισώσεις

Έχοντας υπολογίσει τις τιμές της θερμοκρασίας **στους κόμβους του πλέγματος**, για τις **κατάλληλες** χρονικές στιγμές, μπορούμε να βρούμε τις τιμές της θερμοκρασίας σε **οποιοδήποτε σημείο**, με μεθόδους **παρεμβολής** και **παρεκβολής**.

Η **πλήρης λύση** του προβλήματος, απαιτεί τον ορισμό των **κατάλληλων συνοριακών συνθηκών** που συνίστανται στον καθορισμό **σταθερών** τιμών θερμοκρασίας για την **καθεμία από τις τέσσερις ακμές** της τετράγωνης πλάκας.

Αντικαθιστώντας τις **μερικές παραγώγους δευτέρας τάξεως** με τις αντίστοιχες **κεντρικές** διαφορές

$$\frac{\partial^2 T}{\partial x^2} = \frac{T_{i+1,j} - 2T_{i,j} + T_{i-1,j}}{\Delta x^2} \quad \text{και} \quad \frac{\partial^2 T}{\partial y^2} = \frac{T_{i,j+1} - 2T_{i,j} + T_{i,j-1}}{\Delta y^2}$$

η **εξίσωση του Laplace** γίνεται

$$\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} = \frac{T_{i+1,j} - 2T_{i,j} + T_{i-1,j}}{\Delta x^2} + \frac{T_{i,j+1} - 2T_{i,j} + T_{i,j-1}}{\Delta y^2} = 0$$

και θέτοντας **$\Delta x = \Delta y = h$** , η παραπάνω σχέση οδηγεί στην **εξίσωση διαφορών του Laplace**

$$T_{i+1,j} + T_{i-1,j} + T_{i,j+1} + T_{i,j-1} - 4T_{i,j} = 0$$



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Η οργάνωση των υπολογισμών στις ομοιόμορφες αναδρομικές εξισώσεις

Η κατασκευή αυτής της εξίσωσης για το κάθε σημείο, οδηγεί σε ένα σύστημα εξισώσεων διαφορών ή αναδρομικών εξισώσεων, το πλήθος των οποίων είναι ίσο με το πλήθος των κόμβων του πλέγματος, δηλαδή ίσο με το πλήθος των συναρτήσεων προς υπολογισμό, η επίλυση του οποίου οδηγεί στη ζητούμενη λύση.

Στην γενική περίπτωση, ένα τέτοιο σύστημα εξισώσεων, ορίζεται ως ένα πεπερασμένο σύνολο εξισώσεων της μορφής

$$\alpha_i(\mathbf{p}) = f_i(\alpha_i(\mathbf{p} - \mathbf{d}_{i1}), \alpha_i(\mathbf{p} - \mathbf{d}_{i2}), \dots, \alpha_i(\mathbf{p} - \mathbf{d}_{im}))$$

όπου οι συναρτήσεις $\alpha_i(\mathbf{p})$ νοούνται ως μεταβλητές τύπου πίνακα, η τιμή των οποίων θα πρέπει να υπολογιστεί σε όλα τα ακέραια σημεία που περιγράφονται από το διάνυσμα $\mathbf{p}$ που χρησιμοποιείται ως όρισμα στη συνάρτηση.

Αυτό το διάνυσμα είναι γνωστό ως διάνυσμα επανάληψης, αφού η επαναληπτική αυτή διαδικασία, συνίσταται στον υπολογισμό της τιμής της συνάρτησης σε όλα τα ακέραια σημεία ενός υποσυνόλου R του χώρου Z^n το οποίο είναι γνωστό ως χώρος επαναλήψεων και με τις συνοριακές τιμές, να αποτελούν ακέραια σημεία που δεν ανήκουν στο σύνολο R .

Τα διανύσματα $\mathbf{d}_{ij}$ ($1 \leq i \leq n, 1 \leq j \leq m$) είναι γνωστά ως διανύσματα εξάρτησης



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Η οργάνωση των υπολογισμών στις ομοιόμορφες αναδρομικές εξισώσεις

Στην πράξη, ενδιαφερόμαστε κυρίως για **κανονικούς επαναληπτικούς αλγορίθμους** (Regular Iterative Algorithms, RIA) στους οποίους η κάθε μεταβλητή αρχικοποιείται **για μία και μοναδική φορά** κατά τη διάρκεια της εκτέλεσης των υπολογισμών.

Αυτοί οι αλγόριθμοι περιγράφονται ως μία **τριάδα** της μορφής (J, A, F) , όπου **J** είναι ο **χώρος των επαναλήψεων**, **A** είναι το σύνολο των **μεταβλητών** που ορίζονται σε **κάθε σημείο** του χώρου των επαναλήψεων, ενώ **F** είναι το σύνολο των **συναρτησιακών** συσχετίσεων **f** που υφίστανται ανάμεσα στις **μεταβλητές** του αλγόριθμου

Ο υπολογισμός της συνάρτησης $\alpha_i(\mathbf{p})$ στην θέση **p** του χώρου των επαναλήψεων, απαιτεί την γνώση κατά την χρονική στιγμή πραγματοποίησής του, των τιμών όλων των συναρτήσεων $\alpha_i(\mathbf{p}-\mathbf{d}_{i1}), \alpha_i(\mathbf{p}-\mathbf{d}_{i2}), \dots, \alpha_i(\mathbf{p}-\mathbf{d}_{im})$ ($1 \leq i \leq n$), κάτι που σημαίνει πως ανάμεσα στην τιμή $\alpha_i(\mathbf{p})$ και στην **καθεμία** από τις παραπάνω τιμές συναρτήσεων, υφίσταται **εξάρτηση**

Αυτές οι εξαρτήσεις περιγράφονται από **γράφους εξάρτησης** οι οποίοι περιέχουν μία **κορυφή** για κάθε **μεταβλητή** και μία **ακμή** από την κορυφή $(j, \mathbf{q})$ προς στην κορυφή $(i, \mathbf{p})$, εάν η τιμή της συνάρτησης $\alpha_i(\mathbf{p})$ εξαρτάται από την τιμή της συνάρτησης $\alpha_j(\mathbf{q})$, με την **ετικέτα** αυτής της ακμής, να περιέχει το κατάλληλο **διάνυσμα εξάρτησης**, που νοείται ως μία **τιμή βάρους**.

Ο γράφος μπορεί να διαθέτει τόσο **βρόχους**, δηλαδή **ακμές που ξεκινούν και καταλήγουν στην ίδια κορυφή**, όσο και **κύκλους**, δηλαδή **κλειστές διαδρομές που ξεκινούν και καταλήγουν στην ίδια κορυφή**, αλλά διέρχονται **και από άλλες κορυφές**



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Η οργάνωση των υπολογισμών στις ομοιόμορφες αναδρομικές εξισώσεις

Διατυπώνοντας την **εξίσωση διαφορών του Laplace** με τη μορφή

$$T(i, j) = \frac{1}{4}[T(i + 1, j) + T(i - 1, j) + T(i, j + 1) + T(i, j - 1)]$$

και ορίζοντας τα διανύσματα **$\mathbf{p} = (i, j)$** , **$\mathbf{d}_1 = (-1, 0)$** , **$\mathbf{d}_2 = (1, 0)$** , **$\mathbf{d}_3 = (0, -1)$** και **$\mathbf{d}_4 = (0, 1)$** η παραπάνω σχέση γράφεται

$$T(\mathbf{p}) = f(T(\mathbf{p} - \mathbf{d}_1), T(\mathbf{p} - \mathbf{d}_2), T(\mathbf{p} - \mathbf{d}_3), T(\mathbf{p} - \mathbf{d}_4)) = \frac{1}{4}[T(\mathbf{p} - \mathbf{d}_1) + T(\mathbf{p} - \mathbf{d}_2) + T(\mathbf{p} - \mathbf{d}_3) + T(\mathbf{p} - \mathbf{d}_4)]$$

Κατά συνέπεια, στην προκειμένη περίπτωση, το σύστημα των **αναδρομικών εξισώσεων** περιέχει **μία απλή εξίσωση** με **συνάρτηση προς υπολογισμό** την **θερμοκρασία** $T(\mathbf{p})$ σε κάθε σημείο του πλέγματος $\mathbf{p}$

Προκειμένου **να είναι δυνατός** ο υπολογισμός της τιμής **$T(\mathbf{p})$** , θα πρέπει **να έχουν ήδη υπολογιστεί** και να είναι **διαθέσιμες**, οι τιμές **$T(\mathbf{p} - \mathbf{d}_1)$** , **$T(\mathbf{p} - \mathbf{d}_2)$** , **$T(\mathbf{p} - \mathbf{d}_3)$** και **$T(\mathbf{p} - \mathbf{d}_4)$** .

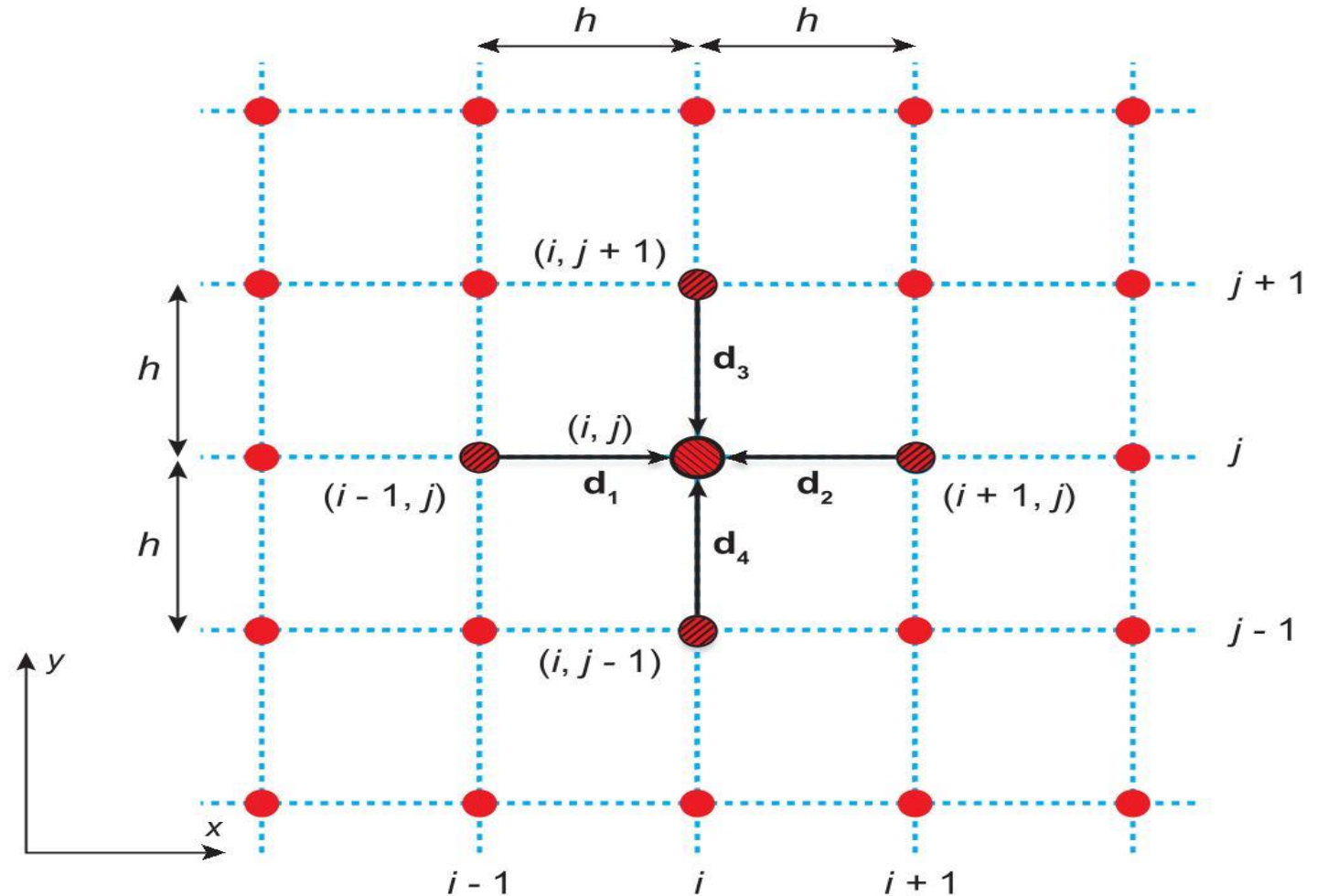
Με άλλα λόγια, η τιμή $T(\mathbf{p})$ **εξαρτάται από τις τέσσερις αυτές τιμές** και για τον λόγο αυτό, τα διανύσματα **$\mathbf{d}_1$** , **$\mathbf{d}_2$** , **$\mathbf{d}_3$** και **$\mathbf{d}_4$** που προσδιορίζουν **με διαγραμματικό τρόπο** αυτές τις εξαρτήσεις είναι γνωστά ως **διανύσματα εξάρτησης**



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Η οργάνωση των υπολογισμών στις ομοιόμορφες αναδρομικές εξισώσεις

Η **γεωμετρία** του προβλήματος της αριθμητικής επίλυσης της μερικής διαφορικής εξίσωσης του Laplace στην οποία απεικονίζονται τα **τέσσερα** διανύσματα εξάρτησης που χρησιμοποιούνται στους σχετικούς υπολογισμούς.



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Η οργάνωση των υπολογισμών στις ομοιόμορφες αναδρομικές εξισώσεις

Θεωρώντας τον **γράφο** του σχήματος, οι **αναδρομικές** εξισώσεις που αντιστοιχούν στην δομή του, είναι οι

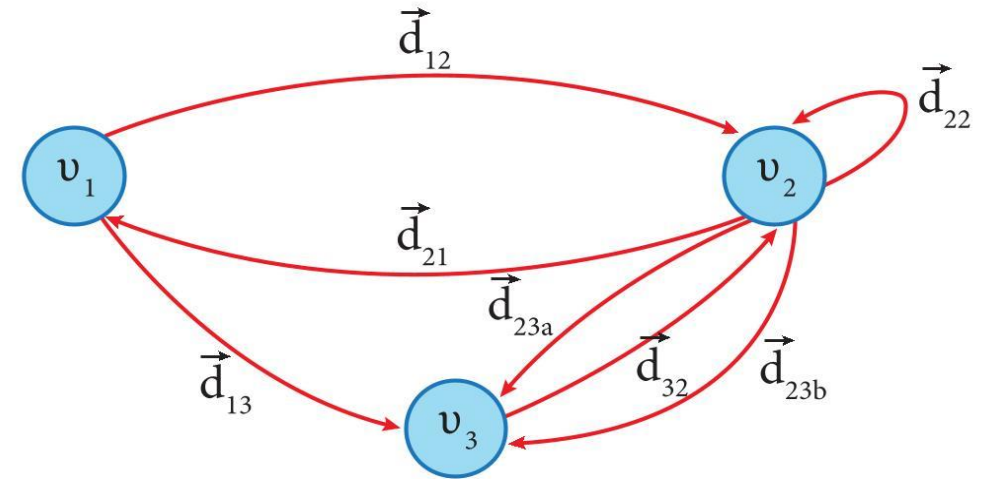
$$\alpha_1(\mathbf{p}) = f_1(\alpha_2(\mathbf{p} - \mathbf{d}_{12}), \alpha_3(\mathbf{p} - \mathbf{d}_{13}))$$

$$\alpha_2(\mathbf{p}) = f_2(\alpha_1(\mathbf{p} - \mathbf{d}_{21}), \alpha_2(\mathbf{p} - \mathbf{d}_{22}), \alpha_3(\mathbf{p} - \mathbf{d}_{23a}), \alpha_3(\mathbf{p} - \mathbf{d}_{23b}))$$

$$\alpha_3(\mathbf{p}) = f_3(\alpha_2(\mathbf{p} - \mathbf{d}_{32}))$$

Παρατηρούμε ότι:

- Ανάμεσα σε δύο κορυφές, μπορούν να οριστούν **περισσότερες από μία** ακμές
- Τα βέλη των ακμών **ξεκινούν** από την κορυφή που αντιστοιχεί στη συνάρτηση που βρίσκεται **στο αριστερό μέλος της κάθε εξίσωσης** και καταλήγουν στην κορυφή, της οποίας η αντίστοιχη συνάρτηση εμφανίζεται στην **λίστα των ορισμάτων** της **αντίστοιχης** συνάρτησης f
- Το **βάρος** κάθε τέτοιας ακμής, είναι το **διάνυσμα** $\mathbf{d}_{ij}$ που εμφανίζεται στο **κάθε όρισμα** της μορφής $\alpha_i(\mathbf{p} - \mathbf{d}_{ij})$, ενώ το **πλήθος** των ακμών που ξεκινούν από την **κάθε** κορυφή, είναι **ίσο με το πλήθος των ορισμάτων** της αντίστοιχης συνάρτησης f_i .
- Υπάρχουν ακμές οι οποίες **ξεκινούν** και **καταλήγουν** στην **ίδια κορυφή** του γράφου.



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Η οργάνωση των υπολογισμών στις ομοιόμορφες αναδρομικές εξισώσεις

Υπολογισιμότητα και χρονοδρομολόγηση

Υπάρχουν **δύο σημαντικά προβλήματα** που σχετίζονται με τα συστήματα ομοιόμορφων αναδρομικών εξισώσεων και το καθένα των οποίων αποδεικνύεται πως είναι το **δυϊκό** του άλλου :

το **πρόβλημα της υπολογισιμότητας**, που προσπαθεί να απαντήσει στο ερώτημα, εάν η **αναδρομική διαδικασία** που περιγράφεται από το σύστημα εξισώσεων σχετίζεται με **τιμές συναρτήσεων $\alpha_k(\mathbf{p})$** , οι οποίες να **μπορούν** να υπολογιστούν.

το **πρόβλημα της χρονοδρομολόγησης** που αφορά στον **τρόπο οργάνωσης** των υπολογισμών των τιμών **υπολογίσιμων συναρτήσεων** και ειδικότερα, στην κατασκευή μιας **συνάρτησης χρονοδρομολόγησης**, η οποία προσδιορίζει μία **λογική χρονική στιγμή εκτέλεσης** του καθενός από αυτούς τους υπολογισμούς, υποθέτοντας πως το υποκείμενο παράλληλο σύστημα διαθέτει **απεριόριστο πλήθος επεξεργαστών**, έτσι ώστε να είναι διαθέσιμοι όσοι επεξεργαστές είναι **αναγκαίοι**.

Η υλοποίηση της **χρονοδρομολόγησης** είναι **αναγκαία**, αφού η **αλγεβρική δομή** που περιγράφει τον χώρο επαναλήψεων, **δεν** περιέχει πληροφορίες σχετικά **με την σειρά** με την οποία θα πρέπει να πραγματοποιηθούν οι εμπλεκόμενοι υπολογισμοί.

Αμφότερα τα προβλήματα της **υπολογισιμότητας** και της **χρονοδρομολόγησης**, σχετίζονται με το γεγονός, πως ο υπολογισμός της τιμής των συναρτήσεων που βρίσκονται στα **αριστερά** μέλη των αναδρομικών εξισώσεων, απαιτεί τον **υπολογισμό** και την **διαθεσιμότητα** των τιμών όλων των παραστάσεων που εμφανίζονται στα **δεξιά μέλη** αυτών των εξισώσεων.



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Η οργάνωση των υπολογισμών στις ομοιόμορφες αναδρομικές εξισώσεις

Υπολογισιμότητα και χρονοδρομολόγηση

Στην πράξη, υφίστανται **δύο** δυνατότητες:

- είτε ο υπολογισμός όλων αυτών των τιμών, θα πραγματοποιηθεί σε **πεπερασμένο** χρονικό διάστημα (**υπολογισιμότητα**)
- είτε οι εξαρτήσεις είναι τέτοιες, ώστε το **χρονικό διάστημα αναμονής** της διαθεσιμότητας μιας τιμής που χρησιμοποιείται ως όρισμα σε κάποια συνάρτηση, να έχει **άπειρη** τιμή (**μη υπολογισιμότητα**)

Κατά συνέπεια, το πρόβλημα της **υπολογισιμότητας**, χαρακτηρίζει μία **αναδρομική** διαδικασία ως **υπολογίσιμη** ή **όχι**.

Εάν η διαδικασία χαρακτηριστεί ως **υπολογίσιμη**, τότε εμφανίζεται στο προσκήνιο το πρόβλημα της **χρονοδρομολόγησης**, που σχετίζεται με τον **προσδιορισμό** των **χρονικών στιγμών εκτέλεσης** του υπολογισμού της κάθε τιμής $\alpha_i(\mathbf{p})$ ($1 \leq i \leq n$) και μάλιστα με **τέτοιο τρόπο** ώστε να **μη παραβιάζονται** οι υφιστάμενες εξαρτήσεις)

Το πρόβλημα της χρονοδρομολόγησης **απλοποιείται** χρησιμοποιώντας συναρτήσεις **γραμμικής χρονοδρομολόγησης** οι οποίες επιτρέπουν την διατύπωση ενός ένθετου βρόχου βάθους $d = n$, ως μία δομή η οποία αποτελείται από έναν **εξωτερικό σειριακό βρόχο** και από $n-1$ **εσωτερικούς παράλληλους** βρόχους.



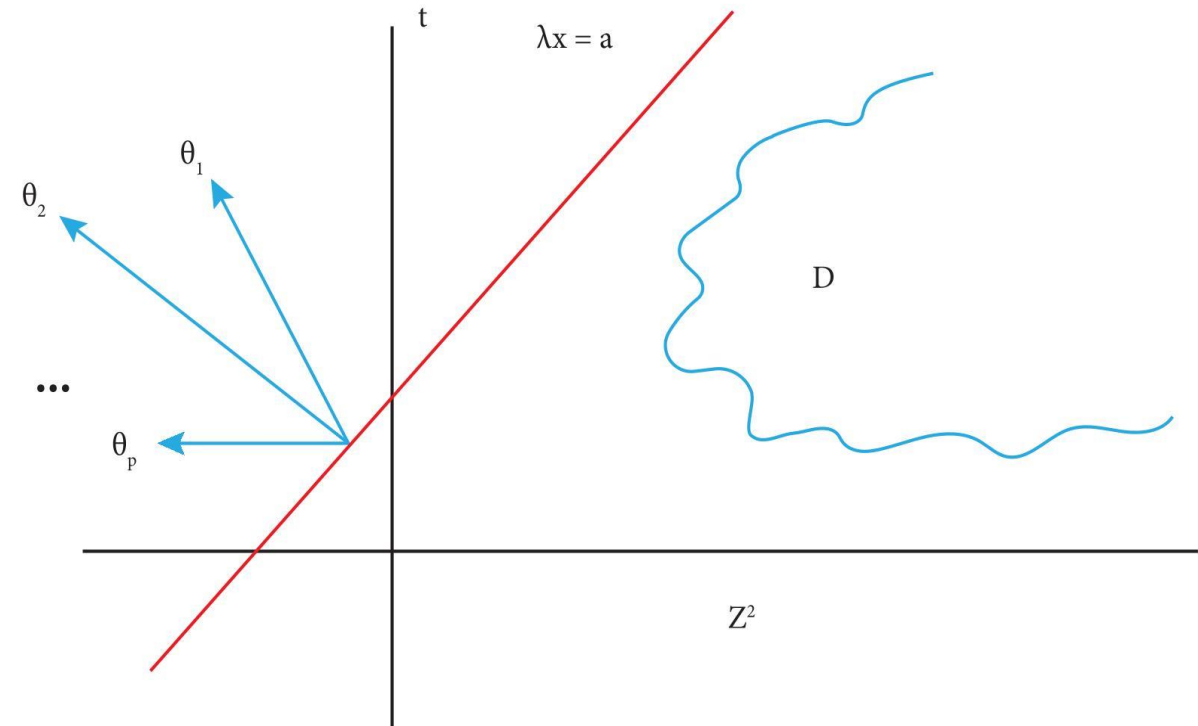
Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Η οργάνωση των υπολογισμών στις ομοιόμορφες αναδρομικές εξισώσεις

Υπολογισιμότητα και χρονοδρομολόγηση

Αυτές οι συναρτήσεις συνήθως αποτελούν **αφινικές** απεικονίσεις $S : V \times \mathbb{Z}^n \rightarrow Q$ της μορφής $S(u, \mathbf{p}) = \mathbf{X}_u \mathbf{p} + \rho_u$ όπου $\mathbf{X}_u \in Q^n$ το **διάνυσμα χρονοδρομολόγησης**, που ικανοποιεί την ιδιότητα $\mathbf{X}_u \mathbf{D} \geq \mathbf{1}$ έτσι ώστε να διασφαλίζεται η διατήρηση των εξαρτήσεων, με τον πίνακα $\mathbf{D}$ που περιέχει ως **στήλες** τα **διανύσματα εξάρτησης** να είναι γνωστός ως **πίνακας εξάρτησης**.

Αποδεικνύεται πως η δυνατότητα **προσδιορισμού** μιας **συνάρτησης χρονοδρομολόγησης** που καθιστά **υπολογίσιμο** έναν **γράφο εξάρτησης**, είναι **ισοδύναμη** με την ύπαρξη ενός **υπερεπιπέδου** που διαχωρίζει τον χώρο σε **δύο ημιχώρους**, εκ των οποίων ο πρώτος περιέχει **το σύνολο των σημείων επαναλήψεων του αλγορίθμου**, ενώ ο δεύτερος τα **διανύσματα εξάρτησης**.



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Η οργάνωση των υπολογισμών στις ομοιόμορφες αναδρομικές εξισώσεις

Υπολογισιμότητα και χρονοδρομολόγηση

Για την **μαθηματική περιγραφή** της χρονοδρομολόγησης, ας ορίσουμε την **σχέση προήγησης**, ως μία σχέση που υφίσταται **ανάμεσα σε δύο υπολογισμούς** και προσδιορίζει **ποιος** από αυτούς θα εκτελεστεί **πρώτος**.

Θεωρώντας ένα **διατεταγμένο** ζεύγος (k, p) του συνόλου $\{1, 2, 3, \dots, m\} \times P$ θα λέμε πως αυτό εξαρτάται **απευθείας** από το ζεύγος (ℓ, q) και θα γράφουμε

$$(k, p) \xrightarrow{1} (\ell, q)$$

αν και μόνο αν είναι $p \in P$ και **ταυτόχρονα** ο γράφος προήγησης περιέχει μία ακμή που κατευθύνεται από την κορυφή u_k προς την κορυφή u_ℓ , τέτοια ώστε να είναι $p - d_{k\ell} = q$ (για κάποιο **διάνυσμα εξάρτησης** d), δηλαδή αν και μόνο αν η συνάρτηση $\alpha_\ell(q)$ είναι **ένα από τα ορίσματα** που χρησιμοποιούνται για τον υπολογισμό της συνάρτησης $\alpha_k(p)$.

Εάν αυτά τα δύο ζεύγη **ταυτίζονται**, δηλαδή εάν είναι $k = \ell$ και $p = q$ θα γράφουμε $(k, p) \xrightarrow{0} (\ell, q)$

Στην περίπτωση μιας t -εξάρτησης, θα λέμε ότι $(k, p) \xrightarrow{t} (\ell, q)$ εάν υπάρχει ζεύγος (h, r) τέτοιο ώστε $(k, p) \xrightarrow{t-1} (h, r)$ και $(h, r) \xrightarrow{1} (\ell, q)$ με το **βασικό βήμα** αυτής της **επαγωγικής** διαδικασίας, να είναι η συνθήκη $(k, p) \xrightarrow{0} (\ell, q)$

Εάν ισχύει η συνθήκη $(k, p) \xrightarrow{t} (\ell, q)$ συντομογραφικά θα γράφουμε $(k, p) \longrightarrow (\ell, q)$



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Η οργάνωση των υπολογισμών στις ομοιόμορφες αναδρομικές εξισώσεις

Μονοδιάστατη και πολυδιάστατη χρονοδρομολόγηση

Η **χρονοδρομολόγηση** μιας διαδικασίας **υπολογισμού**, ορίζεται, ως μία **συνάρτηση** της μορφής $\{ 1,2,3,...,m \} \times P \rightarrow \mathbb{Z}^+$ η οποία δέχεται ως είσοδο ένα **διατεταγμένο ζεύγος** (k,p) που εκφράζει τον **υπολογισμό** της τιμής της μεταβλητής $\alpha_k(p)$ και επιστρέφει την **χρονική στιγμή πραγματοποίησης** αυτού του υπολογισμού, $S(k,p)$.

Προκειμένου μία **συνάρτηση χρονοδρομολόγησης** να αποτελεί μία **έγκυρη** συνάρτηση, θα πρέπει να είναι τέτοια ώστε εάν είναι $(k,p) \rightarrow (\ell,q)$, τότε να είναι και $S(k,p) > S(\ell,q)$ έτσι ώστε να **διατηρούνται** οι **υφιστάμενες εξαρτήσεις**.

Εάν το χρονικό διάστημα εκτέλεσης των υπολογισμών είναι ίσο με 1 θα είναι $S(k,p) \geq S(\ell,q) + 1$ (**συνθήκη αιτιότητας**)

Εκτός από την παραπάνω **μονοδιάστατη χρονοδρομολόγηση**, ορίζεται και η **πολυδιάστατη χρονοδρομολόγηση**, στην οποία η λογική χρονική στιγμή εκτέλεσης των υπολογισμών που επιστρέφεται από αυτή, **δεν αποτελεί** ένα απλό βαθμωτό μέγεθος.

Αντίθετα, αποτελεί ένα διάνυσμα d τιμών, οι οποίες μπορούν να **ερμηνευθούν** με τον τρόπο που ερμηνεύεται μία **ημερομηνία**: δηλαδή, η **πιο σημαντική** από αυτές τις τιμές αναπαριστά τις **ημέρες**, ενώ οι υπόλοιπες τιμές αναπαριστούν, αντίστοιχα, τις **ώρες**, τα **λεπτά**, τα **δευτερόλεπτα**, κ.ο.κ.

Αυτός ο τύπος χρονοδρομολόγησης αφορά στις περιπτώσεις εφαρμογών για τις οποίες η κατασκευή μιας **αφινικής συνάρτησης χρονοδρομολόγησης**, δεν είναι δυνατή και απαιτεί τον ορισμό μιας **διάταξης** (συνήθως χρησιμοποιείται η **λεξικογραφική διάταξη**) για τον ορισμό των σχέσεων του **μικρότερου** και του **μεγαλύτερου** για τις οριζόμενες **πλειάδες**.



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Η οργάνωση των υπολογισμών στις ομοιόμορφες αναδρομικές εξισώσεις

Επιλύοντας το πρόβλημα της χρονοδρομολόγησης

Στην πλέον γενική των περιπτώσεων, το πρόβλημα του προσδιορισμού αφινικών συναρτήσεων χρονοδρομολόγησης τόσο για την μονοδιάστατη όσο και για την πολυδιάστατη περίπτωση αφορά σε μία (δυνητικά) άπειρη οικογένεια γράφων εξάρτησης, ο κάθε γράφος της οποίας περιγράφεται από ένα διάνυσμα παραμέτρων $\mathbf{n}$.

Κατά συνέπεια, η ζητούμενη αφινική συνάρτηση χρονοδρομολόγησης έχει τη μορφή

$$S(v, \mathbf{p}) = \mathbf{X}_v \cdot \mathbf{p} + \sigma_v \cdot \mathbf{n} + \rho_v$$

όπου $\mathbf{X}_v$ και σ_v άγνωστα διανύσματα με σταθερές ρητές συντεταγμένες και ρ_v άγνωστη σταθερά.

Ο άμεσος τρόπος που συνίσταται στην κατασκευή και επίλυση του συστήματος των γραμμικών ανισοτήτων που προκύπτουν αντικαθιστώντας τις κατάλληλες τιμές παραμέτρων στην συνθήκη αιτιότητας για την κάθε ακμή του γράφου, είναι πρακτικά ανεφάρμοστος αφού οδηγεί σε πολύ μεγάλα συστήματα και ο Feautrier παραθέτει δύο εναλλακτικές μεθόδους:

- μία μέθοδο που είναι γνωστή ως μέθοδος των κορυφών
- μία μέθοδο, που στηρίζεται στην αφινική μορφή του λήμματος του Farkas



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Η οργάνωση των υπολογισμών στις ομοιόμορφες αναδρομικές εξισώσεις

Επιλύοντας το πρόβλημα της χρονοδρομολόγησης

Η μέθοδος των κορυφών συνίσταται στον προσδιορισμό του συστήματος κατασκευής για το κατάλληλο σύνολο πολυέδρων και στην κατασκευή και επίλυση του κατάλληλου συστήματος ανισοτήτων για τις κορυφές των εμπλεκόμενων πολυέδρων

Η μέθοδος του Farkas στηρίζεται στην παρατήρηση, πως μία αφινική συνάρτηση χρονοδρομολόγησης λαμβάνει μη αρνητικές τιμές αν και μόνο αν υπάρχουν συντελεστές μ_{sk} οι οποίοι είναι γνωστοί ως πολλαπλασιαστές Farkas τέτοιοι ώστε

$$S(v, \mathbf{p}) = \mu_{S_0} + \sum_{k=1}^{m_S} \underbrace{\mu_{S_k} \left(a_{S_k} \begin{pmatrix} x \\ n \end{pmatrix} + b_{S_k} \right)}_A$$

Οι παραπάνω συντελεστές προκύπτουν από την επίλυση ενός συστήματος γραμμικών εξισώσεων με αγνώστους τους πολλαπλασιαστές Farkas, οι οποίοι είναι εγγυημένο πως θα έχουν θετική τιμή (όπως απαιτείται).

Έχοντας υπολογίσει αυτούς τους συντελεστές, μπορούμε να προσδιορίσουμε τη ζητούμενη συνάρτηση χρονοδρομολόγησης.

Καταφεύγοντας σε μία τεχνική απαλοιφής Gauss-Jordan, μπορούμε να μειώσουμε σημαντικά το μέγεθος του προβλήματος και να καταστήσουμε τον αλγόριθμο ακόμη πιο αποδοτικό.

Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Η οργάνωση των υπολογισμών στις ομοιόμορφες αναδρομικές εξισώσεις

Επιλογή της βέλτιστης συνάρτησης χρονοδρομολόγησης

Το πλήθος των **έγκυρων συναρτήσεων** χρονοδρομολόγησης **δεν μπορεί** προφανώς να **καταμετρηθεί** και για τον λόγο αυτό, θα πρέπει να επιλέξουμε εκείνη τη συνάρτηση που θεωρούμε **ως πιο κατάλληλη** για την περίπτωση μας.

Μία τεχνική (**Feautrier**) που προσπαθεί να προσδιορίσει απευθείας την **βέλτιστη συνάρτηση** αφινικής χρονοδρομολόγησης, στηρίζεται σε μία **σχέση διάταξης** που ορίζεται ανάμεσα στις **συναρτήσεις χρονοδρομολόγησης** της μορφής

$$S_1 \prec S_2 \equiv \forall p \in R : S_1(p) \leq S_2(p)$$

και σε ένα **θεώρημα** σύμφωνα με το οποίο, εάν οι συναρτήσεις **S_1** και **S_2** ικανοποιούν την συνθήκη της **αιτιότητας** για έναν **γενικευμένο γράφο εξάρτησης**, τότε **το ίδιο** ισχύει και για τη συνάρτηση **$S_3(p) = \min [S_1(p), S_2(p)]$** . Καταφεύγοντας στην **αφινική μορφή** του **λήμματος του Farkas**, οδηγούμαστε σε ένα **γραμμικό πρόγραμμα** της μορφής

$$S(v, x) = \min \mu_{S_0} + \sum_{k=1}^{m_S} \nu_{S_k} \left(a_{S_k} \binom{x}{n} + b_{S_k} \right)$$
$$\mu \geq 0, \quad \lambda \geq 0$$
$$G \begin{pmatrix} \mu \\ \lambda \end{pmatrix} \geq 0$$

όπου **μ** και **λ** είναι **διανύσματα**, οι συνιστώσες των οποίων αποτελούν **πολλαπλασιαστές Farkas**.



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Η οργάνωση των υπολογισμών στις ομοιόμορφες αναδρομικές εξισώσεις

Επιλογή της βέλτιστης συνάρτησης χρονοδρομολόγησης

Προκειμένου να λάβουμε για την **συνάρτηση** $S(u, x)$, μία λύση **εκπεφρασμένη σε κλειστή μορφή**, κατασκευάζουμε το **δυϊκό πρόγραμμα** του παραπάνω προγράμματος που έχει τη μορφή

$$S(v, x) = \max \nu \cdot h, \quad \nu \geq 0$$
$$\nu G \leq \underbrace{(0, 0, 0, \dots, 0)}_N, 1, a_{S_1} \binom{x}{n} + b_{S_1}, \dots, a_{S_{m_S}} \binom{x}{n} + b_{S_{m_S}}, \underbrace{(0, 0, 0, \dots, 0)}_{N'}$$

με τις παραμέτρους **N** και **N'** να ορίζονται ως

$$N = \sum_{R < S} (m_R + 1) \quad \text{και} \quad N' = \sum_{S < T} (m_T + 1) + |\lambda|$$

Για την **ολοκλήρωση** της διαδικασίας θα πρέπει να κατασκευάσουμε ένα τέτοιο πρόβλημα **για κάθε δήλωση** S και να επιλύσουμε το καθένα από αυτά τα προβλήματα **σε κλειστή μορφή**, οδηγούμενοι με τον τρόπο αυτό στην τελική λύση



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Η οργάνωση των υπολογισμών στις ομοιόμορφες αναδρομικές εξισώσεις

Χρονική υστέρηση και ασυμπτωτική ευστάθεια συνάρτηση χρονοδρομολόγησης

Η **αξιολόγηση** της **απόδοσης** μιας **συνάρτησης χρονοδρομολόγησης** S , συνίσταται στην καταμέτρηση του **πλήθους** των υπολογισμών που απαιτούνται για την εκτέλεση του συστήματος των **ομοιόμορφων εξισώσεων διαφορών**, στον κατάλληλο σε κάθε περίπτωση **χώρο επαναλήψεων** και κατά τις **χρονικές στιγμές** που υπαγορεύονται από τη συνάρτηση S .

Εάν ο κάθε υπολογισμός πραγματοποιείται σε **μοναδιαίο χρονικό διάστημα**, αυτό το πλήθος υπολογισμών αντιστοιχεί σε ένα χρονικό διάστημα εκτέλεσης, που είναι γνωστό ως η **χρονική υστέρηση** $L(S,P)$ της συνάρτησης χρονοδρομολόγησης

Εάν ενδιαφερόμαστε για **υπολογισμούς μεγάλου όγκου**, μπορούμε να καταφύγουμε στην χρήση ενός άλλου **παράγοντα ποιότητας**, ο οποίος είναι γνωστός ως **ασυμπτωτική ευστάθεια** και δίδεται από την εξίσωση₁

$$\varepsilon_p = \frac{1}{1 + \nu P \frac{\mathcal{L}(S,P)}{\tau}}$$

όπου P είναι το **πλήθος των επεξεργασιών** που υπάρχουν στο παράλληλο σύστημα, ν είναι η **χρονική διάρκεια των διαδικασιών συγχρονισμού** και T είναι ο **συνολικός όγκος υπολογισμών**

Στην περίπτωση αυτή, ο λόγος $T/L(S,P)$ μπορεί να θεωρηθεί ως ο **μέσος βαθμός παραλληλισμού** που σχετίζεται με την συνάρτηση χρονοδρομολόγησης και η **μέγιστη τιμή** του οποίου αποτελεί **χαρακτηριστικό** του προγράμματος που εκτελείται.



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Η οργάνωση των υπολογισμών στις ομοιόμορφες αναδρομικές εξισώσεις

Η συνάρτηση ελεύθερης χρονοδρομολόγησης

Η συνάρτηση ελεύθερης χρονοδρομολόγησης αντιστοιχεί στην περίπτωση κατά την οποία, εάν δεν υπάρχει ζεύγος $(\ell, \mathbf{q})$ τέτοιο ώστε να είναι $\mathbf{q} \in R$ και $(k, \mathbf{p}) \rightarrow (\ell, \mathbf{q})$,¹ τότε θα ισχύει ότι $T(k, \mathbf{p}) = 1$ ενώ σε κάθε άλλη περίπτωση θα είναι

$$T(k, \mathbf{p}) = 1 + \max\{T(\ell, \mathbf{q}) \mid \mathbf{q} \in R \text{ και } (k, \mathbf{p}) \xrightarrow{1} (\ell, \mathbf{q})\}$$

Εάν ορίζεται κάποια συνάρτηση χρονοδρομολόγησης θα ορίζεται και μία μοναδική συνάρτηση ελεύθερης χρονοδρομολόγησης, η οποία αποτελεί και την πιο γρήγορη χρονοδρομολόγηση, υπό την έννοια πως ορίζει ως χρονική στιγμή εκτέλεσης ενός υπολογισμού, την νωρίτερη δυνατή χρονική στιγμή κατά την οποία αυτό είναι δυνατόν να συμβεί.

Δηλαδή, από όλες τις συναρτήσεις χρονοδρομολόγησης, η συνάρτηση ελεύθερης χρονοδρομολόγησης είναι αυτή που οδηγεί στην μικρότερη χρονική υστέρηση, αλλά και στον μέγιστο βαθμό παραλληλισμού

Η συνάρτηση ελεύθερης χρονοδρομολόγησης υπάρχει αν και μόνο αν για κάθε $(k, \mathbf{p})$ τέτοιο ώστε $\mathbf{p} \in R$, ορίζεται η συνάρτηση

$$F(k, \mathbf{p}) = 1 + \max\{t \geq 0 \mid \text{υπάρχει } (\ell, \mathbf{q}) : (k, \mathbf{p}) \xrightarrow{t} (\ell, \mathbf{q})\}$$

Εάν υπάρχει συνάρτηση ελεύθερης χρονοδρομολόγησης, αυτή απλά ορίζεται ως $T(k, \mathbf{p}) = F(k, \mathbf{p})$.



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Η οργάνωση των υπολογισμών στις ομοιόμορφες αναδρομικές εξισώσεις

Η συνάρτηση ελεύθερης χρονοδρομολόγησης

Προκειμένου να **γενικεύσουμε** αυτή την περιγραφή και να ορίσουμε τη συνάρτηση **$T(k,p)$** ακόμη και όταν η συνάρτηση ελεύθερης χρονοδρομολόγησης **δεν υφίσταται**, μπορούμε πολύ απλά να γράψουμε,

$$T(k,p) = \begin{cases} F(k,p) & \text{εάν ορίζεται η συνάρτηση } F(k,p) \\ \infty & \text{στην αντίθετη περίπτωση} \end{cases}$$

Αυτός ο ορισμός καλύπτει όλες τις περιπτώσεις, με την συνθήκη **$T(k,p) = F(k,p)$** να περιγράφει την κλάση των **υπολογίσιμων συναρτήσεων** και την συνθήκη **$T(k,p) = \infty$** να περιγράφει την κλάση των **μη υπολογίσιμων συναρτήσεων**

Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Η οργάνωση των υπολογισμών στις ομοιόμορφες αναδρομικές εξισώσεις

Υπολογισιμότητα και γράφοι προήγησης

Χρησιμοποιώντας την έννοια του **γράφου προήγησης** και θεωρώντας πως το **χρονικό διάστημα εκτέλεσης** ενός υπολογισμού είναι ίσο με **μία μονάδα χρόνου**, μπορούμε να περιγράψουμε την χρονική στιγμή εκτέλεσης ενός υπολογισμού, ως **συνάρτηση** του **μήκους της διαδρομής** που ορίζεται πάνω στον γράφο, η οποία έχει ως **αφετηρία** την κορυφή που αντιστοιχεί στην συνάρτηση προς υπολογισμό και διασχίζεται κατά την **φορά απομάκρυνσης** από αυτή την κορυφή.

Στην περίπτωση αυτή, η τιμή της συνάρτησης **ελεύθερης χρονοδρομολόγησης** $T(k, p)$, ορίζεται ως το **μέγιστο μήκος της διαδρομής** επί του γράφου προήγησης, η οποία διασχίζεται κατά την **φορά απομάκρυνσης** από την κορυφή $a_i(p)$.

Εάν **δεν υπάρχει** μία τέτοια **μέγιστη** διαδρομή ή ισοδύναμα, όταν το μήκος μιας τέτοιας διαδρομής **δεν είναι φραγμένο**, τότε θέτουμε $T(k, p) = \infty$ και χαρακτηρίζουμε την εμπλεκόμενη συνάρτηση ως **μη υπολογίσιμη**.

Αντίστροφα, προκειμένου μία συνάρτηση να χαρακτηριστεί ως **υπολογίσιμη**, δεν θα πρέπει να υφίσταται μία διαδρομή επί του γράφου εξάρτησης που να **ξεκινά** από την **κορυφή** που αντιστοιχεί σε αυτή τη συνάρτηση και να χαρακτηρίζεται από **άπειρο μήκος** όπως συμβαίνει εάν η διαδρομή περιέχει **κύκλους**.

Ένας γράφος στον οποίο **δεν υπάρχουν** τέτοιες διαδρομές άπειρου μήκους, χαρακτηρίζεται ως **συνεπής**

Το πρόβλημα του χαρακτηρισμού ενός γράφου **ως προς τη συνέπειά του**, είναι **διαγνώσιμο** για την περίπτωση των **συστημάτων ομοιόμορφων αναδρομικών εξισώσεων** κάτι που όμως **δεν ισχύει** για άλλες περιπτώσεις.



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Η οργάνωση των υπολογισμών στις ομοιόμορφες αναδρομικές εξισώσεις

Ορίζοντας τον βαθμό του παραλληλισμού

Ο **βαθμός του παραλληλισμού** ορίζεται ως το **πλήθος** των υπολογισμών $\alpha_k(\mathbf{p})$ για τους οποίους η **συνάρτηση χρονοδρομολόγησης** $S(k, \mathbf{p})$ επιστρέφει **την ίδια** χρονική στιγμή εκτέλεσης και επομένως μπορούν να εκτελεστούν **ταυτόχρονα**.

Λαμβάνοντας υπόψη πως κάθε **συνάρτηση χρονοδρομολόγησης** $S(k, \mathbf{p})$ σχετίζεται με ένα σημείο $\mathbf{p} \in R$, ο βαθμός του παραλληλισμού ορίζεται ως το **πλήθος των σημείων** της περιοχής R , των οποίων οι αντίστοιχοι υπολογισμοί θα εκτελεστούν **την ίδια χρονική** στιγμή τ , ή σε μαθηματική γραφή

$$\phi_S(k, \tau) = \|\{\mathbf{p} \mid S(k, \mathbf{p}) = \tau\}\|, \quad \begin{matrix} k = 1, 2, \dots, m \\ \tau = 1, 2, \dots \end{matrix}$$

Μία συνάρτηση **χρονοδρομολόγησης** S , χαρακτηρίζεται από **φραγμένο παραλληλισμό**, εάν υπάρχει **ακέραιος** K τέτοιος ώστε να είναι $|\phi_S(k, \tau)| < K$ για **όλες** τις τιμές των K και τ ενώ στην **αντίθετη** περίπτωση, ο παραλληλισμός χαρακτηρίζεται ως **μη φραγμένος**, κάτι που σημαίνει πως

- είτε η συνάρτηση $\phi_S(k, \tau)$ λαμβάνει **άπειρη τιμή** για κάποιο **ζεύγος τιμών** (k, τ)
- είτε η τιμή αυτής της συνάρτησης **αυξάνει απεριόριστα** για κάποιες τιμές του k

Σε περιπτώσεις κατά τις οποίες ο βαθμός του παραλληλισμού που περιέχεται σε ένα σύστημα **ομοιόμορφων αναδρομικών** εξισώσεων **δεν είναι ικανοποιητικός**, μπορούμε να καταφύγουμε σε μία ενδιαφέρουσα **επέκταση** της χρονοδρομολόγησης που οδηγεί στην έννοια της **πολυδιάστατης χρονοδρομολόγησης**.

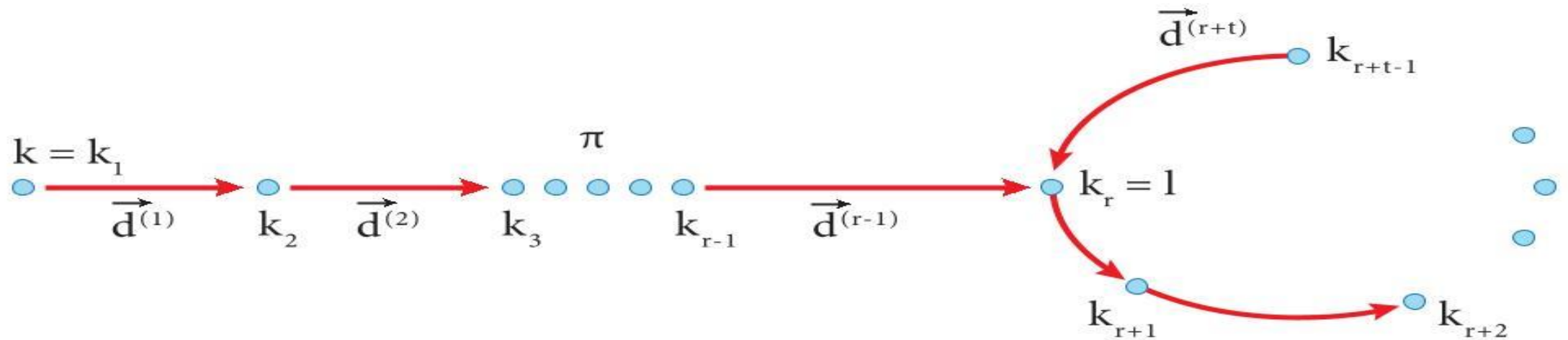


Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Η οργάνωση των υπολογισμών στις ομοιόμορφες αναδρομικές εξισώσεις

Προσδιορίζοντας τις συνθήκες υπολογισιμότητας

Θεώρημα: Θεωρώντας έναν γράφο εξάρτησης που περιγράφει συστήματα ομοιόμορφων αναδρομικών εξισώσεων που ορίζονται σε μία περιοχή R , μία συνάρτηση $\alpha_k(\mathbf{p})$ είναι υπολογίσιμη, αν και μόνο αν δεν υφίσταται διαδρομή από την αντίστοιχη κορυφή u_k προς μία οποιαδήποτε άλλη κορυφή του γράφου u_i , η οποία να περιλαμβάνεται σε μη θετικό κύκλο.



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Η οργάνωση των υπολογισμών στις ομοιόμορφες αναδρομικές εξισώσεις

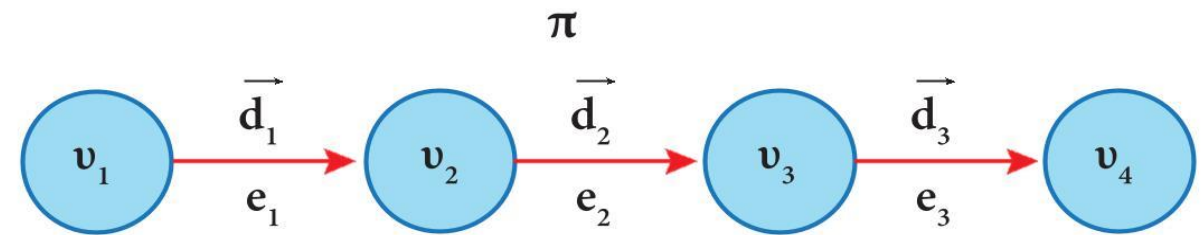
Προσδιορίζοντας τις συνθήκες υπολογισιμότητας

Θεωρώντας μία **διαδρομή** επί του γράφου εξάρτησης της μορφής $\pi = (e_1, e_2, \dots, e_n)$ ορίζουμε το **βάρος** αυτής της διαδρομής ως

$$w(\pi) = \sum_{i=1}^n w(e_i) = \sum_{i=1}^n w_i$$

όπου $w_i = w(e_i)$ το **βάρος** της ακμής e_i και με την πράξη της **πρόσθεσης** να πραγματοποιείται ανάμεσα στις **αντίστοιχες συνιστώσες** των διανυσμάτων

Εάν ένα διάνυσμα είναι **μη θετικό** όταν **όλες** οι συντεταγμένες του είναι **μη θετικές** (δηλαδή έχουν **αρνητική** ή **μηδενική** τιμή), μία διαδρομή είναι **μη θετική**, εάν το βάρος της $w(\pi)$ είναι ένα **μη θετικό** διάνυσμα.



$$\vec{d}_1 = \begin{pmatrix} d_{11} \\ d_{12} \\ d_{13} \end{pmatrix}$$

$$\vec{d}_2 = \begin{pmatrix} d_{21} \\ d_{22} \\ d_{23} \end{pmatrix}$$

$$\vec{d}_3 = \begin{pmatrix} d_{31} \\ d_{32} \\ d_{33} \end{pmatrix}$$

$$\vec{w}(\pi) = \sum_{i=1}^3 \vec{d}_i = \vec{d}_1 + \vec{d}_2 + \vec{d}_3 =$$

$$= \begin{pmatrix} w_{11} + w_{21} + w_{31} \\ w_{12} + w_{22} + w_{32} \\ w_{13} + w_{23} + w_{33} \end{pmatrix} = \begin{pmatrix} \pi_{11} \\ \pi_{12} \\ \pi_{13} \end{pmatrix}$$

Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Η οργάνωση των υπολογισμών στις ομοιόμορφες αναδρομικές εξισώσεις

Προσδιορίζοντας τις συνθήκες υπολογισιμότητας

Στην περίπτωση μιας απλής ομοιόμορφης αναδρομικής εξίσωσης, το πρόβλημα της υπολογισιμότητας της συνάρτησης $\alpha(\mathbf{p})$ στην κατάλληλη περιοχή, είναι ισοδύναμο με την ύπαρξη μιας συνάρτησης γραμμικής χρονοδρομολόγησης

Οι ικανές και αναγκαίες συνθήκες που διασφαλίζουν την υπολογισιμότητα της συνάρτησης για αυτή την περίπτωση, υπαγορεύονται από το επόμενο θεώρημα:

Θεώρημα: Θεωρώντας μία απλή ομοιόμορφη αναδρομική εξίσωση της μορφής

$$\alpha(\mathbf{p}) = f(\alpha(\mathbf{p} - \mathbf{d}_1), \alpha(\mathbf{p} - \mathbf{d}_2), \dots, \alpha(\mathbf{p} - \mathbf{d}_m))$$

στην οποία τα διανύσματα $\mathbf{p}$ και $\mathbf{d}_i$ ($1 \leq i \leq m$) που εμφανίζονται σε αυτή, είναι διανύσματα n ακεραίων, αποδεικνύεται πως οι επόμενες δηλώσεις είναι λογικά ισοδύναμες η μία ως προς την άλλη:



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Η οργάνωση των υπολογισμών στις ομοιόμορφες αναδρομικές εξισώσεις

Προσδιορίζοντας τις συνθήκες υπολογισιμότητας

(1) Η συνάρτηση $\alpha(p)$ είναι μία υπολογίσιμη συνάρτηση και κατά συνέπεια, επιδέχεται χρονοδρομολόγηση σε όλα τα σημεία p που ανήκουν στην περιοχή ενδιαφέροντος.

(2) Δεν υπάρχει ημιθετικό διάνυσμα - γραμμή u με συνιστώσες $(u_1, u_2, \dots, u_n)$ τέτοιο ώστε

$$\sum_{j=1}^n -u_j d_j \geq 0$$

(3) Το σύστημα των ανισοτήτων $d_j x \geq 1$ ($1 \leq j \leq m$) και $x_i \geq 0$ ($1 \leq i \leq n$) διαθέτει λύση. Από τις δύο αυτές ανισότητες η δεύτερη υποδηλώνει πως το διάνυσμα x που αποτελεί ένα διάνυσμα - γραμμή n στοιχείων είναι ένα ημι-θετικό διάνυσμα, ενώ η πρώτη μπορεί πιο συνοπτικά να γραφεί ως $x D \geq 1$ όπου x διάνυσμα - γραμμή με n ακέραια στοιχεία, D ο πίνακας εξάρτησης διαστάσεων $n \times m$ που περιέχει ως στήλες τα διανύσματα εξάρτησης d_j ($1 \leq j \leq m$) και 1 ένα διάνυσμα στήλη m στοιχείων, οι συνιστώσες του οποίου έχουν όλες τιμή ίση με τη μονάδα. Το σύνολο των διανυσμάτων x που ικανοποιούν αυτή την ιδιότητα συμβολίζεται με $T(D)$ - θα είναι λοιπόν

$$T(D) = \{x \in \mathbb{Z}^n \mid x D \geq 1\}$$



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Η οργάνωση των υπολογισμών στις ομοιόμορφες αναδρομικές εξισώσεις

Προσδιορίζοντας τις συνθήκες υπολογισιμότητας

(4) Για κάθε διάνυσμα $\mathbf{p} \in F_n$ όπου F_n είναι το σύνολο των διανυσμάτων n στοιχείων με θετικές συνιστώσες, τα επόμενα δύο γραμμικά προγράμματα το καθένα των οποίων είναι το δυϊκό του άλλου

$$I \begin{cases} p - \sum_{j=1}^m u_j d_j \geq 0 \\ u_j \geq 0 (j = 1, 2, \dots, m) \\ \max \sum_{j=1}^m u_j \end{cases} \quad \Pi \begin{cases} d_j \cdot x \geq 1 (j = 1, 2, \dots, m) \\ x_i \geq 0 (i = 1, 2, \dots, n) \\ \min p \cdot x \end{cases}$$

έχουν μία κοινή βέλτιστη τιμή $m(\mathbf{p})$. Αυτό με άλλα λόγια σημαίνει, πως η κατασκευή μιας γραμμικής συνάρτησης χρονοδρομολόγησης (η οποία μάλιστα είναι αποδεικνύεται πως είναι βέλτιστη) για την περίπτωση μιας απλής ομοιόμορφης αναδρομικής εξίσωσης, είναι δυνατή, αν και μόνο αν ικανοποιείται αυτή η συνθήκη.

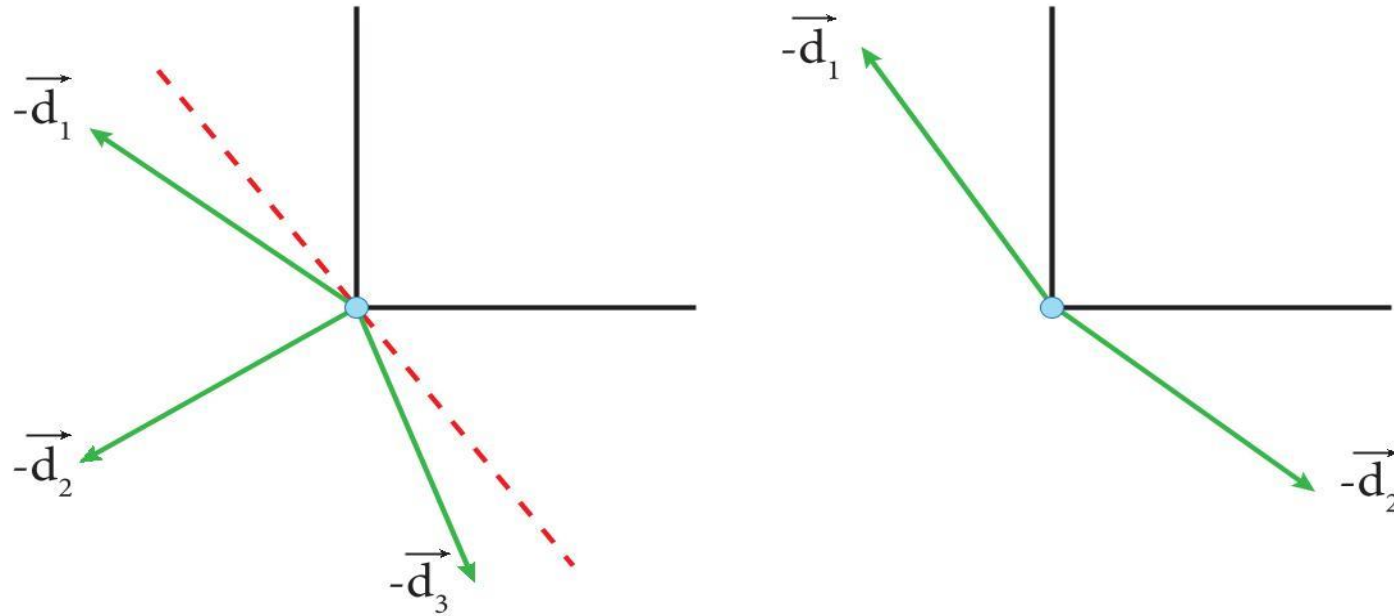
Η τρίτη συνθήκη του θεωρήματος επιδέχεται μία γεωμετρική ερμηνεία, η οποία συνηγορεί υπέρ της ύπαρξης ενός υπερεπιπέδου n διαστάσεων, που διέρχεται από την αρχή του συστήματος συντεταγμένων και διαχωρίζει το πρώτο τεταρτημόριο n διαστάσεων από τα διανύσματα $-\mathbf{d}_1, -\mathbf{d}_2, \dots, -\mathbf{d}_m$ και το οποίο ονομάζεται επίπεδο διαχωρισμού (αντίστοιχα, το διάνυσμα που είναι κάθετο σε αυτό το επίπεδο, ονομάζεται διάνυσμα διαχωρισμού).



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Η οργάνωση των υπολογισμών στις ομοιόμορφες αναδρομικές εξισώσεις

Προσδιορίζοντας τις συνθήκες υπολογισιμότητας



Η δυνατότητα χάραξης ενός **υπερεπιπέδου** που να **διαχωρίζει** τα διανύσματα $-\mathbf{w}_1, -\mathbf{w}_2, \dots, -\mathbf{w}_s$ από το **πρώτο** τεταρτημόριο **η** **διαστάσεων** (δείτε την **διακεκομμένη** γραμμή στην **αριστερή** εικόνα) επιτρέπει τον χαρακτηρισμό της συνάρτησης και κατ' επέκταση της σχετιζόμενης **ομοιόμορφης αναδρομικής εξίσωσης**, ως **υπολογίσιμη**. Αυτό είναι κάτι που συμβαίνει στην **αριστερή** εικόνα, όχι όμως και στην **δεξιά** εικόνα

Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Η οργάνωση των υπολογισμών στις ομοιόμορφες αναδρομικές εξισώσεις

Προσδιορίζοντας τις συνθήκες υπολογισιμότητας

Στην περίπτωση ενός **συστήματος εξισώσεων** ο προσδιορισμός των **συνθηκών** που επιτρέπουν τον **χαρακτηρισμό** του συστήματος ως **υπολογίσιμο**, είναι **πιο περίπλοκος** και περιλαμβάνει την **επαναληπτική αποσύνθεση** του **γράφου εξάρτησης** στις **συνιστώσες** του, καθώς και την **επίλυση γραμμικών προγραμμάτων** σε κάθε βήμα της διαδικασίας.

Η διερεύνηση του **παραλληλισμού** είναι και αυτή πιο περίπλοκη, αφού αν και για την περίπτωση μιας απλής εξίσωσης ο παραλληλισμός για $n \geq 2$ είναι **υποχρεωτικά** μη φραγμένος, ωστόσο, υπάρχουν **συστήματα** τέτοιων εξισώσεων, που για τις **ίδιες ακριβώς συνθήκες** χαρακτηρίζονται από **φραγμένο** παραλληλισμό.

Ένα σύστημα **ομοιόμορφων αναδρομικών εξισώσεων** χαρακτηρίζεται ως **υπολογίσιμο**, αν και μόνο αν ο γράφος εξάρτησης **δεν περιέχει** κύκλους. Υποθέτουμε πως ο χώρος R είναι **πεπερασμένος**, αλλά **αρκετά** μεγάλος, έτσι ώστε να μπορέσουμε να **ισχυριστούμε** πως το σύστημα είναι **υπολογίσιμο** αν και μόνο αν ο γράφος G **δεν διαθέτει** κύκλους με **μηδενικό** βάρος.

Η διαδικασία στηρίζεται και πάλι στην **αναζήτηση** επί του γράφου **μη θετικών κύκλων**, η οποία ανάγεται στην **αναζήτηση κύκλων μηδενικού βάρους**. Στην πραγματικότητα, δεν αναζητούμε **απλούς κύκλους** μηδενικού βάρους, αλλά αντίθετα, **πολλαπλούς κύκλους** μηδενικού βάρους, με έναν **πολλαπλό κύκλο** να ορίζεται ως **η ένωση ενός συνόλου απλών κύκλων** οι οποίοι **δεν είναι** υποχρεωτικά **συνδεδεμένοι** μεταξύ τους.



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Η οργάνωση των υπολογισμών στις ομοιόμορφες αναδρομικές εξισώσεις

Προσδιορίζοντας τις συνθήκες υπολογισιμότητας

Βασικό ρόλο στον αλγόριθμο αποσύνθεσης των Karp, Miller & Winograd παίζει μία δομή γράφου G' , ο οποίος ορίζεται ως ένας υπο-γράφος του γράφου G , οι ακμές e_i του οποίου αποτελούν ταυτόχρονα και ακμές του γράφου G , αν και μόνο αν το σύστημα των γραμμικών ανισοτήτων

$$\text{III} \quad \begin{cases} u_i \geq 0 & 1 \leq i \leq m \\ \sum_{i=1}^m u_i > 0 \\ \sum_{i \in I_k(G)} u_i - \sum_{i \in O_k(G)} u_i = 0 & 1 \leq k \leq s \\ \sum_{i=1}^m (u_i w_i)_j \leq -1 & 1 \leq j \leq n \end{cases}$$

διαθέτει εφικτή λύση, στην οποία είναι $u_i > 0$. Στην παραπάνω έκφραση, τα σύνολα $I_k(G)$ και $O_k(G)$ είναι τα σύνολα των εισερχόμενων και εξερχόμενων ακμών που σχετίζονται με την κορυφή u_k αντίστοιχα.



Κεφάλαιο 2: Ζητήματα Παράλληλου Προγραμματισμού

Η οργάνωση των υπολογισμών στις ομοιόμορφες αναδρομικές εξισώσεις

Προσδιορίζοντας τις συνθήκες υπολογισιμότητας

Αυτός ο υπο-γράφος G' χαρακτηρίζεται από τις επόμενες ιδιότητες:

- Ο γράφος G περιέχει έναν κύκλο μηδενικού βάρους αν και μόνο αν το ίδιο ισχύει και για τον υπο-γράφο G' .
- Ο γράφος G' προκύπτει από την ένωση ξένων μεταξύ τους ισχυρά συνδεδεμένων γράφων.
- Εάν ο γράφος G' είναι ένας ισχυρά συνδεδεμένος γράφος, τότε ο γράφος G περιέχει έναν κύκλο μηδενικού βάρους.

Η διαδικασία αποσύνθεσης γράφου των Karp, Miller & Winograd (KMW) διατυπωμένη ως αλγόριθμος, υλοποιείται από τη συνάρτηση $KMW(G)$ όπου G είναι ένας γράφος εξάρτησης και επιστρέφει τιμή **TRUE** ή **FALSE**, με κριτήριο το εάν το σύστημα των ομοιόμορφων αναδρομικών εξισώσεων που περιγράφεται από τον γράφο G , είναι υπολογίσιμο ή όχι. Ο ψευδοκώδικας αυτής της συνάρτησης που υλοποιεί την παραπάνω διαδικασία, ακολουθεί στη συνέχεια

Boolean KMW (Graph G)

Build G' , the subgraph of zero-weight multi-cycles of G .

Compute $G_1', G_2, \dots, G_s'$, namely the s strong connected components of G'

IF $s = 0$ then G' is empty $\implies$ return TRUE

IF $s = 1$ then G' is strongly connected $\implies$ return FALSE

Otherwise return the logical AND of all the function calls $KMW(G_i')$





Το παρόν συνοδευτικό έργο **παρέχεται** αποκλειστικά και μόνο στους διδάσκοντες που έχουν επιλέξει το αντίστοιχο βιβλίο μέσω του συστήματος του **Ευδόξου** ως διδακτικό σύγγραμμα για τη διδασκαλία των μαθημάτων τους και την αξιολόγηση της εκμάθησης των φοιτητών και για όσο χρονικό διάστημα διατηρείται η επιλογή του συγκεκριμένου συγγράμματος. Η **διάδοση, δημοσίευση, αναπαραγωγή ή πώληση** οπουδήποτε μέρους αυτού του έργου με οποιοδήποτε μέσο καταστρέφει την ακεραιότητα του έργου **και για σκοπούς διαφορετικούς από αυτούς για τους οποίους έχει χορηγηθεί η σχετική άδεια δεν επιτρέπεται**. Το περιεχόμενο του έργου **δεν θα πρέπει να διατίθεται** στους φοιτητές παρά μόνο όταν αυτό αναφέρεται ρητά ότι μπορεί να γίνει. Η **χρήση** του παρόντος έργου γίνεται αποκλειστικά από τον διδάσκοντα στα πλαίσια των εκπαιδευτικών καθηκόντων του και με τη χρήση των μέσων που αυτός διαχειρίζεται και έχει στη διάθεσή του από τις **επίσημες υπηρεσίες του εκπαιδευτικού ιδρύματος** όπου ανήκει, διασφαλίζοντας τη μη περαιτέρω διάδοση, δημοσίευση και αναπαραγωγή του έργου προς τρίτους. Ο διδάσκων δύναται να **προσαρμόζει** μέρος του υλικού των διαφανειών σύμφωνα με τις διδακτικές του ανάγκες, αναφερόμενος όμως πάντοτε στην αρχική πηγή αναφοράς. Το παρόν έργο προστατεύεται από την ελληνική και ευρωπαϊκή νομοθεσία περί πνευματικής ιδιοκτησίας.



Επιστημονικές Εκδόσεις

T Z I O Λ Α